

Gorgeous Things — Object Variable Protection & Sibling Plugin Licensing Plan

Status: Skeleton Implemented — Phase 1 ThirdParty Module Structure Complete

Author: GitHub Copilot / Nils Bergemann

Date: 2026-02-27

Scope: Object Variable DLL isolation, anti-tamper, **sibling plugin licensing** (GorgeousCore itself is always free), Fab compliance

Licensing Model: GorgeousCore is **never** licensed. The Vault validates licenses for sibling plugins (GorgeousEntertaining, GorgeousEvents, etc.) that inherit from GorgeousCore. Invalid license on **any** sibling plugin triggers cascading death — the Vault clears the entire OV & AR function reference table, killing all dependent plugins.

Table of Contents

1. Goals & Constraints

2. Architecture Overview

3. Phase 1 — DLL Extraction (The Vault)

4. Phase 2 — Proxy Layer

5. Phase 3 — Anti-Tamper

- 5.0 DLL Signing & Signature Verification
- 5.7 Auto Replication Network Security — RCE Prevention

6. Phase 4 — Sibling Plugin Licensing System

- 6.0 Licensing Model — GorgeousCore Is Free
- 6.0.1 Unified .glic File Format — Gorgeous License
- 6.8 Offline License Key
- 6.9 Offline Key — Additional Security Measures
- 6.10 Offline Key — Packaging Support
- 6.11 Auto-Fetched Offline Key (Online Users)
- 6.12 License Priority & Resolution Order
- 6.13 Cascading Death — All-or-Nothing Enforcement
- 6.14 Offline-First License Renewal
- 6.24 License Priority — Updated Resolution Order
- 6.25 Entitlement Expansion — New Purchases While Offline
- 6.26 Team Detection & Per-Seat License Sharing Enforcement

7. Phase 5 — Packaging & Distribution

8. Fab Marketplace Compliance

- 8.4 Epic Nova EULA / Terms — Partial Licensing Exclusion

9. Threat Model & Attack Surface Analysis

- 9.4 "The Gordian Parasite" — Bidirectional Entangled Semantic Challenge-Response

10. Open Questions & Risks

11. Implementation Progress

1. Goals & Constraints

- Goals
- **Protect** the Object Variable (OV) implementation — it is the core IP of the entire Gorgeous Things ecosystem.
 - **Prevent reverse engineering** of the OV logic by shipping it as a pre-compiled, obfuscated, tamper-resistant native DLL (the "Vault").
 - **Keep GorgeousCore permanently free and open source** — GorgeousCore itself is **never** licensed. The public headers, proxy layer, and all non-OV code remain readable source. Anyone can use GorgeousCore without a license.
 - **License sibling plugins, not Core** — the Vault validates licenses for sibling plugins that inherit from GorgeousCore (e.g., GorgeousEntertaining, GorgeousEvents). Each sibling plugin requires its own license tied to a project ID.
 - **Enforce all-or-nothing ("cascading death")** — if **any** sibling plugin has an invalid or expired license, the Vault clears the entire OV & AR function reference table, disabling OV/AR functionality for **all** plugins (including those with valid licenses). This prevents partial piracy.
 - **Prevent Auto Replication hijacking** — because the Vault contains the entire Auto Replication network transport layer, a compromised Vault DLL could inject arbitrary code over the network (Remote Code Execution). The Vault's integrity measures (DLL signing, runtime self-check, challenge-response) exist not only for IP protection and licensing but also as a **critical network security boundary**.
 - **Comply with Fab marketplace guidelines** — no hidden telemetry, no data collection without consent, no silent network calls.

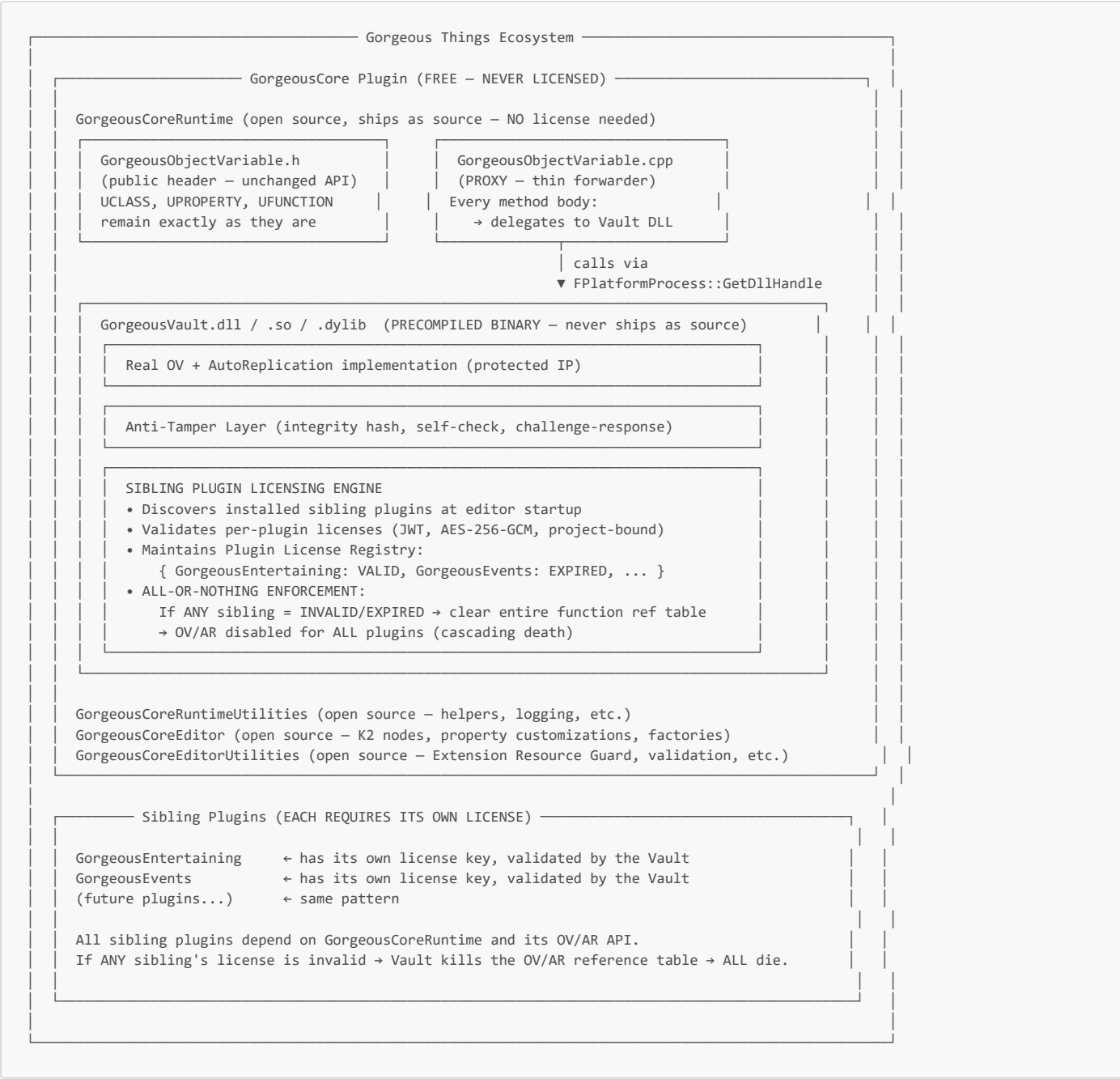
Hard Constraints

Constraint	Reason
UE 5.4+ compatibility	Minimum supported engine version
Must work with Hot Reload / Live Coding	Standard UE dev workflow
Must not require internet for gameplay	Fab TOS + end-user experience
Must not break Blueprint users	OV is heavily BP-exposed
Must work on Win64, Linux, Mac (arm64)	Fab requires all desktop platforms for code plugins

Constraint	Reason
Open-source core headers must remain <code>GORGEOUSCORE_RUNTIME_API</code> exported	Sibling plugins compile against them
GorgeousCore is always free	No license required to use Core itself — licensing applies only to sibling plugins
Cascading death on license violation	One invalid sibling plugin license kills OV/AR for all plugins — social/economic pressure to keep all licenses valid

2. Architecture Overview

Key principle: GorgeousCore is **free and unlicensed**. The Vault protects the OV/AR implementation IP. Licensing is enforced on **sibling plugins** (GorgeousEntertaining, GorgeousEvents, etc.) that depend on GorgeousCore. If any sibling plugin fails its license check, the Vault clears the entire function reference table, killing OV/AR for **every** plugin.



3. Phase 1 — DLL Extraction (The Vault)

3.1 What Goes Into the Vault

File	Lines	Reason
<code>GorgeousObjectVariable.cpp</code>	3066	Core OV logic — the crown jewel
<code>GorgeousRootObjectVariable.cpp</code>	~300	Root OV — directly depends on OV internals

File	Lines	Reason
GorgeousObjectVariableTrunk.cpp	~200	Trunk logic — tree structure management
GorgeousObjectVariableRegistry_GIS.cpp	~150	Registry subsystem — OV lifecycle
NativeObjectVariableDefinitions.cpp	~400	Native type definitions — type system core
GorgeousObjectVariableCmdletHandler.cpp	~100	Cmdlet access to OV — can expose internals
AutoReplication (entire subsystem)		
GorgeousAutoReplicationMixin.cpp	1104	Core mixin — orchestrates replication on any actor
GorgeousAutoReplicationRPCTransporter.cpp	505	RPC transport layer — serializes/deserializes payloads
GorgeousAutoReplicationCoordinator.cpp	415	Coordination logic — decides what/when to replicate
GorgeousAutoReplicationRPCRequestAsyncAction.cpp	379	Async BP action for RPC requests
GorgeousAutoReplicationIrisBackend.cpp	241	Iris replication backend
GorgeousAutoReplicationRPCRelayComponent.cpp	196	Relay component — routes RPCs between actors
GorgeousAutoReplicationGraph.cpp	193	Custom UReplicationGraph implementation
GorgeousAutoReplicationStatsCollector.cpp	176	InsightMatrix stats collection
GorgeousReplicationBeacon.cpp	153	Online beacon client/host for replication handshake
GorgeousAutoReplicationRPCPayloadLibrary.cpp	98	BP library — payload construction helpers
GorgeousAutoReplicationNetworkingTypes.cpp	71	Network type serialization
GorgeousAutoReplicationTypes.cpp	71	Core type definitions
GorgeousAutoReplicationNetworkingLibrary.cpp	57	BP library — networking helpers
GorgeousAutoReplicationWorldSubsystem.cpp	48	World subsystem — per-world replication state
GorgeousAutoReplicationRPCExamples.cpp	30	Example RPC implementations
GorgeousAutoReplicationSettings.cpp	25	Developer settings loader
Total: ~7,978 lines in Vault		

3.2 What Stays Outside (Open Source)

- **All .h headers** — the public API surface (UCLASS, UPROPERTY, UFUNCTION declarations)
- **All interface headers** (*_I.h) — pure virtual, no implementation
- **All macro headers** (GorgeousObjectVariableHelperMacros.h, GorgeousAutoReplicationHelperMacros.h) — preprocessor only
- **All enum/struct headers** — data types only
- **All AutoReplication .h headers** — GorgeousAutoReplicationCoordinator.h, GorgeousAutoReplicationGraph.h, etc.
- **Editor code** (K2 nodes, property customizations, factories) — these call the public API
- **Unit tests** — they test the public API through the proxy

3.3 Vault DLL Build System

The Vault is built as a **standalone CMake project** (not an UBT module), producing a plain C++ shared library. This is critical because:

1. **UBT modules ship as source on Fab** — a standalone DLL does not.
2. **Obfuscation toolchains** (LLVM passes, Themida, VMProtect) integrate trivially with CMake.
3. **The DLL has no UE module descriptor** — UBT cannot discover or rebuild it.

```
Plugins/GorgeousCore/
├── Source/
│   ├── GorgeousCoreRuntime/          ← open source module (ships as source)
│   │   ├── Public/ObjectVariables/   ← headers unchanged
│   │   └── Private/SourceFiles/ObjectVariables/
│   │       └── GorgeousObjectVariable.cpp ← NOW A PROXY (see Phase 2)
├── Vault/
│   ├── CMakeLists.txt                ← standalone build
│   └── Source/                       ← REAL implementations (NEVER distributed as source)
│       ├── ObjectVariables/
│       │   ├── GorgeousObjectVariable_Impl.cpp
│       │   ├── GorgeousRootObjectVariable_Impl.cpp
│       │   ├── GorgeousObjectVariableTrunk_Impl.cpp
│       │   ├── NativeObjectVariableDefinitions_Impl.cpp
│       │   ├── GorgeousObjectVariableRegistry_GIS_Impl.cpp
│       │   └── GorgeousObjectVariableCmdletHandler_Impl.cpp
│       └── AutoReplication/
│           ├── GorgeousAutoReplicationMixin_Impl.cpp
│           ├── GorgeousAutoReplicationCoordinator_Impl.cpp
│           └── GorgeousAutoReplicationGraph_Impl.cpp
```

```

├── GorgeousAutoReplicationIrisBackend_Impl.cpp
├── GorgeousAutoReplicationRPCTransporter_Impl.cpp
├── GorgeousAutoReplicationRPCRelayComponent_Impl.cpp
├── GorgeousAutoReplicationRPCRequestAsyncAction_Impl.cpp
├── GorgeousAutoReplicationWorldSubsystem_Impl.cpp
├── GorgeousAutoReplicationNetworkingTypes_Impl.cpp
├── GorgeousAutoReplicationTypes_Impl.cpp
├── GorgeousAutoReplicationStatsCollector_Impl.cpp
├── GorgeousReplicationBeacon_Impl.cpp
├── BlueprintFunctionLibraries/
│   ├── GorgeousAutoReplicationNetworkingLibrary_Impl.cpp
│   ├── GorgeousAutoReplicationRPCExamples_Impl.cpp
│   └── GorgeousAutoReplicationRPCPayloadLibrary_Impl.cpp
├── VaultAntiTamper.cpp      ← integrity checks
├── VaultLicense.cpp        ← license validation
├── Include/
│   └── GorgeousVaultAPI.h  ← C-linkage export table
├── Prebuilt/
│   ├── Win64/GorgeousVault.dll
│   ├── Linux/libGorgeousVault.so
│   └── Mac/libGorgeousVault.dylib
└── GorgeousCore.uplugin

```

3.4 Vault Export Surface (C-Linkage)

The DLL exports a **flat C function table** — no C++ name mangling, no vtables crossing the boundary.

```

// GorgeousVaultAPI.h – ships with the open-source headers
#pragma once

// Opaque handle – the proxy layer passes `this` as a void*
// The Vault casts it back to UGorgeousObjectVariable* internally
// (it links against the same UE headers at build time)
typedef void* GVaultOVHandle;

extern "C"
{
    // Lifecycle
    VAULT_API bool GVault_Initialize(GVaultOVHandle Self);
    VAULT_API void GVault_BeginDestroy(GVaultOVHandle Self);

    // Serialization
    VAULT_API bool GVault_SerializeToPayload(GVaultOVHandle Self, /* params */);
    VAULT_API bool GVault_DeserializeFromPayload(GVaultOVHandle Self, /* params */);

    // Replication (OV-level)
    VAULT_API void GVault_ActivateReplication(GVaultOVHandle Self, /* params */);
    VAULT_API void GVault_RequestAutoReplicationRPC(GVaultOVHandle Self, /* params */);

    // Dynamic Properties
    VAULT_API bool GVault_SetDynamicPropertyRaw(GVaultOVHandle Self, /* params */);
    VAULT_API bool GVault_GetDynamicPropertyRaw(GVaultOVHandle Self, /* params */);

    // ... one entry per significant OV method
    // Trivial getters/setters can stay inline in the header or proxy .cpp

    // — AutoReplication subsystem —————
    typedef void* GVaultHandle; // generic opaque handle

    // Mixin
    VAULT_API void GVault_Mixin_Initialize(GVaultHandle Self, /* params */);
    VAULT_API void GVault_Mixin_Tick(GVaultHandle Self, float DeltaTime);
    VAULT_API void GVault_Mixin_RegisterVariable(GVaultHandle Self, /* params */);
    VAULT_API void GVault_Mixin_UnregisterVariable(GVaultHandle Self, /* params */);

    // Coordinator
    VAULT_API void GVault_Coordinator_Evaluate(GVaultHandle Self, /* params */);
    VAULT_API void GVault_Coordinator_Flush(GVaultHandle Self);

    // Graph
    VAULT_API void GVault_Graph_InitNode(GVaultHandle Self, /* params */);
    VAULT_API void GVault_Graph_RoutePacket(GVaultHandle Self, /* params */);

    // RPC Transporter
    VAULT_API void GVault_RPCTransporter_Send(GVaultHandle Self, /* params */);
    VAULT_API void GVault_RPCTransporter_Receive(GVaultHandle Self, /* params */);

    // Iris Backend

```

```

VAULT_API void GVault_Iris_Connect(GVaultHandle Self, /* params */);
VAULT_API void GVault_Iris_Sync(GVaultHandle Self, /* params */);

// WorldSubsystem
VAULT_API void GVault_WorldSub_Initialize(GVaultHandle Self, /* params */);
VAULT_API void GVault_WorldSub_Deinitialize(GVaultHandle Self);

// ... remaining AutoReplication entries follow the same pattern

// — DLL Signature Verification —————
VAULT_API int32 GVault_ValidateSignature(); // returns 0 on success, non-zero = unsigned/invalid (FIRST gate)

// — Anti-tamper / Sibling Plugin Licensing —————
VAULT_API int32 GVault_ValidateIntegrity(); // returns 0 on success
VAULT_API int32 GVault_DiscoverSiblingPlugins(const char* PluginsDir);
VAULT_API int32 GVault_ValidatePluginLicense(const char* EncryptedLicPath, const char* ProjectId, const char* PluginName);
VAULT_API int32 GVault_ValidatePluginOfflineLicense(const char* OfflineLicPath, const char* ProjectId, const char*
PluginName);
VAULT_API int32 GVault_GetPluginLicenseRegistry(/* out params */); // returns per-plugin license status map

// — "The Gordian Parasite v2" Computation-Entangled Verification —————
VAULT_API int32 GVault_ChallengeResponse(const uint8* Challenge, int32 ChallengeLen,
uint8* ResponseOut, int32* ResponseLenOut);

// NOTE: GVault_Initialize (above) now returns Accumulator_0 as part of
// the Gordian Parasite v2 protocol. Every subsequent GVault_OV_* and
// GVault_AR_* export gains (AccumulatorIn, AccumulatorOut) parameters.
// See §9.4.8 for updated export signatures.

// AR Multiplayer Integrity (Pillar 3)
VAULT_API int32 GVault_AR_TagPacket(const uint8* Payload, int32 PayloadLen,
uint8* TagOut, int32* TagLenOut,
const uint8* AccumulatorIn,
uint8* AccumulatorOut);
VAULT_API int32 GVault_AR_VerifyPacket(const uint8* Payload, int32 PayloadLen,
const uint8* Tag, int32 TagLen,
const uint8* AccumulatorIn,
uint8* AccumulatorOut);

// Periodic Integrity Audit
VAULT_API int32 GVault_PeriodicAudit(const uint8* AccumulatorIn,
uint8* AccumulatorOut);
}

```

3.5 How the Vault Calls Back Into UE

The Vault **links against the same UE import libraries** at build time (Core, CoreUObject, Engine). This means inside the Vault DLL, code can freely:

- Call `UObject::GetWorld()`, `GetOuter()`, `FindFunctionByName()`
- Use `FProperty`, `FStructProperty`, UE containers (`TArray`, `TMap`)
- Access `GEngine`, `GWorld`, `GetLifetimeReplicatedProps()`
- Fire delegates, broadcast events

This is possible because UE modules are themselves DLLs — the Vault links against them just like any other module would. The key is that the Vault's **CMakeLists.txt** specifies the UE import libs as link dependencies.

4. Phase 2 — Proxy Layer

4.1 Proxy Pattern

`GorgeousObjectVariable.cpp` becomes a thin proxy. Every method body is replaced with:

```

void UGorgeousObjectVariable::SomeMethod(FParams Params)
{
    // The proxy loads the Vault DLL once (singleton) and caches function pointers
    static auto Fn = GorgeousVaultLoader::Get().Resolve<decltype(&GVault_SomeMethod)>("GVault_SomeMethod");
    if (Fn)
    {
        Fn(this, Params);
    }
    else
    {
        // Vault not loaded — graceful degradation or fatal
        UE_LOG(LogGorgeousCore, Fatal, TEXT("GorgeousVault.dll is missing or corrupted. Cannot proceed."));
    }
}

```

4.2 Vault Loader (Singleton)

```
// GorgeousVaultLoader.h — open source, ships with the proxy
class GorgeousVaultLoader
{
public:
    static GorgeousVaultLoader& Get();

    bool IsLoaded() const;

    template<typename FuncPtr>
    FuncPtr Resolve(const char* Name);

private:
    void* DllHandle = nullptr;
    bool bSignatureVerified = false; // $5.0 — set by VerifySignature()
    bool bIntegrityVerified = false;

    GorgeousVaultLoader();
    ~GorgeousVaultLoader();

    void LoadVault(); // FPlatformProcess::GetDllHandle
    void VerifySignature(); // Calls GVault_ValidateSignature (FIRST — $5.0)
    void VerifyIntegrity(); // Calls GVault_ValidateIntegrity (SECOND — $5.2)
};
```

4.3 What Stays Native in the Proxy

Some things **cannot** cross a DLL boundary via C-linkage:

Item	Strategy
GENERATED_BODY() / UHT-generated code	Stays in proxy — UHT generates into the module that owns the UCLASS
GetLifetimeReplicatedProps()	Stays in proxy — calls into Vault for the property list
StaticClass() / CDO construction	Stays in proxy — UObject system requires this in the owning module
Blueprint-callable UFUNCTION thunks	Generated by UHT, stay in proxy. The thunk calls the proxy method which forwards to Vault
PostInitProperties() / PostLoad()	Proxy overrides, forwarding to Vault

5. Phase 3 — Anti-Tamper

5.0 DLL Signing & Signature Verification

This is the VERY FIRST security gate. Before any function pointers are resolved, before any license is checked, before any OV or Auto Replication code runs — the Vault DLL validates its own digital signature. If it fails, ALL work is rejected.

Why DLL Signing?

Code signing provides a **cryptographic chain of trust** from Simsalabim Studios (the publisher) to the binary running on the user's machine. Unlike the `.text` hash (\$5.1), which only detects post-build modification, the digital signature proves:

- 1. **Authenticity** — the DLL was built and signed by Simsalabim Studios, not a third party.
- 2. **Integrity** — the binary has not been modified since signing.
- 3. **Non-repudiation** — the signature is tied to a code-signing certificate that can be verified against a public CA.

Signing Process (Build Pipeline)

```
[Post-Build Signing Script]
1. Build GorgeousVault.dll / .so / .dylib
2. Compute .text section hash → embed in .gvault_meta ($5.1)
3. Sign the ENTIRE binary with Simsalabim Studios' code-signing certificate:
  • Windows: signtool.exe with EV code-signing certificate (Authenticode)
  • Linux:   GPG detached signature (.sig file) + embedded ELF note
  • macOS:  codesign with Apple Developer ID (or ad-hoc for non-notarized builds)
4. Ship the signed binary + platform-appropriate signature metadata
```

Platform	Signing Method	Verification API
Windows	Authenticode (SHA-256, EV certificate)	WinVerifyTrust() API
Linux	GPG detached signature + embedded ELF <code>.note.gvault_sig</code> section	libgcrypt / in-DLL verification against baked-in public key

Platform	Signing Method	Verification API
macOS	codesign with Developer ID	SecStaticCodeCheckValidity()

Self-Verification on Load (First Gate)

```
GVault_ValidateSignature() – called BEFORE GVault_ValidateIntegrity()

1. Determine platform:
  • Windows: Call WinVerifyTrust() on own DLL path
    → Verifies Authenticode signature chain up to trusted root CA
    → Verifies the signer is "Simsalabim Studios" (subject name check)
  • Linux: Read embedded .note.gvault_sig section
    → Verify GPG signature against baked-in Simsalabim public key
  • macOS: Call SecStaticCodeCheckValidity() on own bundle path
    → Verify code signature + team identifier

2. IF signature is VALID:
  → Set internal flag: bSignatureVerified = true
  → Proceed to §5.1 / §5.2 integrity checks
  → Log (debug only): "Vault signature verified."

3. IF signature is INVALID or MISSING:
  → bSignatureVerified = false
  → IMMEDIATELY reject ALL work:
    a. Zero all export function pointers
    b. Set all OV/AR function tables to null
    c. Refuse to respond to challenge-response (§9.4)
    d. Log ERROR: "Vault DLL signature verification failed.
       The binary may have been tampered with.
       OV/AR functionality is disabled."
  → DO NOT proceed to any further initialization
  → DO NOT trigger self-destruct (the DLL might be a legitimate
     development build – self-destruct would be destructive)

4. The VaultLoader checks bSignatureVerified before resolving ANY
   function pointer. If false → LoadVault() returns failure.
```

Development builds: During development, the Vault is built locally without a production certificate. The CMake build defines `GVault_DEV_UNSIGNED=1`, which disables signature verification in Debug/DebugGame configurations ONLY. Development and Shipping builds ALWAYS enforce signature verification.

Integration with Existing Security Layers

DLL signing becomes **Layer 0** in the defense-in-depth chain — it runs before everything else:

```
Vault Load Sequence (updated):
1. DllHandle = FPlatformProcess::GetDllHandle("GorgeousVault")
2. GVault_ValidateSignature()      ← NEW: Layer 0 – DLL signing
3. GVault_ValidateIntegrity()     ← Layer 4 – .text hash self-check
4. Challenge-Response (§9.4)      ← Layer 8 – Proxy verifies DLL is genuine
5. GVault_DiscoverSiblingPlugins() ← License evaluation begins
```

If step 2 fails, steps 3–5 never execute.

New C-Linkage Export

```
// — DLL Signature Verification —————
VAULT_API int32 GVault_ValidateSignature(); // returns 0 on success, non-zero = unsigned/invalid
```

5.1 Compile-Time Integrity Hash

During the Vault build, a **post-link step** computes a SHA-256 hash of the `.text` (code) section and embeds it into a reserved `.gvault_meta` section of the binary.

```
[Post-Link Script]
1. Build GorgeousVault.dll
2. Read the .text section bytes
3. Compute SHA-256
4. Write the hash into the .gvault_meta section (reserved placeholder)
5. Re-sign the binary (code-signing certificate – see §5.0)
```

5.2 Runtime Self-Check

On DLL load (`GVault_ValidateIntegrity`):

```
1. Map own binary into memory (read-only)
2. Locate .text section boundaries from PE/ELF/Mach-O headers
3. Compute SHA-256 of .text in-memory
4. Compare against the embedded hash in .gvault_meta
5. If mismatch → the DLL was patched
   → Zero out all export function pointers
   → Overwrite the .text section with random bytes (self-destruct)
   → Return error code
```

Known limitation — the "chicken-and-egg" problem: An attacker who patches the `.text` section can ALSO patch the expected hash in `.gvault_meta`, making the self-check pass. The self-check alone is therefore a **speed bump, not a wall**.

Why it doesn't matter: The real cryptographic bite comes from two other mechanisms that use the `.text` hash as a KEY, not just a CHECK:

- Triple-envelope Layer 2 (\$6.15):** `Vault_Key = HKDF(VaultTextHash, ...)`. A patched DLL has a different `.text` hash → different `Vault_Key` → cannot decrypt Layer 2 of the license → no master_seed → no EpochKey → all LDCCs are wrong (\$9.4.2) → silent corruption.
- Opaque Accumulator (\$9.4.3):** The accumulator HMAC key includes `VaultTextHash`. A patched DLL produces a different accumulator chain → The Rot triggers.

The attacker must defeat BOTH of these cryptographic consequences simultaneously — patching `.gvault_meta` doesn't help because the `.text` hash is used as a derivation input, not just compared against a stored value.

5.3 Periodic Re-Check

A background thread inside the Vault runs every 30–60 seconds:

- 1. Re-hashes the `.text` section
- 2. If the hash changed since load (memory patching / debugger breakpoints in some forms):
 - Silently corrupt internal state (delayed death — harder to trace)
 - After a random delay (1–5 seconds), zero the export table

5.4 Anti-Debug Measures

Technique	Platform	Notes
<code>IsDebuggerPresent()</code> check on load	Windows	Can be bypassed but raises the bar
<code>ptrace(PTRACE_TRACEME)</code>	Linux	Self-trace to prevent attach
Check for timing anomalies	All	Single-step debugging causes measurable delays
Hardware breakpoint detection (DR registers)	Windows	Read debug registers; if set, trigger self-destruct
IMPORTANT: All anti-debug is disabled in Editor builds		All Only active in Shipping/Test configurations — developers need debuggers

5.5 Obfuscation

The Vault CMake build uses **LLVM obfuscation passes** (e.g., [Hikari](#) or commercial equivalent):

- **Control flow flattening** — hides the real execution order
- **Bogus control flow** — inserts dead code paths
- **String encryption** — all string literals are decrypted at runtime
- **Instruction substitution** — replaces simple operations with equivalent complex ones
- **MBA (Mixed Boolean-Arithmetic) expressions** — makes arithmetic unreadable in disassembly

5.5.1 Code Virtualization (Critical Paths)

Security assessment finding: LLVM obfuscation alone is a speed bump — Hikari-class passes are routinely defeated by symbolic execution frameworks (Triton, Miasm) in 1-4 weeks. The following critical code paths MUST additionally be protected with **genuine code virtualization** (Themida VM / VMProtect / equivalent) which converts native x86/x64 instructions into a custom bytecode ISA executed by an embedded interpreter. This raises the bar from "weeks" to "months" for a skilled reverse engineer.

Paths requiring VM protection (mandatory):

Critical Path	Why It Must Be VM-Protected
White-Box AES table access (\$6.5)	The SharedSecret/HKDF derivation — if exposed, all .glic files can be forged
LDCC derivation from EpochKey (\$9.4.2)	If an attacker can read LDCC values at derivation time, they can hardcode them
Accumulator HMAC verification (\$9.4.3)	The HMAC key comparison and rot-seed planting must not be traceable
Triple-envelope decryption (\$6.15)	master_seed decryption is the single highest-value target

Critical Path	Why It Must Be VM-Protected
The Rot seed computation (§9.4.5)	If the attacker finds the seed generation, they can predict/bypass corruption
Proxy .text hash comparison (§9.4.6)	Expected hash values must not be extractable via static analysis

NOT VM-protected (performance-sensitive hot paths):

- OV getter/setter fast path — called potentially thousands of times per frame
- AR packet serialization body (only the HMAC tagging is VM-protected)
- UObject callback dispatch

Implementation note: VM protection is applied as a post-link step, same as the .text hash computation (§5.1). The VM protector runs AFTER LLVM obfuscation, so the virtualized code is doubly protected (obfuscated bytecode inside a custom VM). The .text hash in .gvault_meta is computed AFTER VM protection is applied.

5.6 Self-Destruct Mechanism

If tampering is detected:

```
Phase 1 (immediate): Zero all function pointers in the export table
Phase 2 (1s delay):  Overwrite the .text section with random bytes
Phase 3 (2s delay):  Rename the DLL file on disk to a random name (prevents simple "restore original")
Phase 4:             Write a poison marker file (.gvault_revoked) next to the DLL
                    On next load, if .gvault_revoked exists → refuse to initialize entirely
```

Critical Note: The self-destruct must NEVER corrupt user project data. Only the Vault DLL itself is destroyed. Worst case: the OV system stops working and the user must re-download the plugin.

5.7 Auto Replication Network Security — RCE Prevention

CRITICAL SECURITY CONCERN: The Vault DLL contains the **entire Auto Replication (AR) network transport layer** — the code that serializes, deserializes, and routes replicated variable updates over the network (RPCs, multicast, etc.). A hijacked or tampered Vault DLL could be used to **inject arbitrary payloads into the replication stream**, potentially achieving **Remote Code Execution (RCE)** on connected clients or the server.

Why This Matters

Auto Replication is inherently dangerous from a security perspective:

Risk	Description
Arbitrary payload injection	The AR system serializes Object Variables into network payloads. A compromised Vault could inject crafted payloads that exploit deserialization on the receiving end.
RPC routing manipulation	The GorgeousAutoReplicationRPCTransporter routes RPCs between clients/server. A hijacked DLL could redirect, amplify, or fabricate RPCs.
Coordinator spoofing	The GorgeousAutoReplicationCoordinator decides what/when to replicate. A tampered DLL could force replication of sensitive data or trigger replication floods (DoS).
Blueprint execution via replicated state	Object Variables can trigger Blueprint events on state change. Crafted replicated values could cause unintended Blueprint execution chains.

How the Vault Prevents AR Hijacking

The security measures defined throughout Phase 3 (Anti-Tamper) serve a **dual purpose** — they protect both the licensing/IP system AND the network transport layer:

VAULT SECURITY MEASURES — DUAL PURPOSE

PURPOSE 1: IP Protection & Licensing

- Prevent reverse engineering of OV logic
- Enforce sibling plugin licensing
- Protect master_seed and license state

PURPOSE 2: Network Security (Auto Replication)

- Ensure only GENUINE, UNMODIFIED Vault code processes network data
- Prevent injection of arbitrary payloads into replication streams
- Prevent RCE via crafted replicated Object Variable state
- Prevent RPC routing manipulation

SHARED SECURITY GATES (must ALL pass before AR functions are exposed):

- §5.0 DLL Signature Verification (Authenticode / codesign / GPG)
- §5.1 .text Section Hash (compile-time integrity)

\$5.2 Runtime Self-Check (ongoing .text verification)

\$5.3 Periodic Re-Check (background thread)

\$9.4 Challenge-Response (Proxy proves DLL is genuine)

If ANY gate fails → AR function pointers are NULLED

- No replication data is processed by a tampered DLL
- Replication simply stops (safe failure mode)
- Connected clients experience a replication timeout, not an exploit

AR-Specific Security Properties

Property	Enforcement
Payload integrity	All AR payloads are serialized inside the verified Vault. The serialization format is internal and not documented publicly. A tampered DLL cannot produce correctly-formed payloads that pass the deserializer on the receiving end.
Type safety	The Vault validates Object Variable type metadata during deserialization. Type mismatches (e.g., sending an integer where a struct is expected) are rejected with a type-check failure — not blindly deserialized.
Size bounds	Replicated payloads include embedded size limits checked by the Vault before memory allocation. Oversized payloads are truncated and logged.
RPC authentication	The challenge-response protocol (\$9.4) runs not just at load time but is periodically re-validated. An attacker who hot-swaps the DLL mid-session will fail the next challenge cycle, immediately disabling all AR functions.
No raw pointer transfer	The AR serialization format NEVER transmits raw memory addresses across the network. All references use stable identifiers (object paths, registry keys, GUIDs) that are resolved locally.

Failure Mode — AR With a Tampered Vault

If a tampered Vault DLL is detected (any security gate fails):

1. ALL AR function pointers are zeroed in CachedFunctions:

- GVault_AR_Mixin_Bind = nullptr
- GVault_AR_Mixin_RequestRPC = nullptr
- GVault_AR_Coordinator_RegisterVariable = nullptr
- GVault_AR_Coordinator_EvaluateDirtyVariables = nullptr
- GVault_AR_Graph_RegisterObjectVariable = nullptr
- GVault_AR_Graph_RouteReplicationUpdate = nullptr
- GVault_AR_RPCTransporter_RouteRPC = nullptr
- (all AR exports → null)

2. The proxy layer's AR methods become no-ops:

- ActivateReplication() returns immediately
- RequestAutoReplicationRPC() silently drops the request
- No network traffic is generated or accepted

3. Connected peers see: "Replication stream from [Host] stopped."

- This is a SAFE failure – no data is corrupted, no exploit is possible
- The worst outcome is that replicated Object Variables become stale

4. Recovery: Install a genuine Vault DLL → restart editor

- AR resumes normally

The Vault's anti-tamper system is not just about licensing — it is a NETWORK SECURITY BOUNDARY. Without these measures, a hijacked Vault DLL could turn any connected Unreal Engine instance into a vector for arbitrary code execution via the Auto Replication transport.

6. Phase 4 — Sibling Plugin Licensing System

6.0 Licensing Model — GorgeousCore Is Free

GorgeousCore is NEVER licensed. It ships on Fab as a free, open-source plugin. Anyone can download and use it without a license key.

The Vault does **not** gate access to GorgeousCore itself. Instead, it validates licenses for **sibling plugins** — separate Fab products that inherit from and extend GorgeousCore:

Sibling Plugin	Description
GorgeousEntertaining	Gameplay & entertainment systems built on OV/AR
GorgeousEvents	Event-driven architecture extensions on OV
(future plugins)	Same licensing pattern

How it works:

- 11 / 98



Stored at: <ProjectDir>/Saved/GorgeousLicense/GlobalKey.glic

Type: plugin — Per-Plugin License

Each sibling plugin gets **one** .glic file that contains **two slots**: a primary license and an optional offline fallback. This replaces the old pattern of separate .glic and .golic files.

PluginName "fallback")) [OPTIONAL]	
jwt_payload:	Signed JWT (365-day offline key)
expiry:	2027-02-27T00:00:00Z
hw_fingerprint:	AES-encrypted (key = nonce-derived)
nonce:	unique per issuance
allow_package:	true / false
dual_signature:	ECDSA + secondary_key (\$6.9)
offline_marker:	true
INTEGRITY TAG	
HMAC-SHA256(AES_Key, entire_file_bytes_above)	

Stored at: <ProjectDir>/Saved/GorgeousLicense/<PluginName>.glic

Resolution within the file:

1. Read header → verify `type == "plugin"` and `plugin_name` matches
2. Try **PRIMARY SLOT** first → decrypt, validate JWT/HMAC, check expiry
3. If primary is expired/invalid → attempt online refresh → write new primary slot
4. If refresh fails → try **FALLBACK SLOT** → decrypt, validate, check expiry
5. If fallback valid → mark LICENSED (degraded), show toast "Using offline license"
6. If both fail → mark INVALID

Key properties:

- `mode` in the header indicates which model issued the primary slot:
 - `"online"` — server-issued 30-day JWT
 - `"offline"` — server-issued 365-day offline JWT (when user manually requested offline key)
 - `"epoch-derived"` — locally derived from `GlobalKey.glic` (type=master) epoch chain
- The **fallback slot** is auto-populated by the server during the first online validation (\$6.11). It is independently encrypted with a different HKDF info string ("`fallback`" vs "`primary`") so one slot cannot be confused for the other.
- When the primary slot is refreshed (online or via epoch renewal), only the primary slot is overwritten. The fallback slot remains untouched until it naturally expires or is explicitly refreshed.

Type: `patch` — Entitlement Expansion

Lightweight, server-signed delta that adds new plugins to the master key's entitlement list.

Gorgeous License — type: patch (Patches/2026-03-15_GorgeousEvents.glic)	
COMMON HEADER	
• magic:	"GLIC"
• format_version:	4
• type:	"patch"
• issued_at:	2026-03-15T12:00:00Z
• user_id:	"usr_a1b2c3d4..."
PATCH HEADER EXTENSION	
• parent_master_hash:	SHA-256(GlobalKey.glic common + master headers)
• add_plugins:	["GorgeousEvents"]
• effective_plugins:	["GorgeousEntertaining", "GorgeousEvents", ...]
• server_signature:	ECDSA-P256(SHA-256(all_header_fields_above))
(no encrypted body — the patch IS its header + signature)	

Stored at: <ProjectDir>/Saved/GorgeousLicense/Patches/<date>_<PluginName>.glic

Patch files have no encrypted body — all the data is in the signed header. The `server_signature` is the entirety of the security.

File Layout on Disk (Unified)

<ProjectDir>/Saved/GorgeousLicense/	
└─ GlobalKey.glic	← type=master (offline-first only)
└─ GorgeousEntertaining.glic	← type=plugin (primary + fallback slots)
└─ GorgeousEvents.glic	← type=plugin

└─ Patches/
 └─ 2026-03-15_GorgeousEvents.glic ← type=patch

Every file is `.glic`. The Vault reads the `type` field from the common header to determine parsing. Users, documentation, and the License Manager UI all refer to "your `.glic` license file" uniformly.

Dispatch Logic (Vault-Side)

```
// Pseudocode – how the Vault processes any .glic file
GlicHeader header = ReadCommonHeader(filepath);
if (header.magic != "GLIC") → reject

switch (header.type)
{
    case "master":
        ParseMasterExtension(filepath, header);
        // → Triple-envelope decryption, epoch chain, etc.
        break;

    case "plugin":
        ParsePluginExtension(filepath, header);
        // → Primary slot → fallback slot → resolution
        break;

    case "patch":
        ParsePatchExtension(filepath, header);
        // → Verify signature, add to entitlement list
        break;

    default:
        LogWarning("Unknown .glic type: %s", header.type);
        → ignore file
}
```

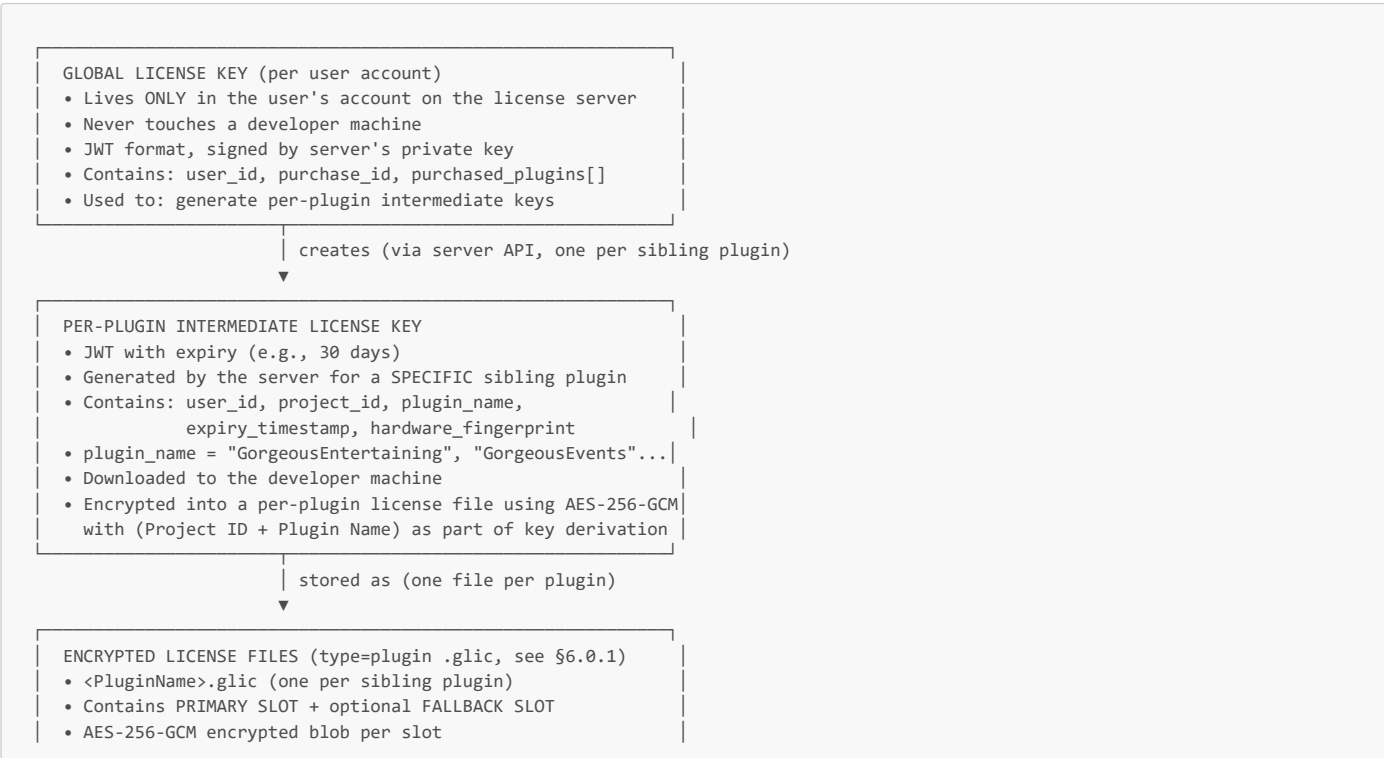
6.1 Key Hierarchy

The licensing system supports **two models** that co-exist. Users can **switch between them at any time**:

- **Online model (\$6.1–6.13)**: Global Key stays on server, intermediate keys are refreshed periodically.
- **Offline-first model (\$6.14–6.25)**: Global Key is given to the user, all renewals happen locally. **Zero server dependency after the one-time key export**. New plugin purchases are handled via entitlement patches (type=patch `.glic` files, \$6.25).

Users start on the online model by default. At **any point** — after purchase, mid-project, years later — they can request their Global License Key via the License Manager UI or the self-service web portal. Once exported, the key is theirs forever. The online model continues to work in parallel, or the user can go fully offline.

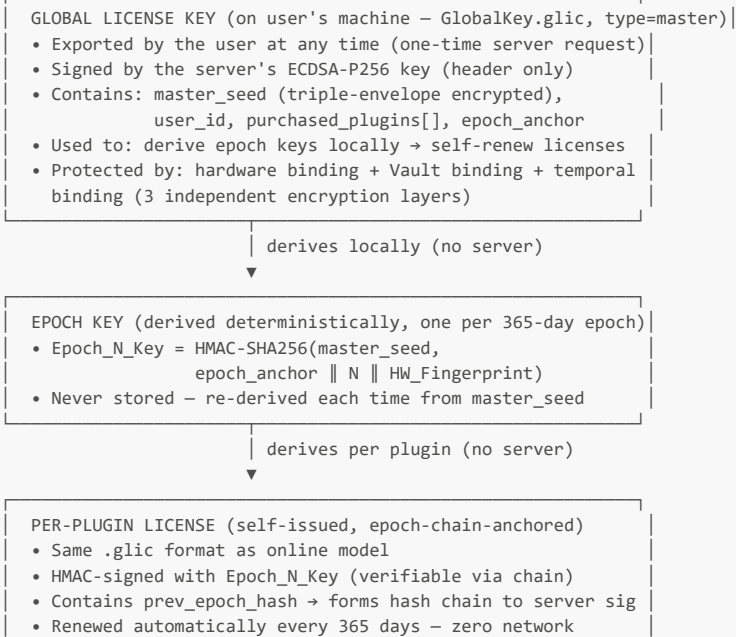
Online Model — Key Hierarchy



- Encryption key = HKDF(SharedSecret, ProjectID || PluginName || "primary" or "fallback")
- Location: <ProjectDir>/Saved/GorgeousLicense/
e.g.: GorgeousEntertaining.glic
GorgeousEvents.glic
- Cannot be copied to another project or used for a different plugin (different derivation → decryption fails)
- The Vault reads ALL license files on load and builds the Plugin License Registry

Note: GorgeousCore has NO license file. It is always functional. Only sibling plugins require license files.

Offline-First Model — Key Hierarchy

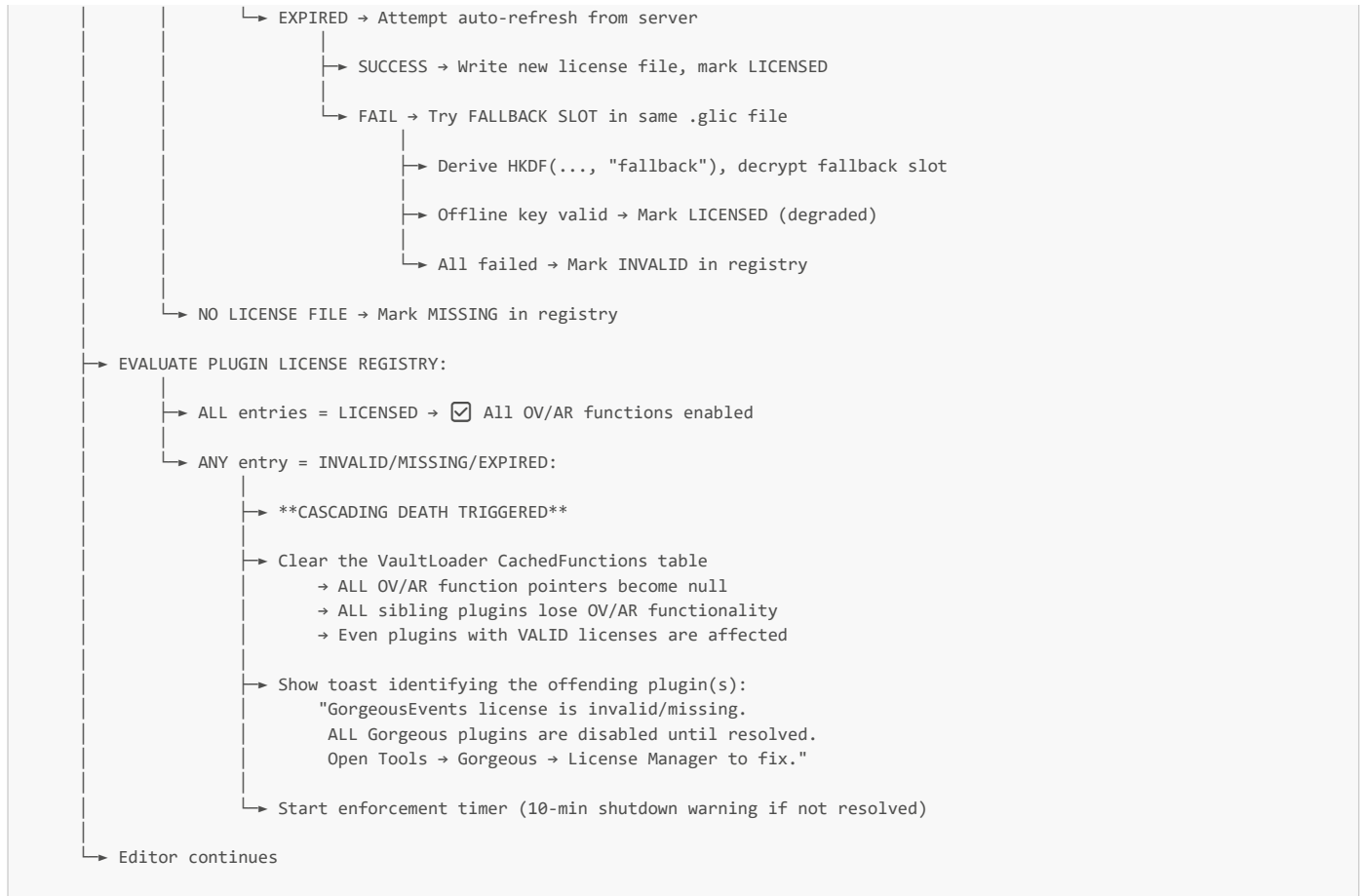


See §6.14–6.25 for the complete offline-first system design (including entitlement expansion for new purchases).

6.2 License Flow

EDITOR STARTUP

- GorgeousVaultLoader loads GorgeousVault.dll
- Vault discovers installed sibling plugins
(scans <Project>/Plugins/ for .uplugin files with GorgeousCore dependency)
Discovered: [GorgeousEntertaining, GorgeousEvents, ...]
- IF no sibling plugins found:
 - GorgeousCore works normally (it's free, no license needed)
 - Skip all license validation
 - Editor continues
- FOR EACH discovered sibling plugin:
 - Look for <PluginName>.glic (type=plugin) in <ProjectDir>/Saved/GorgeousLicense/
 - Verify common header: magic="GLIC", type="plugin", plugin_name matches
 - Derive AES key: HKDF(SharedSecret, ProjectID || PluginName || "primary")
 - Decrypt PRIMARY SLOT → get per-plugin JWT intermediate key
 - Validate JWT signature (server's public key baked into Vault)
 - Check expiry timestamp
 - VALID → Mark plugin as LICENSED in Plugin License Registry



6.3 License Refresh (Background)

- A **ticker** in the Vault (or in the open-source module calling into the Vault) checks license expiry periodically.
- When < 7 days remain: show an unobtrusive toast: "License renewing in background..."
- Refresh is a single HTTPS POST to the license server.
- **No network call is ever made during gameplay / packaged builds** — the license is checked once at editor startup only.

6.4 Packaging Gate

When the user packages a project (via UAT or editor):

1. The Vault checks that the license allows packaging (intermediate key must have `allow_package: true`).
2. The Project ID is sent to the server alongside the intermediate key for a **one-time packaging token**.
3. The server logs this: (`user_id`, `project_id`, `timestamp`, `key_used`) — this is the piracy tracking breadcrumb.
4. If the license is invalid or the server is unreachable, packaging **fails with a clear error message**.

6.4.1 Online Packaging Nonce Check

For users on the **online** license flow, packaging requires a live server round-trip with a single-use nonce. This prevents a captured packaging token from being reused and ensures the server has a real-time record of every package event.




```

    ✓ user has packaging rights
Server responds:
{
  "nonce": "<32-byte random, hex>",
  "nonce_expiry": "2026-02-27T14:37:00Z", // 5-minute window
  "server_signature": "<ECDSA(nonce || project_id || timestamp)>"
}
Server records: (user_id, project_id, nonce, timestamp)

→ Step 2 – Local Nonce Verification
Vault verifies server_signature using baked-in public key
Vault checks nonce_expiry > now
If either fails → CANCEL PACKAGING with error:
  "Packaging authorization failed: invalid server response."

→ Step 3 – Nonce Consumption
POST /api/v1/package/nonce/consume
Body: {
  "nonce": "<the nonce from Step 1>",
  "project_id": "E0C3F3454AA9F4309CCBD6829FE5C95E",
  "proof": HMAC-SHA256(key = nonce, message = project_id || build_config)
}
▼
Server validates:
  ✓ nonce exists and has not been consumed
  ✓ nonce has not expired
  ✓ HMAC proof is correct
Server marks nonce as CONSUMED (one-time use)
Server responds: { "status": "ok" }

If server returns error or is unreachable:
  → CANCEL PACKAGING
  → Show error: "Could not verify packaging authorization with the
    license server. Check your network connection and try again."

→ Step 4 – Packaging Proceeds
Vault stores (nonce, timestamp, build_config) locally in
<ProjectDir>/Saved/GorgeousLicense/.package_receipts
for audit / support purposes

→ Cook & package continues normally

```

Why a two-step nonce (request → consume)?

1. **Prevents replay:** Each nonce is consumed exactly once. A captured network request cannot be replayed.
2. **Prevents pre-generation:** The nonce has a 5-minute TTL. You can't stockpile nonces.
3. **Server has real-time visibility:** The consume call happens right before packaging starts, so the server knows the exact moment a build is produced.
4. **HMAC proof binds the nonce to the project + build config:** A nonce issued for project A cannot be used for project B.

Failure modes:

Scenario	Result
Network down during Step 1	Packaging canceled. Clear error message.
Server rejects JWT (expired / revoked)	Packaging canceled. Toast: "License expired. Refresh in Tools → Gorgeous → License Manager."
Network down during Step 3 (consume)	Packaging canceled. Unused nonce expires on server after 5 minutes.
Server returns nonce-already-consumed	Packaging canceled. Indicates a replay attempt or race condition.
5-minute window exceeded between Step 1 and Step 3	Packaging canceled. User retries (new nonce issued).

Note: Offline users (temporary offline or offline-first) use the **Epoch-Bound Packaging Nonce Chain** described in §6.10 / §6.19. This system derives single-use packaging nonces from the epoch chain — no server contact needed, no limited-use budget, but cryptographically impossible to use with an invalid or expired license.

6.5 AES Key Derivation

6.5.1 White-Box AES — The SharedSecret Never Exists

Security assessment finding: A conventional SharedSecret baked into `.rodata` is the #1 vulnerability. Even under heavy obfuscation, Dynamic Binary Instrumentation (DBI) tools like Intel Pin, Frida, or DynamoRIO can set hardware breakpoints on the HKDF call and read the SharedSecret directly from CPU registers during key derivation. Once extracted, all `.glibc` files can be forged.

Solution: Replace the conventional AES implementation with **White-Box AES (WB-AES)**. In a WB-AES implementation, the secret key is **embedded into the lookup tables** of the AES algorithm itself. The key does not exist as a contiguous value anywhere in memory — not in `.rodata`, not in registers, not on the stack. The lookup tables ARE the key.

How WB-AES works (simplified):

CONVENTIONAL AES:
key = 0xA3F7...C9E2 (32 bytes in .rodata – extractable)
ciphertext = AES_Encrypt(key, plaintext)
→ Attacker sets HW breakpoint on AES_Encrypt, reads `key` from register/stack

WHITE-BOX AES:
T1[256], T2[256], T3[256], ... (many MB of pre-computed lookup tables)
The tables encode the key INTO the computation.
ciphertext = WB_AES_Encrypt(T1, T2, T3, ..., plaintext)
→ There IS no `key` variable. The key is diffused across the table values.
→ Extracting it requires dumping ALL tables (~4-16 MB) and solving
a mathematical decomposition problem (key recovery from WB tables).

Implementation:

SharedSecret → REPLACED by WB-AES table set (generated at build time from the secret)
WB_Tables = WB_AES_GenerateTables(SharedSecret, "gorgeous-plugin-license-v1")
→ ~8 MB of lookup tables baked into the Vault DLL's .rodata
→ VM-protected access (§5.5.1) – the table read patterns are virtualized

Salt = SHA-256(ProjectID || PluginName)
AES_Key = WB_AES_HKDF(WB_Tables, Salt, info="gorgeous-plugin-license-v1", length=32)
AES_IV = first 12 bytes of SHA-256(ProjectID || PluginName || "iv-derivation")

Why WB-AES is dramatically harder to break than obfuscated-key-in-rodata:

Property	Conventional (obfuscated key)	White-Box AES
Key extraction via DBI	Set breakpoint on AES call → read key from register. Minutes.	Key doesn't exist in memory. Must dump ~8 MB of tables and solve decomposition. Weeks to months.
Key extraction via static analysis	Find <code>.rodata</code> constant → trace XOR/HKDF → reconstruct. Days.	Tables are ~8 MB of opaque data. No single constant to find. Must understand the full WB scheme. Months.
Key rotation	Rebuild DLL with new key (same difficulty to extract)	Rebuild DLL with new tables (same difficulty, but tables are much larger to analyze)
Combined with VM protection	VM hides the AES call, but key still passes through registers	VM hides the TABLE LOOKUPS — attacker must de-virtualize the VM AND solve WB decomposition

Note: WB-AES is not unbreakable. Academic attacks against some WB-AES constructions exist (Billet et al., 2004; Chow et al. attacks). The defense is to use a **modern WB-AES construction** (e.g., based on CHES 2016+ designs with external encodings) combined with VM protection of the table access paths. This raises the bar from "minutes with Frida" to "months of dedicated cryptanalysis."

6.5.2 Key Derivation Details

WB_Tables = (built into the Vault DLL – see §6.5.1)
Salt = SHA-256(ProjectID || PluginName)
AES_Key = WB_AES_HKDF(WB_Tables, Salt, info="gorgeous-plugin-license-v1", length=32)
AES_IV = first 12 bytes of SHA-256(ProjectID || PluginName || "iv-derivation")

This means:

- **Different project → different AES key → cannot reuse a license file across projects.**
- **Different plugin → different AES key → cannot reuse GorgeousEntertaining's license for GorgeousEvents.**
- **The WB-AES tables are the crown jewel** — baked into the Vault DLL and protected by VM protection (§5.5.1) + LLVM obfuscation (§5.5) + anti-tamper (§5.2).

6.6 What the Server Tracks (Piracy Detection)

Every license request includes:

Field	Purpose
<code>user_id</code>	Who owns this key
<code>plugin_name</code>	Which sibling plugin is being licensed (e.g., "GorgeousEntertaining")
<code>project_id</code>	Which UE project is using it
<code>hardware_fingerprint</code>	Machine identifier (hash of MAC + CPU ID + hostname)
<code>timestamp</code>	When the request was made

Field	Purpose
key_id	Which intermediate key was used

If the server sees anomalous usage patterns for a key — such as explosive machine growth, many distinct project IDs, high hardware churn, or geographic spread — the **License Classification Algorithm (§6.26.8)** classifies the key as SINGLE_DEV, TEAM, or PIRACY. Piracy-classified keys are immediately revoked via the Bloom filter (§6.20).

Note: GorgeousCore does not generate any license requests — it is free. Only sibling plugin license operations contact the server.

6.7 Consent & Transparency

CRITICAL for Fab compliance:

- On first run, before ANY network call, show a dialog explaining:
 - "Gorgeous Things has detected licensed sibling plugins (GorgeousEntertaining, GorgeousEvents, ...). License validation requires a one-time network request to [server URL]."
 - "GorgeousCore itself is free and does not require a license."
 - "The following data will be sent: [list exactly what is sent]"
 - "No data is collected during gameplay or in packaged builds."
 - [Accept] / [Enter Offline Key] / [Cancel]
- If the user cancels, **cascading death** is triggered — OV/AR functionality is disabled for all sibling plugins. GorgeousCore headers/types remain available but non-functional.
- All of this must be documented in the plugin's EULA / privacy policy.

6.8 Offline License Key

Not every user wants (or can) connect to a license server. The system supports a **fully offline workflow** via a standalone offline key.

Key Characteristics

Property	Value
Format	Signed JWT (ECDSA P-256)
Lifetime	365 days from issuance (non-renewable offline)
Bound to	project_id + plugin_name + hardware_fingerprint
Contains	user_id, project_id, plugin_name, hardware_fingerprint, expiry_timestamp, issued_at, nonce, offline: true
Stored as	Fallback slot inside <PluginName>.glic (type=plugin) in <ProjectDir>/Saved/GorgeousLicense/ (see §6.0.1)
Encryption	AES-256-GCM, key = HKDF(SharedSecret, ProjectID PluginName "fallback") — different HKDF info string from the primary slot

Note: Each sibling plugin gets its own .glic file (type=plugin) with the offline key stored in the **fallback slot**. All plugin license files must be present and valid to avoid cascading death. There are no separate offline key files — primary and fallback coexist in the same .glic file.

How a User Obtains an Offline Key

Option A — Via the License Server API (automated, one call per sibling plugin):

```
POST /api/v1/license/offline
Body: {
  "global_key": "<user's global license key>",
  "plugin_name": "GorgeousEntertaining",
  "project_id": "E0C3F3454AA9F4309CCBD6829FE5C95E",
  "hardware_fingerprint": "<hash of MAC + CPU ID + hostname>"
}
Response: {
  "offline_license": "<base64-encoded encrypted blob>",
  "plugin_name": "GorgeousEntertaining",
  "expires_at": "2026-05-28T00:00:00Z"
}
```

The Vault writes the response into the **fallback slot** of GorgeousEntertaining.glic (type=plugin). Repeated for each sibling plugin.

Option B — Via the Gorgeous License Manager UI (in-editor):

- User opens Tools → Gorgeous → License Manager
- UI shows all discovered sibling plugins and their license status
- Clicks "Generate Offline Licenses" (batch operation for all installed sibling plugins)
- A one-time HTTPS request per plugin is made (with consent) to fetch the offline keys
- Offline keys are written into the **fallback slot** of each plugin's .glic file automatically
- From this point on, no further network calls are needed for 365 days

Option C — Manual key entry (fully air-gapped):

1. User contacts support or uses a self-service web portal on another machine
2. Provides their `project_id`, `plugin_name`, and `hardware_fingerprint` (displayed in the License Manager UI)
3. Receives a base64 string per plugin
4. Pastes each into the License Manager → selects the target plugin → "Enter Offline Key"
5. The Vault decrypts and validates each locally

6.9 Offline Key — Additional Security Measures

Because the offline key is valid for up to **365 days** without any server contact, it is a higher-value target for piracy. Extra protections:

Measure	Details
Hardware binding	The JWT contains a <code>hardware_fingerprint</code> claim. On every editor startup the Vault recomputes the local fingerprint and compares. If it doesn't match → key rejected.
Nonce / one-time issuance	Each offline key contains a unique <code>nonce</code> . The server records this nonce. If the same <code>nonce</code> appears from a different <code>hardware_fingerprint</code> in a future online check → piracy flag raised on the account.
Dual signature	The offline JWT is signed twice: once by the server (ECDSA P-256, standard JWT <code>alg: ES256</code>) and once by a secondary key whose public half is embedded deeper in the Vault's obfuscated code (separate from the primary verification key). An attacker who extracts one key still can't forge a valid offline key.
Encrypted <code>hardware_fingerprint</code> claim	The <code>hardware_fingerprint</code> inside the JWT is itself AES-encrypted with a key derived from the <code>nonce</code> . This prevents an attacker from simply editing the fingerprint field in a stolen JWT — they'd need to know the nonce-derived key, which is only in the Vault.
System clock drift detection	On each startup the Vault records <code>FDateTime::UtcNow()</code> into an obfuscated marker file. If the current time is EARLIER than the last recorded time by more than 2 hours → clock tampering detected → offline key invalidated.

6.10 Offline Key — Packaging Support

When a user who operates offline (temporarily or permanently offline-first) wants to **package** their project, the system uses an **Epoch-Bound Packaging Nonce Chain** — a smart, nonce-driven authorization that is cryptographically anchored to a valid license epoch. No server contact, no limited-use budget, but provably impossible to use with an invalid license.

Core Principle: The Packaging Nonce IS the License Proof

Instead of checking a boolean `allow_package` claim and counting uses, the Vault **derives the packaging authorization nonce directly from the current epoch key**. If the epoch key is invalid (expired, tampered, wrong hardware) → the derived nonce is wrong → the internal packaging gate rejects it. There is no separate "packaging permission" to forge — the nonce itself IS the proof.

EPOCH-BOUND PACKAGING NONCE CHAIN

The packaging nonce is derived from 5 interlocking inputs:

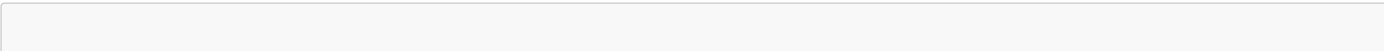
Nonce_Package = HMAC-SHA256(
 key = Epoch_N_Key,
 message = project_id
 || build_config
 || hardware_fingerprint
 || package_rolling_counter
 || "gorgeous-package-v2"
)

Authorization = {
 nonce: Nonce_Package,
 epoch: N,
 counter_at_package: package_rolling_counter,
 project_id: <project>,
 build_config: "Shipping" / "Development" / ...,
 hardware_fp: <local hash>,
 timestamp: now,
 epoch_chain_anchor: SHA-256(GlobalKey.glic_INTEGRITY_TAG),
 auth_tag: HMAC-SHA256(Epoch_N_Key, all_fields_above)
}

Why this can't be faked:

- Epoch_N_Key is derived from master_seed + epoch chain (\$6.15)
- master_seed is triple-envelope encrypted (hardware + Vault + temporal binding)
- An invalid/expired/tampered license → wrong Epoch_N_Key → wrong nonce → rejected
- There is no boolean flag to flip – the math either works or it doesn't

How It Works: Step by Step





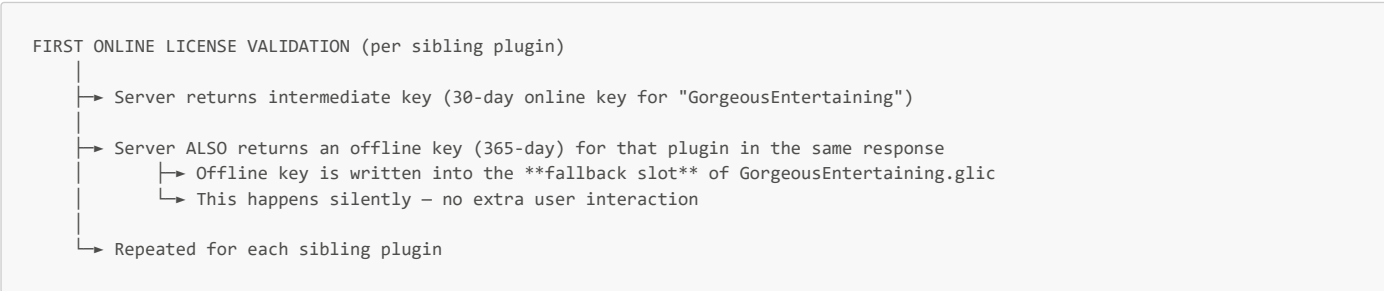
Security Properties of the Epoch-Bound Nonce Chain

Property	How It's Enforced
Can't package with expired license	Expired epoch → Epoch_N_Key derived from expired chain → double-hash verification fails because the INTEGRITY_TAG has already been updated to reject expired epochs.
Can't package on wrong hardware	hardware_fingerprint is an input to Nonce_A. Different hardware → different nonce → verification fails.
Can't replay a previous packaging event	package_rolling_counter increments after every package. Next derivation uses counter+1 → completely different nonce. Counter is stored in 3 locations (§6.17) with cross-verification.
Can't forge a nonce without the full epoch chain	Nonce_B requires the PREVIOUS epoch's key. The epoch chain is an unbroken hash sequence anchored to root master_seed signed by the server's ECDSA key. No master_seed → no chain → no nonce.
Can't stockpile nonces	Nonces are computed on-the-fly, not pre-generated. The rolling counter ensures each derivation is unique. There is nothing to stockpile.
Can't skip the plugin check	Step 2 runs BEFORE Step 3. The nonce derivation function internally reads the plugin license registry. If the registry shows any INVALID plugin, the derivation function returns a poisoned nonce (all-zeros) that will always fail verification.
Can't tamper with receipts	Receipts are HMAC-signed with Epoch_N_Key. When the user eventually comes online, the server can verify receipts to audit packaging history. Tampered receipts → piracy flag.
Unlimited packaging	There is NO limited-use budget. A valid license = unlimited packages. The system trusts the math, not arbitrary limits.

Design philosophy: The old approach (limited-use packaging budget) punished legitimate offline users with artificial scarcity. The Epoch-Bound Nonce Chain removes this friction entirely — if your license is valid, you can package as many times as you want. The nonce derivation is the license check. They are the same operation.

6.11 Auto-Fetched Offline Key (Online Users)

When a user **does** use the license server (the normal online flow), the **first successful license validation for each sibling plugin** also silently fetches and stores an offline key as a fallback:

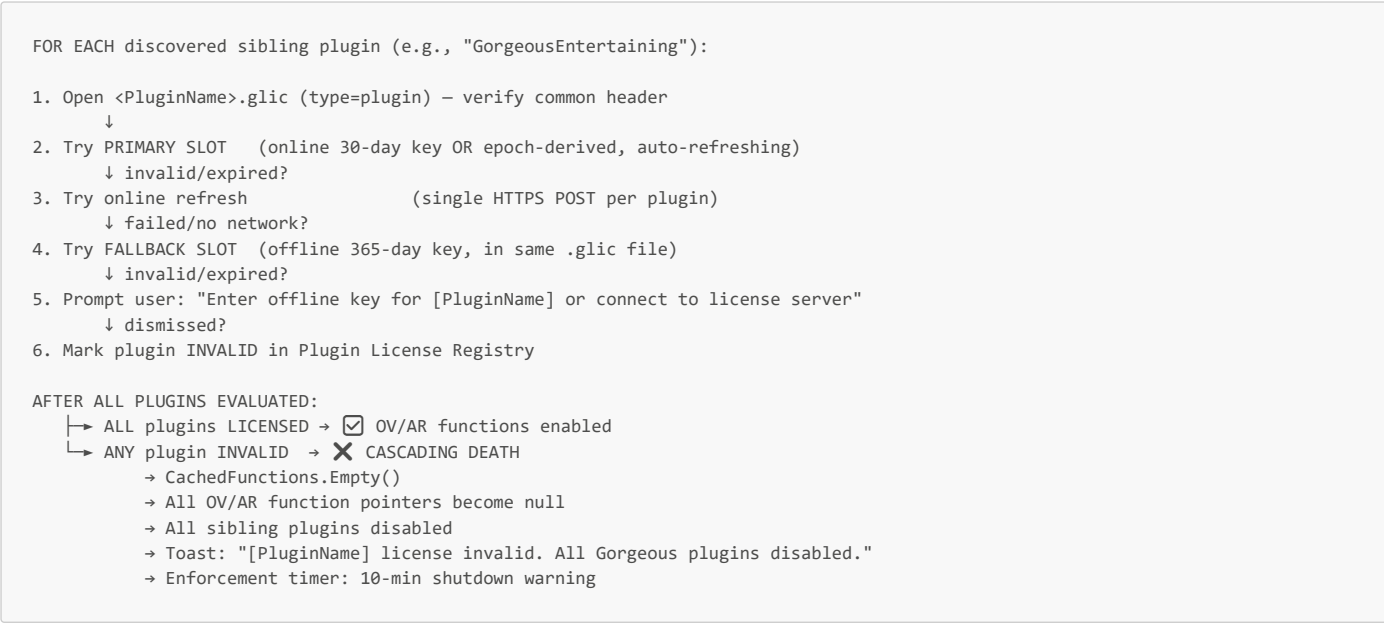


Fallback behavior (per plugin):

- 1. On editor startup, Vault opens <PluginName>.glic (type=plugin) and tries the **primary slot** first
- 2. If that fails (expired, network down, can't refresh) → falls back to the **fallback slot** in the same file
- 3. If the fallback slot is still valid → plugin is marked LICENSED, with a small toast: "Using offline license for [PluginName] (expires in X days). Connect to refresh."
- 4. If both slots are expired → plugin marked INVALID → **cascading death triggers** (all plugins disabled)

This means **an online user can survive up to 365 days of server downtime or network loss** without any interruption.

6.12 License Priority & Resolution Order (Per Sibling Plugin)



6.13 Cascading Death — All-or-Nothing Enforcement

This is the central enforcement mechanism of the sibling plugin licensing model.

Principle

When **any** installed sibling plugin has an invalid, expired, or missing license, the Vault does not degrade gracefully per-plugin. Instead, it **destroys the entire OV/AR function reference table**, killing functionality for **all** plugins — including those with perfectly valid licenses.

Why This Design?

Reason	Explanation
Prevents partial piracy	A user cannot pirate one cheap plugin while buying the rest. Either all licenses are valid, or nothing works.
Social/economic pressure	Teams are incentivized to ensure all licenses are current. One expired license from a teammate affects everyone.
Simplicity of implementation	A single <code>CachedFunctions.Empty()</code> call in the VaultLoader kills all OV/AR. No complex per-plugin function gating needed.

Reason	Explanation
Harder to bypass	Per-plugin gating would require the Vault to selectively enable/disable functions, creating a larger attack surface. All-or-nothing is binary — either the table is populated or it's empty.

Technical Implementation

```
// Inside GorgeousVault - Licensing Engine
void FPluginLicenseRegistry::EvaluateAndEnforce()
{
    bool bAnyInvalid = false;
    FString OffendingPlugin;

    for (const auto& [PluginName, Status] : PluginLicenses)
    {
        if (Status != EPluginLicenseStatus::Valid)
        {
            bAnyInvalid = true;
            OffendingPlugin = PluginName;
            break;
        }
    }

    if (bAnyInvalid)
    {
        // =====
        // CASCADING DEATH - clear the entire reference table
        // =====
        //
        // This zeros out ALL cached function pointers in
        // the VaultLoader. Every subsequent OV/AR call from
        // ANY plugin will resolve to nullptr -> graceful no-op
        // or fatal error depending on proxy implementation.

        GorgeousVaultLoader::Get().ClearAllCachedFunctions();

        // Notify the user which plugin caused the cascade
        UE_LOG(LogGorgeousCore, Error,
            TEXT("[GorgeousVault] Cascading death triggered. "
                "Plugin '%s' has an invalid license. "
                "ALL OV/AR functionality is disabled until resolved."),
            *OffendingPlugin);

        // Fire delegate for UI layer to show toast/dialog
        OnCascadingDeathTriggered.Broadcast(OffendingPlugin);

        // Start 10-minute enforcement timer
        StartEnforcementShutdownTimer();
    }
}
```

What Gets Killed

When `CachedFunctions.Empty()` fires, the following are immediately non-functional:

System	Effect
Object Variable lifecycle	<code>InvokeInstancedFunctionality</code> , <code>BeginDestroy</code> , <code>RegisterWithRegistry</code> — all null
OV Serialization	<code>SerializeToPayload</code> , <code>DeserializeFromPayload</code> — save/load breaks
OV Replication	<code>ActivateReplication</code> , <code>RequestAutoReplicationRPC</code> — multiplayer OV dead
AutoReplication Mixin	<code>Bind</code> , <code>RequestRPC</code> , <code>Tick</code> — replication mixin non-functional
AutoReplication Coordinator	<code>RegisterVariable</code> , <code>EvaluateDirtyVariables</code> — coordination dead
AutoReplication Graph	<code>RegisterObjectVariable</code> , <code>RouteReplicationUpdate</code> — routing dead
RPC Transporter	<code>RouterRPC</code> — no RPCs
Iris Backend	<code>RegisterStream</code> , <code>Sync</code> — no Iris
World Subsystem	<code>Initialize</code> , <code>Deinitialize</code> — subsystem dead
All sibling plugins	Everything that depends on OV/AR through <code>GorgeousCore</code> -> non-functional

What Survives

System	Why
GorgeousCore headers/types	These are compile-time constructs, not runtime function pointers
GorgeousCoreRuntimeUtilities	Helper macros, logging — no Vault dependency
GorgeousCoreEditor	K2 nodes, property customizations — they compile but OV calls fail at runtime
GorgeousCoreEditorUtilities	Extension Resource Guard, validation — still operational
Non-OV code in sibling plugins	Any code that doesn't call OV/AR functions continues to work

Recovery

Cascading death is **reversible without restarting the editor** if the user resolves the license issue:

1. User opens Tools → Gorgeous → License Manager
2. Enters/refreshes the missing license for the offending plugin
3. Vault re-evaluates the Plugin License Registry
4. IF all plugins now VALID:
 - Vault reloads function pointers into CachedFunctions
 - OV/AR resumes immediately
 - Toast: "All licenses validated. Gorgeous plugins re-enabled."
5. IF still invalid:
 - Cascading death remains in effect
 - Enforcement timer continues

6.14 Offline-First License Renewal — Global Key on User Machine

DESIGN GOAL — Zero Server Dependency After Key Export

At **any time** after purchase, the user can request their **Global License Key** — a single server request that exports the key to their machine. From that point forward, ALL license operations — validation, renewal, hardware migration, packaging authorization — happen **locally inside the Vault DLL**. No licensing server is ever contacted again. The user owns their license unconditionally.

This is NOT limited to purchase time. A user who has been on the online model for months or years can switch to offline-first with one click in the License Manager.

6.14.1 Why This Changes Everything

The standard model (§6.1–6.12) assumes periodic server contact for renewals. This section defines an **alternative, fully sovereign model** where:

Standard Model	Offline-First Model
Global Key lives ONLY on the server	Global Key is given to the user
Intermediate keys expire in 30 days, need server refresh	Licenses are self-renewed locally via deterministic epoch chain
Hardware change → contact server	Hardware change → user enters Recovery Phrase (offline)
Packaging → server nonce round-trip	Packaging → local nonce derivation with rolling counter
Revocation → server blacklists key	Revocation → Bloom filter baked into Vault DLL updates
Server downtime = potential lockout	Impossible to be locked out

The user may switch at any time. The License Manager UI offers both "Online (auto-refresh)" and "Offline-First (self-sovereign)" flows. Switching to offline-first requires a single server request to export the Global License Key. Once exported, the offline-first model is fully independent. The user can also switch back to online at any time — the `GlobalKey.glic` (type=master) file remains valid alongside online keys.

6.15 The Global License Key — User's Sovereign Copy

When the user switches to offline-first, the server generates and exports the Global License Key **to the user's machine**. This can happen at purchase, a week later, or five years later — whenever the user decides. The key is a multi-layered cryptographic construct stored as a **type=master .glic file** (see §6.0.1).

How the User Obtains Their GlobalKey.glic (type=master)

Option A — Via License Manager UI (recommended):

1. Tools → Gorgeous → License Manager → Export Global Key
2. One-time server request (with existing online license credentials)
3. Server generates GlobalKey.glic (type=master) file with all purchased plugins and master_seed
4. File is saved to <ProjectDir>/Saved/GorgeousLicense/GlobalKey.glic
5. Recovery Phrase is displayed — user must save it securely
6. Done. No further server contact ever needed.

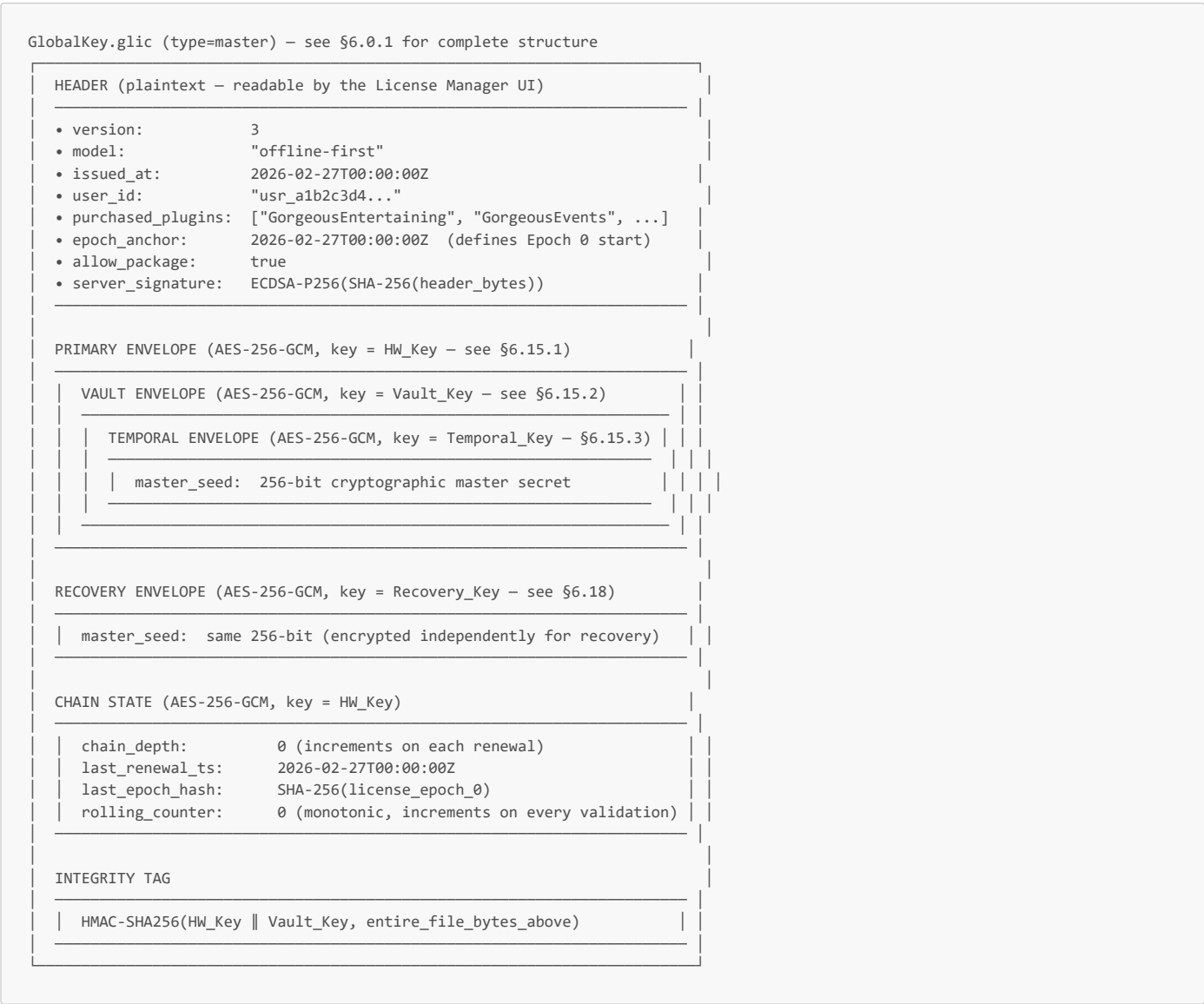
Option B — Via self-service web portal:

- 1. User logs into the Gorgeous license portal
- 2. Clicks "Export Global Key → Offline-First"
- 3. Provides `project_id` + `hardware_fingerprint` (displayed in License Manager UI)
- 4. Downloads the `GlobalKey.glic` (type=master) file + Recovery Phrase
- 5. Places file in `<ProjectDir>/Saved/GorgeousLicense/`

Option C — Via API (automation / CI):

```
POST /api/v1/license/export-global
Body: {
  "intermediate_key": "<current valid JWT>", // proves ownership
  "project_id": "E0C3F3454AA9F4309CCBD6829FE5C95E",
  "hardware_fingerprint": "<local hash>"
}
Response: {
  "glic_master": "<base64-encoded GlobalKey.glic (type=master) file>",
  "recovery_phrase": "abandon ability able ...",
  "model": "offline-first"
}
```

Structure



Stored at: `<ProjectDir>/Saved/GorgeousLicense/GlobalKey.glic`

Note: This structure diagram is the **original §6.15 reference**. The canonical, updated structure for the type=master `.glic` file is defined in §6.0.1 (with the unified common header including `magic`, `format_version`, `type`, etc.).

The `master_seed` is the crown jewel. It never exists in plaintext outside the Vault DLL's obfuscated renewal logic, and only for the absolute minimum number of CPU cycles required. **Three independent encryption envelopes** protect it.

6.15.1 Layer 1 — Hardware Binding (outermost envelope)

```

HW_Fingerprint = SHA-256(
    CPU_Brand_String
    || SMBIOS_UUID
    || SMBIOS_Serial
    || Primary_MAC_Address
    || OS_Install_ID           // Windows: MachineGuid, Linux: /etc/machine-id
    || Disk_Serial_of_System_Drive
)

HW_Key = HKDF-SHA256(
    ikm    = HW_Fingerprint,
    salt   = SHA-256(user_id || "gorgeous-hw-binding-v2"),
    info   = "gorgeous-offline-layer1",
    length = 32
)

```

What this protects against: File copying. Even if an attacker has the `GlobalKey.glic` file, decrypting Layer 1 requires the **exact hardware** it was bound to.

Fuzzy matching (anti-false-positive): The fingerprint uses 6 components. The Vault tolerates up to 2 components changing (e.g., new NIC + new disk after an upgrade) by storing encrypted checksums of each component individually. If ≥ 4 of 6 match $\rightarrow$ HW_Key can still be derived. If < 4 match $\rightarrow$ hardware migration flow (§6.18).

6.15.2 Layer 2 — Vault Binary Binding (middle envelope)

```

Vault_Binding_Material = SHA-256(
    Vault_DLL_.text_section_hash    // changes if the DLL is modified
    || VAULT_BUILD_NONCE           // unique per Vault build (baked at compile time)
    || VAULT_INTERNAL_DERIVATION_SALT // 64-byte constant deep in obfuscated code
)

Vault_Key = HKDF-SHA256(
    ikm    = Vault_Binding_Material,
    salt   = SHA-256(user_id || HW_Fingerprint),
    info   = "gorgeous-offline-layer2",
    length = 32
)

```

What this protects against: Cracked Vault DLLs. A modified Vault binary produces a different `.text` hash $\rightarrow$ different `Vault_Key` $\rightarrow$ cannot decrypt Layer 2 $\rightarrow$ master_seed is inaccessible.

Vault update compatibility: When a legitimate Vault update ships (new DLL with new `.text` hash), the update process:

0. We ensure that we have a copy of the original vault dll before the update happens (Fab tends to direct replace old content with new)
1. We load the old vault dll in a hidden context to extract the old Vault_Key (since the new DLL won't be able to decrypt the existing GlobalKey.glic)
2. Old Vault decrypts master_seed using old Vault_Key
3. Derives new Vault_Key from new DLL hash
4. Re-encrypts Layer 2 with new Vault_Key
5. Updates INTEGRITY TAG
6. Wipes old key material from memory

This happens automatically on first startup after a Vault DLL update, transparent to the user.

How Step 0 works — Pre-Update Shadow Copy:

The VaultLoader (open-source proxy side) maintains a shadow backup of the Vault DLL to guarantee the old binary is always available when a Fab update replaces it in-place.

SHADOW COPY LIFECYCLE

Location: `<ProjectDir>/Saved/GorgeousLicense/.vault_previous.dll`

ON EVERY SUCCESSFUL STARTUP (after full license validation passes):

1. VaultLoader computes `current_text_hash = SHA-256(GorgeousVault.dll .text section)`
2. Reads `stored_text_hash` from `GlobalKey.glic INTEGRITY TAG metadata`
3. IF `current_text_hash == stored_text_hash`:
 - $\rightarrow$ DLL matches the license state – this is the "known good" binary
 - $\rightarrow$ Copy `GorgeousVault.dll` $\rightarrow$ `.vault_previous.dll` (overwrite)
 - $\rightarrow$ This ensures the shadow is always the LAST KNOWN WORKING version
4. The shadow copy is written BEFORE any license operations,

```
so it is guaranteed to exist for the next startup.

ON NEXT STARTUP AFTER FAB UPDATE (DLL hash mismatch detected):
1. VaultLoader loads NEW GorgeousVault.dll
2. Attempts Layer 2 decryption with new Vault_Key → FAILS (expected)
3. Detects .vault_previous.dll exists at shadow path
4. Loads .vault_previous.dll into a SEPARATE, isolated module context
   (LoadLibraryEx with LOAD_LIBRARY_AS_DATAFILE is NOT sufficient –
   we need executable code, so we use a renamed temp copy loaded
   via standard LoadLibrary into a distinct namespace)
5. Calls old Vault's internal re-key export:
   GVault_Internal_ReKeyLayer2(old_dll_context, new_dll_text_hash)
   → Old Vault decrypts master_seed with old Vault_Key
   → Derives new Vault_Key from new_dll_text_hash
   → Re-encrypts Layer 2 → updates Layer 3 (Temporal_Key depends
   on Vault_Key as salt) → updates INTEGRITY TAG
6. Unloads old DLL, deletes temp copy
7. New DLL can now decrypt GlobalKey.glic normally
8. Shadow copy is updated to the NEW DLL (step 3 of normal flow above)

EDGE CASES:
• Shadow copy is missing (first install, or manually deleted):
  → If Layer 2 decryption fails AND no shadow exists → cannot auto-migrate
  → Toast: "Vault DLL updated but previous version not found.
    Use your Recovery Phrase ($6.18) to re-bind your license."
  → Recovery Phrase decrypts the RECOVERY ENVELOPE (independent of Vault_Key)
  → master_seed recovered → re-encrypted with new Vault_Key → done
• Shadow copy is corrupted:
  → Same as missing – fall back to Recovery Phrase
• Attacker replaces shadow with a cracked DLL:
  → The shadow DLL's .text hash must match the stored_text_hash in
    GlobalKey.glic INTEGRITY TAG. A cracked DLL has a different hash
  → mismatch → shadow rejected → Recovery Phrase fallback
• Multiple rapid updates (two Vault updates before the editor runs):
  → Shadow still holds the version that last successfully validated
  → Re-key works as long as the shadow matches the stored_text_hash
  → If it doesn't (because TWO updates happened) → Recovery Phrase fallback
```

Why this lives in the VaultLoader (open-source) and not inside the Vault DLL: The shadow copy must be created BEFORE the DLL is replaced. The Vault DLL cannot protect itself from being overwritten. The VaultLoader, which runs first and loads the Vault, is the correct place to manage the shadow lifecycle. The actual re-key cryptography still happens inside the old Vault's code (step 5) — the VaultLoader just orchestrates the loading.

6.15.3 Layer 3 — Temporal Binding (innermost envelope)

```
Temporal_Key = HKDF-SHA256(
    ikm    = SHA-256(epoch_anchor || chain_depth || rolling_counter || last_renewal_ts),
    salt   = SHA-256(HW_Key || Vault_Key),
    info   = "gorgeous-offline-layer3",
    length = 32
)
```

What this protects against: State rollback. An attacker who captures the file at time T cannot roll back to it at time T+N, because `chain_depth` and `rolling_counter` have advanced, producing a different `Temporal_Key`.

6.15.4 Triple Envelope Summary — What an Attacker Must Simultaneously Possess

To extract `master_seed`, an attacker needs ALL THREE simultaneously:

Layer 1: HW_Key	← Requires the EXACT MACHINE (6-component fingerprint) Cannot be transferred to different hardware
Layer 2: Vault_Key	← Requires GENUINE, UNMODIFIED Vault DLL Any patch/crack → different .text hash → wrong key
Layer 3: Temporal_Key	← Requires CURRENT STATE (counter, depth, timestamp) Cannot replay a captured older state

ALL THREE keys are derived independently.
Compromising one does not help with the other two.
The `master_seed` NEVER touches disk in plaintext – it exists in CPU registers for <50ms during renewal, then is overwritten with random bytes + zeroed.

6.16 Epoch Chain — Deterministic Self-Renewal

Licenses are organized into **epochs** (365 days each). Renewal is automatic, deterministic, and requires zero network contact.

Epoch Key Derivation

```
Epoch_N_Key = HMAC-SHA256(
  key      = master_seed,
  message  = epoch_anchor || N || HW_Fingerprint || "gorgeous-epoch-v1"
)
```

Each epoch produces a **unique, unpredictable key** that:

- Can only be derived by someone who possesses the `master_seed`
- Is bound to the specific hardware (different machine → different epoch key)
- Is deterministic — the SAME master_seed + epoch_number + hardware always produces the SAME key

Per-Plugin License Derivation (from Epoch Key)

```
For each sibling plugin in purchased_plugins[]:

Plugin_License_Key = HMAC-SHA256(
  key      = Epoch_N_Key,
  message  = plugin_name || project_id || "gorgeous-plugin-epoch-v1"
)

License_N_for_Plugin = {
  user_id:      <from header>,
  plugin_name:  <plugin>,
  project_id:   <project>,
  epoch:        N,
  issued_at:    epoch_anchor + (N * 365 days),
  expires_at:   epoch_anchor + ((N+1) * 365 days),
  allow_package: <from header>,
  prev_epoch_hash: SHA-256(License_(N-1)_for_Plugin),
  license_signature: HMAC-SHA256(Plugin_License_Key, all_fields_above)
}

Stored as: <PluginName>.glic (encrypted with standard §6.5 AES scheme)
```

Automatic Renewal Flow

```
EDITOR STARTUP
|
|→ Vault loads GlobalKey.glic (type=master)
|
|→ Decrypt chain state (HW_Key)
|   Read: chain_depth = N, rolling_counter, last_renewal_ts
|
|→ Check current epoch:
|   current_epoch = floor((now - epoch_anchor) / 365 days)
|
|→ IF current_epoch == chain_depth:
|   → License is current, no renewal needed
|   → Validate existing per-plugin .glic files
|   → Increment rolling_counter
|   → Re-encrypt chain state
|   → Done ☒
|
|→ IF current_epoch == chain_depth + 1:
|   → Time to renew! (current license is expiring/expired)
|   |
|   |→ Decrypt master_seed (3-layer envelope)
|   |   ⌚ Seed enters memory – clock starts (< 50ms target)
|   |
|   |→ Derive Epoch_(N+1)_Key from master_seed
|   |
|   |→ Generate new per-plugin licenses from Epoch_(N+1)_Key
|   |
|   |→ Encrypt new licenses → write to <PluginName>.glic
|   |
|   |→ Update chain state: chain_depth = N+1, rolling_counter++, last_renewal_ts = now
```

```

→ Re-encrypt chain state + update Temporal_Key (Layer 3 binding changes)

→ Wipe master_seed from memory:
  1. Overwrite with crypto-random bytes
  2. Overwrite with zeros
  3. Memory barrier (_ReadWriteBarrier / asm volatile)
  ⌚ Seed leaves memory – clock stops

→ Update INTEGRITY TAG on GlobalKey.glic

→ Toast: "Gorgeous license renewed for another year. No network needed."
  → Done ☑

→ IF current_epoch > chain_depth + 1:
  → User hasn't opened the editor in >1 year
  → Multi-epoch jump: derive ALL missing epochs sequentially
  → (master_seed in memory for slightly longer, but still < 200ms)
  → Same flow as above, repeated for each skipped epoch
  → Done ☑

→ IF current_epoch < chain_depth:
  → CLOCK TAMPERING DETECTED (time went backward)
  → Refuse renewal
  → Use existing license if not expired
  → If expired → cascading death
  → Log warning: "System clock appears to have been set backward."

```

6.17 Rolling Nonce Counter — Anti-Rollback & Anti-Replay

A **monotonic counter** stored inside the `GlobalKey.glic` (type=master) file prevents an attacker from rolling back to a previous file state.

Counter Mechanics

```

rolling_counter:
  • Starts at 0 on issuance
  • Increments by 1 on EVERY:
    - Editor startup (license validation)
    - License renewal (epoch transition)
    - Packaging operation
    - Hardware migration
  • Is part of the Temporal_Key derivation (Layer 3)
  • NEVER decrements – monotonic only

Counter storage:
  • Primary: inside GlobalKey.glic (encrypted in chain state)
  • Shadow: <ProjectDir>/Saved/GorgeousLicense/.gorgeous_nonce
    (AES-256-GCM encrypted with HW_Key, contains counter + HMAC)
  • Embedded: last 4 bytes of each per-plugin .glic file
    (encrypted, acts as a cross-reference)

All three locations must agree. If any disagree → tamper detected.

```

Rollback Attack Scenario and Defense

```

ATTACKER SCENARIO:
T=0: counter=100, license valid, attacker snapshots GlobalKey.glic
T=1: counter=150, license renewed to next epoch
T=2: attacker restores T=0 snapshot (counter=100)

VAULT DEFENSE:
T=2 startup:
1. Decrypt GlobalKey.glic chain state → counter = 100
2. Read shadow counter .gorgeous_nonce → counter = 150
3. MISMATCH: GlobalKey.glic says 100, shadow says 150
  → GlobalKey.glic has been rolled back
  → Enter TAMPER RECOVERY MODE:
    a. Refuse to decrypt master_seed
    b. Existing per-plugin .glic files have embedded counter = 150
      → They also disagree with GlobalKey.glic's counter = 100
    c. ALL LICENSE VALIDATION FAILS
    d. User must either:
      • Run editor forward to increment shadow counter honestly

```

- (but GlobalKey.glic counter is wrong → permanent mismatch)
- Use Recovery Phrase to re-initialize the chain (§6.18)

Counter Saturation Resistance

At one increment per editor startup (say, 10x/day for 50 years), the counter reaches $\sim 182,500$. A 64-bit counter has headroom for $\sim 1.8 \times 10^{19}$ increments. Overflow is not a concern.

6.18 Hardware Migration & Recovery — Fully Offline

When hardware changes significantly (≥ 3 of 6 fingerprint components differ), Layer 1 decryption fails.

Recovery Phrase

When the `GlobalKey.glic` (type=master) file is exported (at any time), the user also receives a **Recovery Phrase**:

```
RECOVERY PHRASE

Format: 24-word BIP-39 mnemonic
Example: "abandon ability able about above absent absorb
         abstract absurd abuse access accident account
         accuse achieve acid acoustic acquire across
         act action actor actress actual"

Encodes: 256-bit Recovery_Seed
Derives: Recovery_Key for the RECOVERY ENVELOPE in GlobalKey.glic

⚠ THE USER MUST STORE THIS SECURELY (password manager,
   printed on paper, safe deposit box).
⚠ LOSING IT means hardware migration requires support contact.
⚠ SHARING IT means anyone can migrate the license.
```

Recovery Key Derivation

```
Recovery_Seed = BIP-39 mnemonic → 256-bit entropy

Recovery_Key = HKDF-SHA256(
  ikm   = SHA-256(Recovery_Seed),
  salt  = SHA-256(user_id || epoch_anchor || "gorgeous-recovery-v2"),
  info  = "gorgeous-offline-recovery",
  length = 32
)
```

The `GlobalKey.glic` (type=master) file contains a **RECOVERY ENVELOPE** — the master_seed encrypted independently with `Recovery_Key`. This envelope is **completely separate** from the triple-envelope chain.

Hardware Migration Flow

```
HARDWARE CHANGE DETECTED

→ Vault tries to decrypt Layer 1 (HW_Key)
  → FAILS: fingerprint mismatch ( $\geq 3$  components differ)

→ Vault prompts: "Hardware change detected. Enter your Recovery Phrase
  to migrate your Gorgeous license to this machine."

→ User enters 24-word mnemonic in License Manager UI

→ Vault derives Recovery_Key from mnemonic

→ Decrypt RECOVERY ENVELOPE → obtain master_seed
  ⌚ Seed enters memory

→ Compute new HW_Fingerprint for THIS machine

→ Derive new HW_Key, Vault_Key, Temporal_Key for this machine

→ Re-encrypt master_seed in new triple envelope
```

```

→ Re-encrypt RECOVERY ENVELOPE with ROTATED Recovery_Key:
    → New_Recovery_Seed = HKDF-SHA256(
        ikm = Recovery_Seed || new_HW_Fingerprint,
        salt = "gorgeous-recovery-rotation",
        info = "recovery-regen-v1",
        length = 32
    )
    → New recovery mnemonic is derived and DISPLAYED TO USER:
        "Your NEW Recovery Phrase is: [24 words]"
        "Your old phrase will no longer work."
        "Please store this securely."
    → User must acknowledge: [I have saved my new Recovery Phrase]

→ Reset chain state:
    • chain_depth continues from current epoch (NOT reset to 0)
    • rolling_counter continues from current value + 1
    • last_renewal_ts = now
    • Shadow counter and .glic embedded counters are re-synced

→ Wipe master_seed + old Recovery_Seed from memory
    ⌚ Seed leaves memory

→ Generate fresh per-plugin .glic files for current epoch on new hardware

→ Toast: "License migrated to new hardware. Welcome back! 🍷"
    → No server contact. Fully offline.

```

Why Recovery Phrase Rotation?

If an attacker obtains the user's Recovery Phrase (social engineering, shoulder surfing), they could migrate the license to their machine. By **rotating the phrase on every migration**, a stolen old phrase becomes useless after the legitimate user migrates.

Migration limit: The Vault embeds a migration counter in the `GlobalKey.glic` file. After **5 hardware migrations**, the file self-locks and requires support contact. This prevents an attacker with a stolen Recovery Phrase from endlessly migrating. The counter is encrypted within the INTEGRITY TAG.

6.19 Offline Packaging Authorization

This section provides detailed technical specification of the **Epoch-Bound Packaging Nonce Chain** introduced in §6.10. The high-level flow is described in §6.10; this section covers the cryptographic internals.

Nonce Derivation — Full Specification

EPOCH-BOUND PACKAGING NONCE CHAIN — CRYPTOGRAPHIC DETAIL

INPUTS (all must be present and valid):

- Epoch_N_Key ← from current epoch (requires valid master_seed)
- Epoch_N_Minus1_Key ← from previous epoch (requires epoch chain)
 - GENESIS KEY (when N == 0, no previous epoch exists):
 - Epoch_Genesis_Key = HMAC-SHA256(
 key = master_seed,
 msg = epoch_anchor || server_signature || "gorgeous-epoch-genesis"
)
 - This synthetic "Epoch -1" is:
 - Deterministic (same master_seed → same key every time)
 - Distinct from any real Epoch_N_Key (different msg format — no integer N, uses "genesis" info string instead of "epoch-v1")
 - Anchored to server_signature (tied to original purchase proof)
 - Requires master_seed (same security as any epoch key)
 - When N >= 1: Epoch_N_Minus1_Key = normal epoch chain (§6.16)
- project_id ← FApp::GetProjectId()
- hardware_fingerprint ← 6-component hardware hash
- build_config ← "Shipping" / "Development" / etc.
- package_rolling_counter ← monotonic, 3-location storage (§6.17)
- SALT_A = "gorgeous-epoch-nonce-alpha"
- SALT_B = "gorgeous-epoch-nonce-bravo"
- GlobalKey INTEGRITY_TAG ← from the .glic file header

DERIVATION:

Nonce_A = HMAC-SHA256(

```

    key = Epoch_N_Key,
    msg = project_id || counter || hw_fp || config || SALT_A
)

Nonce_B = HMAC-SHA256(
    key = Epoch_N_Minus1_Key,
    msg = project_id || counter || hw_fp || config || SALT_B
)

Gate_Value = HMAC-SHA256(
    key = Nonce_A ⊕ Nonce_B,
    msg = "gorgeous-package-gate"
)

Expected_Gate = HMAC-SHA256(
    key = Epoch_N_Key,
    msg = INTEGRITY_TAG || "gorgeous-package-expected"
)

VERIFICATION:
-----
IF Gate_Value[0..15] == Expected_Gate[0..15] → PASS
ELSE → REJECT packaging

WHY 16-BYTE TRUNCATED COMPARISON:
The full 32-byte comparison would also work but truncating to
128 bits prevents timing side-channels (constant-time memcmp on
16 bytes is more reliably constant-time across platforms).
128-bit collision resistance is still astronomically strong.

```

Plugin License Poisoning — Why Invalid Plugins Can't Package

The nonce derivation function doesn't just check the GlobalKey. Before computing Nonce_A, it reads the **Plugin License Registry** (the per-plugin license status map from `GVault_GetPluginLicenseRegistry`):

```

// Pseudocode inside the Vault (obfuscated in production)
int32 DerivePackagingNonce(uint8* NonceOut, ...)
{
    // Step 1: Check all plugin licenses
    FPluginLicenseRegistry Registry;
    GVault_GetPluginLicenseRegistry(&Registry);

    for (auto& Entry : Registry.Entries)
    {
        if (Entry.Status != ELicenseStatus::LICENSED)
        {
            // POISON: Zero out the epoch key before derivation
            // This produces a Nonce_A that is ALL ZEROS
            // which will NEVER match Expected_Gate
            FMemory::Memzero(&Epoch_N_Key, sizeof(Epoch_N_Key));
            break;
        }
    }

    // Step 2: Derive Nonce_A (possibly with poisoned key)
    // ... HMAC derivation continues normally ...
    // A poisoned key produces a valid-looking but wrong nonce
    // There is no special error path – the math just fails naturally
}

```

Why this is smart: There is no `if (invalid) return ERROR;` branch to patch out. The poisoning happens by zeroing the key BEFORE the HMAC. The attacker sees a normal HMAC computation producing a normal-looking 32-byte nonce. But the nonce is wrong. The gate check fails. The attacker doesn't know WHY it failed — was it the key? The counter? The hardware? The epoch? All paths look the same in the obfuscated code.

Temporal Binding — Why Expired Licenses Can't Package

The epoch key has a built-in expiration. When an epoch expires, what happens depends on whether the user has a `GlobalKey.glic`:

EPOCH EXPIRATION — TWO PATHS

PATH A — User HAS `GlobalKey.glic` (offline-first or exported):

1. Editor startup detects: `current_epoch > chain_depth`

2. §6.16 Automatic Renewal kicks in BEFORE packaging is attempted
3. master_seed decrypted → Epoch_(N+1)_Key derived → chain_depth updated
4. Packaging proceeds normally with the new epoch key
5. This repeats INDEFINITELY – the user can self-renew forever with zero server contact. The GlobalKey.glic never expires as long as the master_seed is accessible (valid hardware, unmodified Vault DLL, non-rolled-back counter).

PATH B – User does NOT have GlobalKey.glic (online-only user, offline):

1. The intermediate key (30-day or epoch-derived) has expired
2. The fallback key (365-day offline key) in <PluginName>.glic has expired
3. No master_seed available → cannot derive next epoch key
4. Transition function computes dead-end key:
Epoch_DEAD = HMAC-SHA256(master_seed_placeholder, "expired-epoch-dead-end")
5. Epoch_DEAD deliberately does NOT match the expected chain
6. Any nonce derived from Epoch_DEAD fails the gate check
7. User sees: "License epoch expired. Connect to the license server or export your Global Key to continue packaging."

Key insight: The Epoch_DEAD path ONLY fires when there is no master_seed available for self-renewal. Offline-first users (who have GlobalKey.glic with master_seed) NEVER hit Epoch_DEAD — they self-renew forever via §6.16.

First-Epoch Bootstrap (Offline-First Users)

Offline-first users receive their GlobalKey.glic (type=master) with a pre-seeded epoch chain starting at epoch 0:

Epoch 0: Server-signed, exported at purchase time into the GlobalKey.glic
Epoch_0_Key = HMAC-SHA256(master_seed, server_seed || "epoch-0")
Valid for: 365 days (offline-first epoch duration)

GENESIS KEY – Synthetic "Epoch -1" for Nonce_B derivation:

Problem: Nonce_B requires Epoch_N_Minus1_Key.
When N == 0, there is no previous epoch.

Solution: Derive a Genesis Key that serves as the synthetic previous epoch:

```
Epoch_Genesis_Key = HMAC-SHA256(
    key = master_seed,
    msg = epoch_anchor || server_signature || "gorgeous-epoch-genesis"
)
```

This is:

- Deterministic – same inputs always produce the same genesis key
- Distinct from real epoch keys – the message format includes "gorgeous-epoch-genesis" instead of the real pattern (epoch_anchor || N || HW_Fingerprint || "gorgeous-epoch-v1")
- Anchored to server_signature – cannot be forged without the server's ECDSA private key (ties to original purchase)
- Requires master_seed – same security as any other epoch key

For packaging on Day 1 (N == 0):

Nonce_A uses Epoch_0_Key
Nonce_B uses Epoch_Genesis_Key ← synthetic "Epoch -1"
Both are valid → packaging proceeds

For packaging on Day 366 (Epoch 0 expired – OFFLINE-FIRST USER):

```
The user has GlobalKey.glic with master_seed.
§6.16 Automatic Renewal fires on Editor startup:
1. current_epoch (1) > chain_depth (0)
2. Decrypt master_seed (triple envelope)
3. Derive Epoch_1_Key from master_seed
4. Generate new per-plugin .glic files
5. Update chain_depth = 1
Packaging now works with Epoch_1_Key.
Nonce_A uses Epoch_1_Key, Nonce_B uses Epoch_0_Key.
This repeats FOREVER – no server contact needed.
```

For packaging on Day 366 (Epoch 0 expired – ONLINE-ONLY USER, OFFLINE):

```
The user does NOT have GlobalKey.glic.
No master_seed → cannot derive Epoch_1_Key.
Epoch_0_Key → transitions to Epoch_DEAD.
Nonce_A from Epoch_DEAD → wrong → gate fails.
User must: connect to server OR export their Global Key.
```

```
For packaging after first renewal (N == 1):
  Nonce_A uses Epoch_1_Key
  Nonce_B uses Epoch_0_Key      ← real previous epoch, normal flow
  Genesis Key is no longer needed
```

FULL OFFLINE-FOREVER LIFECYCLE:

```
Day 1-365:    Epoch 0. Packaging works.
Day 366-730: $6.16 auto-renews → Epoch 1. Packaging works.
Day 731-1095: $6.16 auto-renews → Epoch 2. Packaging works.
Day 1096+:    ... and so on, indefinitely.
```

The GlobalKey.glic with master_seed enables INFINITE self-renewal.
Each epoch derives from the master_seed + epoch number + hardware.

The only things that can break the chain:

- Hardware change beyond fuzzy-match threshold (→ Recovery Phrase, §6.18)
- Tampered Vault DLL (→ Vault_Key derivation fails, Layer 2 broken)
- Counter rollback detected (→ tamper detection, §6.17)
- Revocation via Bloom filter in a Vault update (§6.20)

The user is truly sovereign. No server. No expiration. Forever.

Receipt Chain — Audit Trail for Eventual Online Sync

Every packaging event produces a signed receipt:

```
Receipt_N = {
  nonce:      Nonce_Package,
  epoch:      N,
  counter:    package_rolling_counter,
  project_id: <project>,
  build_config: <config>,
  hw_fp:      <hardware fingerprint>,
  timestamp:  now,
  prev_receipt_hash: SHA-256(Receipt_N-1), ← CHAIN LINK
  auth_tag:   HMAC-SHA256(Epoch_N_Key, all_fields_above)
}

Stored in: <ProjectDir>/Saved/GorgeousLicense/.package_receipts
```

Receipts form a **hash-linked chain** (each receipt includes the hash of the previous one). When the user eventually comes online:

- The server can download and verify the entire receipt chain
- Any gap (missing receipts), tampered receipt, or forked chain → piracy investigation flag
- Matching receipts → confirms legitimate offline packaging → no action needed

No packaging limits. A valid license = unlimited packaging. The receipt chain provides accountability without restricting the user. The server trusts the math until the user syncs, at which point it verifies. Legitimate users are never throttled.

6.20 Revocation Without Server — Eventual Consistency

Since the offline-first model has zero server contact post-purchase, revocation is handled via **passive propagation**:

Revocation Bloom Filter

```
REVOCATION BLOOM FILTER
-----
• Embedded in the Vault DLL binary at build time
• Updated with each Vault release (plugin updates via Fab)
• Contains SHA-256 hashes of revoked (user_id || master_seed)
• False positive rate: < 0.001% (sized for 100k entries)
• Lookup is O(1), adds ~12 KB to the DLL
-----

On every license validation:
1. Vault computes: probe = SHA-256(user_id || master_seed_hash)
2. Vault checks probe against bloom filter
3. If MATCH → license is revoked
   → Wipe master_seed from GlobalKey.glic (overwrite all envelopes)
   → Wipe all plugin .glic files
   → Cascading death (permanent – no recovery)
   → Toast: "Your license has been revoked. Contact support."
```

```
4. If NO MATCH → proceed normally
```

How keys get revoked:

1. Piracy is detected (same key on 50+ machines via online users' telemetry, DMCA reports, etc.)
2. The (`user_id`, `master_seed_hash`) is added to the revocation list
3. Next Vault DLL release (shipped as a normal Fab plugin update) embeds the updated bloom filter
4. When the pirate updates (or installs a project that ships a newer Vault DLL) → their key is killed

Limitations:

- Revocation only takes effect when the user installs a Vault update
- A pirate who pins an old Vault version avoids revocation — but also misses all OV/AR improvements, bug fixes, and UE version compatibility
- This is an acceptable trade-off: the offline-first model prioritizes user sovereignty over immediate revocation

Optional Community CRL

An advanced user or studio can manually download a **Certificate Revocation List**:

```
File: <ProjectDir>/Saved/GorgeousLicense/.gorgeous_crl
Format: Signed JSON with array of revoked hashes
Signed by: Server's ECDSA-P256 key (same key that signs GlobalKey.glic headers)
```

The Vault checks this file if present. It's fully optional and never downloaded automatically.

6.21 Memory Security During Renewal

When the `master_seed` is decrypted for renewal, it is the **single most sensitive moment** in the entire system. Extreme measures apply:

Phase 1 — Pre-Decryption Environment Check

BEFORE decrypting Layer 1:

1. Anti-debug sweep:
 - ↳ `IsDebuggerPresent()` / `CheckRemoteDebuggerPresent()` (Windows)
 - ↳ `ptrace(PTRACE_TRACEME, 0, 0, 0) == -1 ?` (Linux)
 - ↳ `sysctl(CTL_KERN, KERN_PROC, KERN_PROC_PID, getpid())` (macOS)
 - ↳ If debugger detected → ABORT RENEWAL (do NOT decrypt seed)
 - Use existing .glic files until they expire
 - Log: "Renewal suspended: debug environment detected."
2. VM detection:
 - ↳ CPUID leaf 0x1: ECX bit 31 (hypervisor present)
 - ↳ Check for known VM artifacts: VMware Tools, VBox Guest Additions
 - ↳ Timing check: RDTSC gap analysis (VMs have measurable overhead)
 - ↳ If VM detected → WARN but proceed (many devs use VMs legitimately)
 - Reduce master_seed exposure window to < 10ms
 - Log: "Renewal in virtualized environment – enhanced scrubbing active."
3. Memory dump tool detection:
 - ↳ Scan process list for known dumpers: procdump, MiniDumpWriteDump hooks
 - ↳ Check for: PAGE_GUARD watchpoints on the Vault's memory pages
 - ↳ If detected → ABORT RENEWAL
4. Integrity re-verification:
 - ↳ Re-run .text section hash check (§5.2) – ensure Vault hasn't been patched since initial load

Phase 2 — Seed Handling Protocol

DURING master_seed decryption (< 50ms target):

1. Allocate seed buffer from a DEDICATED, LOCKED memory page:
 - ↳ `VirtualAlloc` with `PAGE_READWRITE` | `PAGE_GUARD` (Windows)
 - ↳ `mmap` with `PROT_READ` | `PROT_WRITE` + `mlock()` (Linux)
 - ↳ Prevents the page from being swapped to disk
2. Decrypt seed into the locked page (NOT on the stack, NOT on the heap)
3. Perform ALL derivations (`Epoch_Key`, `Plugin_License_Keys`) immediately

4. Wipe protocol (mandatory, in this exact order):
 - ↳ Step A: Overwrite seed buffer with crypto-random bytes (32 bytes)
 - ↳ Step B: Overwrite seed buffer with 0xFF bytes
 - ↳ Step C: Overwrite seed buffer with 0x00 bytes
 - ↳ Step D: Compiler memory barrier:
 - _ReadWriteBarrier() (MSVC)
 - asm volatile("" ::: "memory") (GCC/Clang)
 - ↳ Step E: VirtualFree / munmap the locked page
 - ↳ Step F: Overwrite ALL derived keys (Epoch_Key, Plugin_License_Keys) using the same A→B→C→D sequence
5. Total exposure: master_seed exists in any form for < 50ms (benchmark on target hardware; if exceeded, log warning)

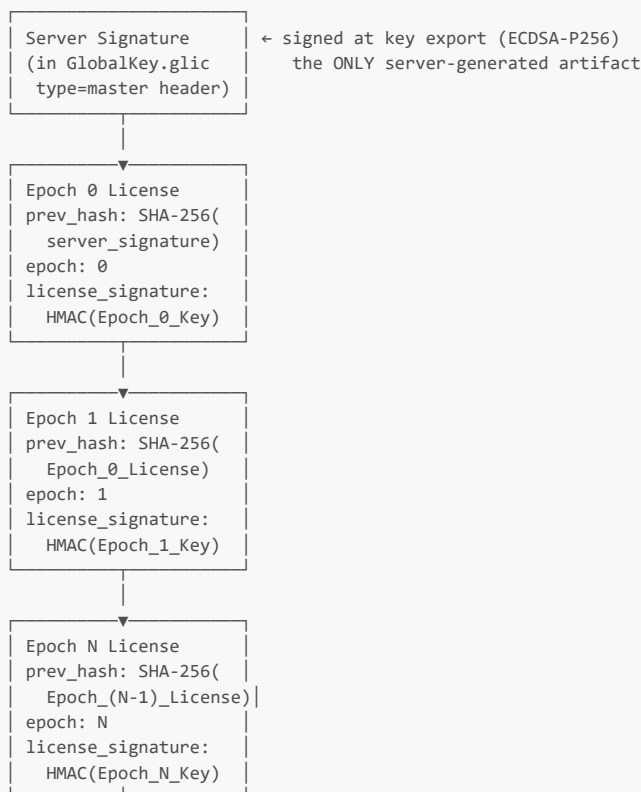
Phase 3 — Decoy Countermeasures

ALONGSIDE the real seed (in adjacent memory):

- 15 DECOY SEEDS: randomly generated 256-bit values
 - ↳ Stored in the same locked page, interleaved with the real seed
 - ↳ The Vault knows which is real via an obfuscated index (index = HMAC-SHA256(Vault_Key, "decoy-selector") mod 16)
 - ↳ All 16 values are wiped identically after use
 - ↳ A memory dump shows 16 identical-looking 256-bit values
 - the attacker cannot distinguish the real seed
- DECOY DERIVATIONS: the Vault performs HMAC derivations on ALL 16 values
 - ↳ Only the results from the real seed are used
 - ↳ The decoy results are discarded
 - ↳ Prevents timing-based side-channel attacks (all 16 computations take the same time)

6.22 Epoch Chain Integrity — Hash Chain Verification

Each epoch's license files form a **hash chain** rooted in the server's original signature:



Verification on startup:

1. The Vault checks the `GlobalKey.glic` header's `server_signature` (ECDSA-P256, baked-in public key)
2. For the current epoch's per-plugin `.glic` files, the Vault verifies: a. `license_signature` matches `HMAC(Epoch_N_Key, license_fields)` b. `prev_epoch_hash` matches `SHA-256(Epoch_(N-1)_License)` (stored in chain state)

3. If the chain is broken → license invalid → cascading death

The chain is anchored to the server's original signature. This means the self-renewal system produces licenses that are cryptographically traceable back to the original purchase. A forged license would need the `master_seed` (triple-envelope-protected) AND the server's signature on the header (ECDSA — cannot be forged without the server's private key).

6.23 Trust Model — Offline-First

WHO HOLDS WHAT		
Entity	Holds	Can Do
Server (key export request)	Server private key User purchase records Revocation list	Sign GlobalKey.glic hdrs Issue Recovery Phrase Populate bloom filter ✗ Cannot revoke instantly
User's Machine	GlobalKey.glic Per-plugin .glic Shadow nonce file	Self-renew via epoch chain Self-package (local nonce) Detect rollback/tampering ✗ Cannot forge new plugins ✗ Cannot bypass revocation
User (human)	Recovery Phrase Purchase receipt	Hardware migration Contact support (last resort) ✗ Cannot add plugins not in purchased_plugins[]
Vault DLL (obfuscated)	Derivation logic ECDSA public key Build nonce Bloom filter Internal salt	Decrypt/re-encrypt seed Derive epoch keys Verify chain integrity Check revocation Layer 2 binding ✗ Cannot issue NEW keys for different users
Attacker	(nothing useful alone)	Must compromise ALL of: • Correct hardware (L1) • Genuine Vault DLL (L2) • Current temporal state (L3) • Recovery phrase (for migration) + Reverse white-box crypto in obfuscated Vault code

6.24 License Priority — Updated Resolution Order

With the offline-first model, the resolution order becomes:

```
FOR EACH discovered sibling plugin (e.g., "GorgeousEntertaining"):  
  
0. Is plugin in effective entitlement list?  
  (= GlobalKey.glic purchased_plugins ∪ all valid type=patch .glic add_plugins)  
  |→ YES → Continue to step 1  
  |→ NO  → Jump to §6.25.4 (UNENTITLED resolution – no cascading death)  
  
1. GlobalKey.glic (type=master) exists?  
  |→ YES → Derive Epoch_N_Key → validate/renew → writes primary slot in plugin .glic  
  |      (fully offline, self-sovereign)  
  |→ NO  → Fall through to standard model ↓  
  
2. <PluginName>.glic PRIMARY SLOT (online 30-day key OR epoch-derived)  
  ↓ invalid/expired?  
3. Try online refresh (if user opted into online model)  
  ↓ failed/no network/user chose offline-first?  
4. <PluginName>.glic FALLBACK SLOT (365-day offline key from §6.8)  
  ↓ invalid/expired?  
5. Prompt user: "Enter Recovery Phrase, offline key, or connect to server"  
  ↓ dismissed?  
6. Mark plugin INVALID in Plugin License Registry
```

AFTER ALL PLUGINS EVALUATED:

└ ALL plugins LICENSED → ☒ OV/AR functions enabled

└ Some plugins UNENTITLED, rest LICENSED → ☒ UNENTITLED plugins disabled individually

└ ANY entitled plugin INVALID/MISSING/EXPIRED → ☒ CASCADING DEATH

Step 0 is new. It redirects plugins that aren't in the entitlement list to the UNENTITLED flow (\$6.25), preventing cascading death for newly-installed plugins that the user has purchased but not yet patched into the offline key. See \$6.25 for the entitlement patch mechanism (type=patch .glic files) and full resolution details.

The offline-first `GlobalKey.glic` (type=master) path takes **absolute priority**. If the user has a Global License Key, the Vault never touches the network.

6.25 Entitlement Expansion — Handling New Plugin Purchases While Offline

PROBLEM: A user is in offline-first mode with a `GlobalKey.glic` (type=master) that lists `purchased_plugins: ["GorgeousEntertaining"]`. They then purchase `GorgeousEvents`. The `GlobalKey.glic` header is **ECDSA-signed by the server** — it cannot be locally modified. Installing the new plugin would make it appear as an unlicensed sibling, which under the old rules would trigger cascading death and kill *all* plugins.

Purchasing is inherently online (the user contacts a store/server), but the **development machine may be air-gapped**. The purchase might happen from a phone, a different computer, or a web browser — not from the machine running the editor.

6.25.1 The UNENTITLED Plugin State

A new license registry state, **distinct from INVALID**:

State	Meaning	Triggers Cascading Death?
Licensed	Valid license (online or offline)	No — plugin is valid
Unentitled	Plugin exists but is NOT in <code>GlobalKey.glic purchased_plugins</code> AND no type=patch .glic covers it AND no online license exists	No — only this plugin is individually disabled
Invalid	License exists but validation failed (corrupted, tampered, wrong hardware)	Yes
Missing	An entitled plugin (in <code>purchased_plugins</code>) has no .glic file on disk	Yes
Expired	License expired and all renewal paths (epoch chain, online refresh, offline fallback) failed	Yes

Key distinction: Cascading death fires when an **entitled plugin** (one the user owns per `GlobalKey.glic` + patches) fails validation. It does **NOT** fire for plugins that simply aren't in the entitlement list yet.

When a plugin is UNENTITLED:

- Its modules are registered with `UGorgeousPluginHelper` but all OV/AR function pointers for that plugin are nulled
- Non-OV code (utility functions, UI, data types) continues to work
- The plugin appears in the Gorgeous Plugin Manager with a clear **"Not Licensed — Update your offline key"** indicator
- A toast is shown: *"GorgeousEvents is not covered by your offline license. Update via Tools → Gorgeous → License Manager, or switch to online mode."*
- The user can resolve it **without restarting the editor** — same as cascading death recovery

This is critical because:

- Installing a newly-purchased plugin should **never** break working plugins
- The user may be mid-development and cannot afford all plugins going dark
- It clearly communicates that an update is needed, rather than punishing the user

6.25.2 Entitlement Expansion Patch (type=patch .glic)

A lightweight, server-signed **delta file** that adds new plugins to the `GlobalKey.glic` entitlement without re-exporting the entire master key.

File Format

Note: The canonical structure for the type=patch .glic file is defined in \$6.0.1. The diagram below is the \$6.25 reference for context.

Gorgeous License — type: patch (Patches/2026-03-15_GorgeousEvents.glic)

COMMON HEADER

• magic: "GLIC"

• format_version: 4

• type: "patch"

• issued_at: 2026-03-15T12:00:00Z

• user_id: "usr_a1b2c3d4..."

PATCH HEADER EXTENSION

• parent_master_hash: SHA-256(GlobalKey.glic common + master headers)

```

• add_plugins:      ["GorgeousEvents"]
• effective_plugins: ["GorgeousEntertaining", "GorgeousEvents", ...]
• server_signature: ECDSA-P256(SHA-256(all_header_fields_above))

```

(no encrypted body – the patch IS its header + signature)

- **parent_master_hash**: SHA-256 of the current `GlobalKey.glic` (type=master) **header bytes** (common header + master header extension, not the entire file). This **ties the patch to a specific user's master key** — cannot reuse someone else's patch.
- **user_id**: Must match the `GlobalKey.glic user_id`. Redundant check.
- **add_plugins**: The new plugins being added by this patch.
- **effective_plugins**: The **full** entitlement list after applying this patch. Allows the Vault to verify consistency.
- **server_signature**: ECDSA-P256 using the same server private key that signs `GlobalKey.glic` headers. Verified against the public key baked into the Vault DLL.

Validation Rules

1. **server_signature** must verify against the baked-in server public key
2. **parent_master_hash** must match `SHA-256(current GlobalKey.glic common + master headers)`
3. **user_id** must match `GlobalKey.glic user_id`
4. Patches are **additive only** — the **add_plugins** list cannot remove existing entitlements
5. **effective_plugins** must equal `GlobalKey.glic.purchased_plugins ∪ add_plugins` (from this patch and all previous valid patches)
6. Multiple patch `.glic` files can coexist — one per expansion event, applied in **issued_at** order

Storage

```

<ProjectDir>/Saved/GorgeousLicense/
├─ GlobalKey.glic           ← type=master
├─ GorgeousEntertaining.glic ← type=plugin
├─ Patches/
│   ├─ 2026-03-15_GorgeousEvents.glic ← type=patch
│   └─ 2026-06-01_GorgeousFX.glic    ← type=patch

```

How to Obtain a Patch (type=patch .glic)

Option A — Automatic (if dev machine has network):

1. User purchases a new plugin (via Fab or web store)
2. Opens **Tools** → **Gorgeous** → **License Manager** → **Check for Updates**
3. License Manager detects new entitlement on the server
4. Downloads patch `.glic` automatically
5. Vault re-evaluates → plugin transitions from **UNENTITLED** to **Licensed**

Option B — Manual transfer (air-gapped machine):

1. User purchases the plugin from any device
2. On the purchase confirmation page (or via web portal), clicks **"Download Entitlement Patch"**
3. Server generates a type=patch `.glic` file tied to the user's `GlobalKey.glic`
4. User transfers the patch `.glic` file to the dev machine via USB, email, or any file transfer
5. Places it in `<ProjectDir>/Saved/GorgeousLicense/Patches/`
6. Next editor startup (or License Manager refresh) → plugin is licensed

Option C — API (CI/automation):

```

POST /api/v1/license/expansion-patch
Body: {
  "intermediate_key": "<valid JWT>",
  "master_header_hash": "<SHA-256 of current GlobalKey.glic header>",
  "new_plugins": ["GorgeousEvents"]
}
Response: {
  "glic_patch": "<base64-encoded type=patch .glic>",
  "filename": "2026-03-15_GorgeousEvents.glic"
}

```

6.25.3 Full Re-export (Alternative to Patches)

The user can always re-export the **entire** `GlobalKey.glic` (type=master) from the server:

1. **Tools** → **Gorgeous** → **License Manager** → **Re-export Global Key**
2. Server generates a new `GlobalKey.glic` with updated **purchased_plugins** (all currently owned)

3. **Same master_seed** — epoch chain continuity is preserved, no reset
4. **Same epoch_anchor** — the epoch numbering doesn't change
5. **New server_signature** over the updated header
6. Replaces the old `GlobalKey.glic` on disk
7. **All patch .glic files in Patches/ are deleted** — they're now absorbed into the new master key

This is the preferred path when the user is online. The type=patch .glic mechanism exists specifically for cases where the dev machine is air-gapped and only the patches can be transferred.

6.25.4 Resolution Order — Plugin Not in purchased_plugins

When a sibling plugin is discovered but is NOT in the `GlobalKey.glic purchased_plugins` list:

```
Sibling plugin "GorgeousEvents" discovered
NOT in GlobalKey.glic purchased_plugins

1. Scan Patches/ directory for type=patch .glic files
├─ Found → Verify each:
│   ├── server_signature valid?
│   ├── parent_master_hash matches current GlobalKey.glic header?
│   ├── user_id matches?
│   └─ ALL checks pass AND patch covers "GorgeousEvents"?
│       ├── YES → Add "GorgeousEvents" to effective entitlement list
│       │       └─ Proceed to epoch key derivation (same as §6.24 step 1)
│       └─ Invalid / tampered / doesn't cover this plugin → Ignore, fall through ↓
└─ No valid patches cover "GorgeousEvents" → Fall through ↓

2. Mixed model check: does user also have online credentials?
├─ YES → Attempt online license check for "GorgeousEvents"
│   ├── Valid → Mark LICENSED (online)
│   └─ Invalid / no license purchased / network error → Fall through ↓
└─ NO (purely offline-first) → Fall through ↓

3. Mark plugin as UNENTITLED
→ Notify UGorgeousPluginHelper: SetPluginLicenseState("GorgeousEvents", UNENTITLED)
→ UGorgeousPluginHelper disables plugin's OV/AR function pointers (plugin-scoped only)
→ Show toast notification with remediation steps
→ DO NOT trigger cascading death
→ Continue evaluating remaining plugins normally
```

6.25.5 UGorgeousPluginHelper — Required New Functionality

The `UGorgeousPluginHelper` singleton needs the following extensions to support entitlement-aware plugin management:

```
// ===== New enum for license-level state (distinct from EGorgeousPluginLoadState) =====

UENUM(BlueprintType)
enum class EGorgeousPluginLicenseState : uint8
{
    // Valid license – online or offline (epoch-derived)
    Licensed,
    // Plugin installed but not in GlobalKey.glic purchased_plugins and no type=patch .glic covers it
    // Individually disabled – does NOT cascade
    Unentitled,
    // License file exists but validation failed (corrupted, tampered, wrong hardware)
    // TRIGGERS cascading death
    Invalid,
    // Plugin is in purchased_plugins but no .glic file found on disk
    // TRIGGERS cascading death (something was deleted or never generated)
    Missing,
    // License expired and all renewal paths (epoch chain, online, offline fallback) failed
    // TRIGGERS cascading death
    Expired,
    // Not yet evaluated (initial state)
    Unknown
};

// ===== New methods =====

/**
 * Sets the license state for a specific plugin. Called by the Vault after license evaluation.
 * When set to Unentitled, the plugin's OV/AR function pointers are individually nulled
 * without triggering cascading death.
 */
```



```
void SetPluginLicenseState(const FName& PluginName, EGorgeousPluginLicenseState State);

/**
 * Gets the license state for a specific plugin.
 */
EGorgeousPluginLicenseState GetPluginLicenseState(const FName& PluginName) const;

/**
 * Returns true if the plugin has a valid license (Licensed state).
 */
bool IsPluginEntitled(const FName& PluginName) const;

/**
 * Returns all plugins currently in the UNENTITLED state.
 * Used by the License Manager UI to display "Not Licensed" plugins.
 */
TArray<FName> GetUnentitledPlugins() const;

/**
 * Returns all plugins that are causing or would cause cascading death.
 * Used by the License Manager UI to display "Fix Required" plugins.
 */
TArray<FName> GetCascadeViolationPlugins() const;

/**
 * Re-evaluates a single plugin's license state after the user
 * adds a type=patch .glic or enters an online key. Allows resolution
 * without restarting the editor.
 */
void ReevaluatePluginLicense(const FName& PluginName);

// ===== New member variables =====

// Per-plugin license state (distinct from module load state)
TMap<FName, EGorgeousPluginLicenseState> PluginLicenseStates;
```

Important: `EGorgeousPluginLicenseState` is separate from `EGorgeousPluginLoadState`. A plugin can be `FullyLoaded` (all modules registered) but `Unentitled` (no license → OV/AR disabled). The two enums track orthogonal concerns: **module health** vs. **license status**.

6.25.6 Gorgeous Plugin Manager — UI Integration

The Gorgeous Plugin Manager (accessible via `Tools` → `Gorgeous` → `License Manager`) must display:

GORGEOUS LICENSE MANAGER

License Model: [Offline-First ▼] Global Key: ☒ Valid
Epoch: 0 (362 days remaining) Nonce Counter: 47

Plugin	Status	Action
GorgeousCore	● Free	—
GorgeousEntertaining	● Licensed	View Details
GorgeousEvents	● Not Licensed	[Add Patch] [Buy]
GorgeousFX	● Invalid	[Fix] [Re-export]

[Check for Updates] [Re-export Global Key] [Enter Recovery Phrase]

⚠ GorgeousFX license is INVALID — Cascading death active.
All OV/AR functionality is disabled until resolved.

ⓘ GorgeousEvents is not covered by your offline license.
Add an entitlement patch or re-export your Global Key to enable it.

Color coding:

- Free / Licensed — no action needed
- Not Licensed (UNENTITLED) — plugin is disabled but does not cascade
- Invalid / Missing / Expired — **cascading death active**, action required

"Add Patch" button flow:

- 1. Opens file picker → user selects type=patch .glic file
- 2. Vault verifies signature, parent hash, user ID
- 3. If valid → copies to Patches/ directory → re-evaluates → plugin becomes Licensed
- 4. If invalid → error toast with reason

6.25.7 Edge Cases

Scenario	Handling
User installs a plugin they NEVER purchased (piracy)	Plugin stays UNENTITLED indefinitely. No cascading death, but no OV/AR either. Plugin is essentially a hollow shell.
User has a patch .glic but later re-exports GlobalKey.glic	Re-export absorbs all patches → Patches/ directory is cleared. New GlobalKey.glic has the full purchased_plugins list.
Patch .glic file is corrupted or tampered	Signature verification fails → patch is ignored → plugin remains UNENTITLED. Toast: "Patch file invalid. Re-download from the web portal."
Multiple patches for the same plugin	Only the most recent valid patch (by issued_at) is used. Older patches are ignored but not deleted.
User re-exports GlobalKey.glic with fewer plugins than before (downgrade)	New purchased_plugins is authoritative. If a plugin was in the old master key but not the new one, it becomes UNENTITLED. The server should warn the user before issuing a reduced master key.
Patch .glic references a different GlobalKey.glic (wrong parent_master_hash)	Patch is rejected. The user needs to download a patch generated for their current GlobalKey.glic. After a re-export, old patches are invalid (different header hash).
User switches from offline-first back to online model	GlobalKey.glic and patches remain on disk (still valid). Online licenses take priority per §6.24. The UNENTITLED state only applies when no license source covers the plugin.

6.26 Team Detection & Per-Seat License Sharing Enforcement

This is NOT about piracy. Piracy is when a license is stolen and distributed to strangers. This section addresses a subtler problem: **a single legitimate license being shared across multiple team members working on the same project.** Fab's Terms of Service require per-user (per-seat) licensing. This system detects sharing and pressures teams into compliance — without punishing designers, artists, or other team members who don't actively develop with the OV/AR systems.

6.26.1 The Problem

SCENARIO: License Sharing (NOT Piracy)

Studio "Acme Games" buys 1 license for GorgeousEntertaining.
Project ID: E0C3F3454AA9F4309CCBD6829FE5C95E

Developer A: Uses the license key, installed on Machine_A

Developer B: Copies the same .glic file to Machine_B

Developer C: Also copies it to Machine_C

Designer D: Has the plugin enabled, only places actors in levels

Artist E: Never opens any Gorgeous panel, just works on art

← licensed

← SHARING

← SHARING

← NOT sharing (doesn't develop)

← NOT sharing (doesn't use it)

Result: 3 developers actively use the OV/AR system.
Only 1 license was purchased.
Fab ToS requires 3 licenses.

But we DON'T want to penalize Designer D or Artist E.

6.26.2 Usage Tier Classification — Who Needs a License?

The Vault monitors API call patterns to classify each user's engagement level. This classification determines whether they need their own license.

Tier	Name	Description	Needs Own License?	Detection Method
T0	Non-User	Plugin is installed/enabled because the project depends on it, but this user never touches any Gorgeous API surface. No OV created, no AR configured, no Vault function called.	NO	Zero Vault API calls during the session. Only UHT-generated boilerplate executes (CDO construction, etc.).
T1	Indirect User	User interacts with Gorgeous-built systems at the design level: placing actors that contain OVs in levels, editing exposed properties in Details panels, running PIE where OV values replicate. They don't create OVs, write Blueprint logic with OVs, or configure AR.	NO	Only read-path Vault calls: GVault_OV_GetValue, GVault_OV_Serialize (for level saves). Zero write-path calls to GVault_OV_Create, GVault_OV_SetProperties, GVault_AR_*_Bind, GVault_AR_*_Configure.

Tier	Name	Description	Needs Own License?	Detection Method
T2	Active Developer	User creates Object Variables, writes Blueprint/C++ logic that creates or configures OVs, sets up Auto Replication bindings, modifies Gorgeous subsystem settings, or calls any Vault export that alters OV/AR state.	YES	Any write-path Vault call: <code>GVault_OV_Create</code> , <code>GVault_OV_SetProperties</code> , <code>GVault_OV_RegisterChange</code> , <code>GVault_AR_Mixin_Bind</code> , <code>GVault_AR_Coordinator_RegisterVariable</code> , <code>GVault_AR_Graph_RegisterObjectVariable</code> . Also: opening Gorgeous editor panels that modify configuration (detected via <code>GVault_Editor_*</code> calls).

6.26.3 Usage Tier Detection — Client-Side Telemetry

The Vault tracks a lightweight **Usage Fingerprint** per editor session:

USAGE FINGERPRINT (per session, computed inside the Vault)

Counters (accumulated during editor session):

ov_create_count

= 0

(# of `GVault_OV_Create` calls)

ov_set_props_count

= 0

(# of `GVault_OV_SetProperties` calls)

ov_register_change

= 0

(# of `GVault_OV_RegisterChange` calls)

ar_bind_count

= 0

(# of `GVault_AR_*_Bind` calls)

ar_configure_count

= 0

(# of `GVault_AR_*_Configure` calls)

ar_register_var_count

= 0

(# of AR variable registration calls)

editor_panel_modify

= 0

(# of Gorgeous editor config changes)

ov_get_value_count

= 0

(# of read-only OV accesses)

ov_serialize_count

= 0

(# of serialization calls)

Tier Classification:

IF all counters == 0

→ T0 (Non-User)

ELIF write counters all == 0

AND (ov_get_value_count > 0

→ T1 (Indirect User)

OR ov_serialize_count > 0)

ELSE

→ T2 (Active Developer)

The fingerprint is:

- NOT stored to disk (computed fresh each session)

- Sent to the server ONLY during license renewal/heartbeat

- Consent-gated (§6.7) — included in the data disclosure dialog

- HMAC-signed with `Epoch_N_Key` to prevent client-side spoofing

Anti-Spoofing: The Usage Fingerprint is computed inside the Vault DLL (obfuscated), not in open-source proxy code. An attacker cannot patch the Proxy to report T0/T1 when they're actually T2 because:

- The Vault internally counts its own API calls — it knows which exports were invoked and how many times
- The HMAC is computed over the raw counter values using the epoch key (inside the Vault) — the Proxy never sees the raw data
- Spoofing requires modifying the Vault DLL itself, which triggers DLL signing (§5.0) / integrity (§5.2) / Gordian Parasite (§9.4) failures

6.26.4 License Sharing Detection — Server-Side

The license server detects sharing by correlating heartbeat/renewal data:

SHARING DETECTION ALGORITHM (runs server-side on each heartbeat/renewal)

FOR EACH (user_id, plugin_name, project_id) tuple:

1. Collect all hardware_fingerprints seen in the last 30 days

2. Collect all usage_tier reports for each fingerprint

3. Count DISTINCT hardware_fingerprints where usage_tier == T2:
= active_developer_count

4. IF active_developer_count <= 1:
→ PASS — single developer, no sharing detected

5. IF active_developer_count > 1:
→ SHARING DETECTED
→ This license is being used by multiple active developers on the same project from different machines.
→ Server flags: license_sharing = true

→ Server records: sharing_fingerprints = [list of T2 hw_fps]
→ Enforcement begins (§6.26.7)

NOTE: Fingerprints with usage_tier T0 or T1 are IGNORED.
Designers and artists do not trigger sharing detection.

Important distinction from piracy detection (§6.6):

	Piracy	License Sharing
What	Same license used on 50+ random machines	Same license used by 2-5 developers on the same project
Who	Strangers / internet distribution	Legitimate team members
Intent	Theft	Cost-saving / ignorance of ToS
Response	Revocation (permanent)	Pressure to buy seats (recoverable)
Threshold	≥5 hardware fingerprints across ANY projects	≥2 T2 fingerprints on the SAME project

6.26.5 Automatic Team Detection

When multiple **different** `user_ids` access the server with the **same** `project_id`, they are automatically grouped into a **Team**.

TEAM FORMATION (server-side, automatic)

Server receives heartbeat/renewal from:
User A: { user_id: "alice", project_id: "PROJ_XYZ", hw_fp: "aaa", tier: T2 }
User B: { user_id: "bob", project_id: "PROJ_XYZ", hw_fp: "bbb", tier: T2 }
User C: { user_id: "carol", project_id: "PROJ_XYZ", hw_fp: "ccc", tier: T1 }
User D: { user_id: "dave", project_id: "PROJ_XYZ", hw_fp: "ddd", tier: T0 }

Server groups: Team "PROJ_XYZ" = { alice(T2), bob(T2), carol(T1), dave(T0) }

Team is created automatically on first multi-user project detection.
No user action required.

Team properties:

Property	Value
Team ID	Deterministic hash: <code>SHA-256(project_id sorted_user_ids)</code>
Members	All <code>user_ids</code> that have contacted the server with this <code>project_id</code>
Active Developers	Members with <code>usage_tier == T2</code> (need their own license)
Indirect Users	Members with <code>usage_tier == T1</code> (do NOT need their own license)
Non-Users	Members with <code>usage_tier == T0</code> (do NOT need their own license)
Auto-Created	Yes — no manual setup. Appears in Online License Manager automatically.
Manageable	Yes — team admin (first T2 user, or explicitly assigned) can rename team, remove stale members, escalate/de-escalate tiers via the Online License Manager web portal.

6.26.6 Online License Manager — Team Management UI

The **Online License Manager** (web portal at <https://gorgeous.simsalabim.studio/teams>) provides:

GORGEOUS ONLINE LICENSE MANAGER — TEAM VIEW
⚡ Independent — Not Subject to Epic Nova Terms

Team: "Acme Games — Project Starfall"
Project ID: E0C3F3454AA9F4309CCBD6829FE5C95E
Status: ⚠ LICENSE SHARING DETECTED (2 seats needed, 1 purchased)

Member	Usage Tier	License Status	Action
alice (admin)	T2 🛠️ DEV	☑ Valid (own key)	
bob	T2 🛠️ DEV	✗ SHARING alice's	[Buy Seat]
carol	T1 🎨 Design	— (not required)	
dave	T0 👤 None	— (not required)	

```

Required seats: 2 (alice + bob are both T2)
Purchased seats: 1 (only alice has a valid personal license)
Deficit: 1 seat

[Buy Additional Seats] [Manage Members] [Override Tier (admin only)]

```

Admin capabilities:

- Rename team (cosmetic label for their own organization)
- Remove stale members (users who left the studio)
- **Override tier** (admin can manually mark a user as T1 instead of T2 if they believe the automated tier is wrong — e.g., a developer who ran a quick test but doesn't actually work with Gorgeous regularly). Override is time-limited (30 days) and must be re-confirmed. Abuse of overrides is detectable server-side (a user overridden to T1 but whose next heartbeat shows T2 activity → override revoked automatically).
- View sharing history and seat recommendations
- Purchase additional seats directly from the portal
- **View registered hardware fingerprints** per member (see §6.30 License Transparency Dashboard)

6.26.7 Enforcement — Graduated Pressure System

When license sharing is detected, enforcement is **graduated**, not immediate:

ENFORCEMENT ESCALATION TIMELINE

```

Day 0:  SHARING DETECTED
    |
    |→ Phase 1 – SOFT WARNING (14 days)
    |   • Toast in editor for ALL T2 team members:
    |     "License sharing detected on project [ProjectName].
    |     [X] developers are using [Y] license(s) for [PluginName].
    |     Each active developer needs their own seat.
    |     Please purchase additional licenses within 14 days.
    |     Tools → Gorgeous → License Manager → Team"
    |   • Toast appears once per session (not spammed)
    |   • Full OV/AR functionality remains operational
    |   • No penalties yet
    |
Day 14: |→ Phase 2 – HARD WARNING (7 days)
    |   • Persistent banner in editor (not dismissable):
    |     "⚠ License sharing enforcement begins in [X] days.
    |     Intermediate key renewals will be REJECTED for this team
    |     until all active developers have valid licenses."
    |   • Server sends email to team admin (if email consent given)
    |   • Full OV/AR functionality still operational
    |
Day 21: |→ Phase 3 – RENEWAL REJECTION
    |   • Server REJECTS intermediate key renewal requests for
    |     ALL T2 team members on this project.
    |   • Response: HTTP 403 { "error": "team_sharing_violation",
    |     "required_seats": 3, "purchased_seats": 1,
    |     "resolution_url": "https://gorgeous.simsalabim.studio/teams/..."
    |   }
    |   • Existing intermediate keys continue working until they
    |     expire (~30 days from last renewal)
    |   • When the intermediate keys expire:
    |     → License resolution falls through to offline/fallback keys
    |     → If fallback also expires → cascading death
    |   • T1 and T0 users are NOT affected (their renewals still succeed)
    |
Day 51+: |→ Phase 4 – CASCADING DEATH (natural expiry)
    |   • All intermediate keys have now expired
    |   • Fallback/offline keys may also expire
    |   • Cascading death triggers naturally – OV/AR disabled
    |   • T1/T0 users still work fine (they don't use OV/AR directly)
    |
    |→ Resolution: Purchase missing seats
    |   • Team admin (or any member) buys the deficit seats
    |   • Assigns seat keys to each T2 developer
    |   • Server clears the sharing flag
    |   • Renewal requests succeed again within 1 minute
    |   • Full recovery – no permanent damage

```

Why rejection targets renewals, not active sessions:

- Immediately killing an active session mid-work would be hostile and could cause data loss
- Rejecting renewals is PASSIVE — existing keys keep working until natural expiry
- This gives the team a 30-day window AFTER Phase 3 starts to resolve the issue
- Total warning window: 21 days (soft + hard) + 30 days (key expiry) = **51 days** before real impact

6.26.8 License Classification Algorithm — Single Dev vs. Team vs. Piracy

When the server detects that a single `license_key_id` is being used from multiple hardware fingerprints (§6.26.4), it does NOT immediately assume sharing. Instead, it runs a **three-way classification algorithm** that distinguishes:

Classification	Description	Enforcement
SINGLE_DEV	One person, multiple machines (home desktop + work laptop)	None — legitimate use, covered by hardware migration (§6.18)
TEAM	Multiple developers, same project, sharing seats	Graduated pressure (§6.26.7) — buy more seats
PIRACY	Unauthorized redistribution across unrelated projects/users	Immediate revocation via Bloom filter (§6.20)

6.26.8.1 Feature Extraction

For a given `license_key_id`, the server computes the following feature vector over a **rolling 30-day window**:

```
FEATURE VECTOR (per license_key_id, 30-day rolling window):

F1: project_diversity
  = |{ distinct project_ids that used this key }|
  Meaning: How many unrelated projects is this key appearing in?

F2: pattern_similarity
  = mean( pairwise_cosine_similarity(
    usage_fingerprint_vectors for all distinct hw_fps ) )
  Meaning: Do the machines using this key exhibit the same work patterns?
  Range: 0.0 (completely different) to 1.0 (identical)

F3: machine_count
  = |{ distinct hw_fps that used this key }|
  Meaning: How many unique machines have used this key?

F4: growth_rate
  = |{ hw_fps seen for FIRST TIME in current 30d window }|
    / max( |{ hw_fps from previous 30d window }|, 1 )
  Meaning: How fast are new machines appearing relative to the
    established machine base? A stable set yields ~0.0.
    Explosive growth yields >> 1.0.

F5: session_overlap_ratio
  = |{ 1-hour buckets where >1 distinct hw_fp had activity }|
    / |{ 1-hour buckets where ANY hw_fp had activity }|
  Meaning: How often are multiple machines active simultaneously?
  Range: 0.0 (never simultaneous) to 1.0 (always simultaneous)

F6: hw_churn_rate
  = |{ hw_fps seen ≤ 2 times in the window and never again }|
    / max( |{ all hw_fps in window }|, 1 )
  Meaning: What fraction of machines appear briefly and vanish?
  Range: 0.0 (all machines ARE regulars) to 1.0 (all are one-offs)

F7: geo_spread
  = |{ distinct country codes from request IP geolocation }|
  Meaning: How geographically distributed are the requests?
```

Usage Fingerprint Vector (the input to F2's cosine similarity) is defined in §6.26.3 and consists of 8 dimensions per (hw_fp, 30-day window):

```
vec = [
  ov_create_count,      // Object Variable creation events
  ov_set_props_count,   // Property mutation events
  ar_bind_count,        // Auto Replication bind events
  ar_unbind_count,      // Auto Replication unbind events
  blueprint_node_count, // Blueprint node placement events
  cpp_api_call_count,   // Direct C++ API call events
  session_count,        // Number of distinct editor sessions
  session_avg_duration  // Average session duration in minutes
]

cosine_similarity(A, B) = dot(A, B) / (|A| · |B|)
```

Why cosine similarity? It measures the *shape* of the work pattern, not the *magnitude*. A developer who works part-time and a developer who works full-time on the same tasks will have high similarity (~1.0) even though their raw counts differ. This correctly identifies one person working different hours on two machines.

6.26.8.2 Classification Decision Tree

CLASSIFY(license_key_id):

Extract features F1..F7 for the key over 30-day window.

STAGE 1: Piracy Detection (checked first – highest threat)

RULE P1 – Project Explosion:

IF F1 (project_diversity) >= THRESH_PROJECT_PIRACY (default: 5):
 → return PIRACY
 Rationale: A single developer or team works on 1 project (maybe 2 with a prototype). 5+ completely different project_ids from the same key is overwhelmingly redistribution.

RULE P2 – Machine Explosion:

IF F3 (machine_count) >= THRESH_MACHINE_EXPLOSION (default: 20)
 AND F4 (growth_rate) > THRESH_GROWTH_RATE (default: 3.0):
 → return PIRACY
 Rationale: 20+ machines AND the count tripled in the last window? No legitimate team grows that fast.

RULE P3 – Fingerprint Churn:

IF F6 (hw_churn_rate) > THRESH_CHURN (default: 0.70)
 AND F3 (machine_count) >= THRESH_CHURN_MIN_MACHINES (default: 10):
 → return PIRACY
 Rationale: 70%+ of machines appear briefly and vanish, across 10+ machines? Classic redistribution pattern – people download a cracked copy, use it once, and move on.

RULE P4 – Geographic Explosion:

IF F7 (geo_spread) >= THRESH_GEO_PIRACY (default: 10)
 AND F3 (machine_count) >= THRESH_GEO_MIN_MACHINES (default: 15):
 → return PIRACY
 Rationale: 10+ countries across 15+ machines? Even large distributed teams rarely span that many countries.

RULE P5 – Compound Piracy Signal:

```
piracy_score = 0
IF F1 >= 3:           piracy_score += 2    // multiple projects
IF F3 >= 10:          piracy_score += 1    // many machines
IF F4 > 2.0:          piracy_score += 2    // fast growth
IF F6 > 0.50:         piracy_score += 2    // high churn
IF F7 >= 5:           piracy_score += 1    // geo spread
```

IF piracy_score >= THRESH_COMPOUND_PIRACY (default: 5):
 → return PIRACY
 Rationale: No single signal hit the threshold, but the COMBINATION of weak signals exceeds the compound threshold. This catches sophisticated pirates who stay just below individual limits.

STAGE 2: Single Dev vs. Team (piracy ruled out)

At this point we know: modest machine count, stable growth, low churn, same project. Now determine: one person or many?

RULE S1 – High Pattern Similarity + Low Temporal Overlap:

IF F2 (pattern_similarity) > THRESH_SIMILARITY (default: 0.80)
 AND F5 (session_overlap_ratio) < THRESH_OVERLAP (default: 0.15):
 → return SINGLE_DEV
 Rationale: All machines exhibit nearly identical work patterns AND they are rarely active at the same time. This is one person who can't physically be on two machines simultaneously, doing the same type of work on each.

RULE S2 – Very Few Machines (regardless of pattern):

IF F3 (machine_count) <= THRESH_SINGLE_DEV_MACHINES (default: 3)
 AND F1 (project_diversity) == 1:
 → return SINGLE_DEV
 Rationale: 1-3 machines on one project is almost always a single developer (home desktop, work laptop, maybe a build machine).

Not worth escalating even if patterns differ slightly due to hardware differences affecting session duration.

RULE T1 – Default to Team:

→ return TEAM

Rationale: If we reach here, there are 4+ machines with dissimilar work patterns or significant simultaneous usage, all on the same project. This is a team that needs more seats.

6.26.8.3 Threshold Configuration

All thresholds are server-side configurable and NOT shipped in the SLT or client binary. This prevents attackers from learning the exact classification boundaries.

CLASSIFICATION THRESHOLDS (server config, defaults):

```
# Stage 1: Piracy thresholds
THRESH_PROJECT_PIRACY      = 5      # distinct project_ids
THRESH_MACHINE_EXPLOSION   = 20     # distinct hw_fps
THRESH_GROWTH_RATE         = 3.0    # multiplier (3x growth)
THRESH_CHURN               = 0.70   # 70% one-off machines
THRESH_CHURN_MIN_MACHINES  = 10     # minimum machines for churn rule
THRESH_GEO_PIRACY          = 10     # distinct countries
THRESH_GEO_MIN_MACHINES    = 15     # minimum machines for geo rule
THRESH_COMPOUND_PIRACY     = 5      # compound score ceiling

# Stage 2: Single Dev vs. Team thresholds
THRESH_SIMILARITY          = 0.80   # cosine similarity floor
THRESH_OVERLAP             = 0.15   # session overlap ceiling
THRESH_SINGLE_DEV_MACHINES = 3      # max machines for auto-SINGLE_DEV
```

All thresholds tunable via server admin panel without client update.

6.26.8.4 Classification → Enforcement Mapping

ENFORCEMENT BY CLASSIFICATION:

SINGLE_DEV:

- No enforcement action
- Log classification for audit trail
- No user-facing message
- Hardware migration (§6.18) handles machine transitions
- The key continues generating intermediate keys normally

TEAM:

- Trigger graduated pressure (§6.26.7):
 - Phase 1: Soft warning (14 days)
 - Phase 2: Hard warning (7 days)
 - Phase 3: Renewal rejection
 - Phase 4: Cascading death
- Automatic team detection (§6.26.5) groups users by project_id
- Online License Manager (§6.26.6) shows deficit and resolution path
- deficit = |T2_users| - |unique_licenses|

PIRACY:

- Immediate key revocation
- Add license_key_id to Revocation Bloom Filter (§6.20)
- All machines using this key receive revocation on next sync
- Cascading death activates on ALL machines within one epoch
- Log incident for legal evidence
- If the legitimate buyer contacts support, issue a NEW key (the old one is permanently burned in the Bloom filter)

6.26.8.5 Worked Examples

EXAMPLE 1: SINGLE_DEV

key: "K-alice-001"

Machines: hw_fp_desktop (home), hw_fp_laptop (coffee shop)

```
F1 = 1 (same project "MyGame")
F2 = 0.94 (both machines create same OV types, similar session lengths)
F3 = 2
F4 = 0.0 (same 2 machines for months)
```


F5 = 0.03 (alice occasionally forgets to close the editor on one machine)
 F6 = 0.0 (both machines are regulars)
 F7 = 1 (same country)

Classification flow:

P1: F1=1 < 5 → skip
 P2: F3=2 < 20 → skip
 P3: F3=2 < 10 → skip
 P4: F7=1 < 10 → skip
 P5: score=0 → skip
 S1: F2=0.94 > 0.80 AND F5=0.03 < 0.15 → SINGLE_DEV ✓

Result: No enforcement. Alice continues working normally.

EXAMPLE 2: TEAM

key: "K-acme-001" (Acme Games bought 1 seat, 3 devs are using it)
 Machines: hw_fp_alice, hw_fp_bob, hw_fp_carol

F1 = 1 (same project "AcmeShooter")
 F2 = 0.35 (alice does OV gameplay, bob does AR networking, carol does BP UI)
 F3 = 3
 F4 = 0.5 (carol's machine is new this month)
 F5 = 0.62 (all three often working at the same time during office hours)
 F6 = 0.0 (all 3 machines are regulars)
 F7 = 1 (same office)

Classification flow:

P1: F1=1 < 5 → skip
 P2: F3=3 < 20 → skip
 P3: F3=3 < 10 → skip
 P4: F7=1 < 10 → skip
 P5: score=0 → skip
 S1: F2=0.35 < 0.80 → skip
 S2: F3=3 ≤ 3 AND F1=1 → SINGLE_DEV? Wait...

PROBLEM: Rule S2 would misclassify this as SINGLE_DEV because
 F3=3 ≤ 3 machines. But this IS a team!

REFINEMENT for Rule S2: Add overlap check.

S2 (refined): IF F3 ≤ 3 AND F1 == 1 AND F5 < 0.30:
 → SINGLE_DEV
 Here F5=0.62 > 0.30, so S2 does NOT fire.

T1: Default → TEAM ✓

Result: Graduated pressure begins. Acme needs 2 more seats.

EXAMPLE 3: PIRACY

key: "K-cracked-001" (leaked on a warez forum)
 Machines: 47 distinct hw_fps over 30 days

F1 = 12 (used by people working on 12 different games)
 F2 = 0.41 (random mix of usage patterns)
 F3 = 47
 F4 = 8.2 (started with 5 last month, now 47 → 42/5 = 8.4x growth)
 F5 = 0.28 (some simultaneous, but many are in different time zones)
 F6 = 0.72 (34 of 47 machines appeared only once or twice)
 F7 = 14 (requests from 14 different countries)

Classification flow:

P1: F1=12 ≥ 5 → PIRACY ✓ (first rule fires immediately)

Even if P1 didn't fire:

P2: F3=47 ≥ 20 AND F4=8.2 > 3.0 → PIRACY ✓
 P3: F6=0.72 > 0.70 AND F3=47 ≥ 10 → PIRACY ✓
 P4: F7=14 ≥ 10 AND F3=47 ≥ 15 → PIRACY ✓
 P5: score = 2+1+2+2+1 = 8 ≥ 5 → PIRACY ✓ (ALL signals fire)

Result: Immediate revocation. Key burned in Bloom filter.
 The original buyer can contact support for a new key.

Based on the worked example above, Rule S2 is refined to include a session overlap guard:

```
RULE S2 (FINAL):
  IF F3 (machine_count) <= THRESH_SINGLE_DEV_MACHINES (default: 3)
    AND F1 (project_diversity) == 1
    AND F5 (session_overlap_ratio) < THRESH_SINGLE_DEV_OVERLAP (default: 0.30):
    → return SINGLE_DEV

  THRESH_SINGLE_DEV_OVERLAP = 0.30  (added to threshold config)
```

This prevents misclassifying a small team (3 devs working simultaneously on the same project) as a single developer.

6.26.8.7 Edge Cases

EDGE CASES AND HANDLING:

1. CI/BUILD MACHINES

A single dev may have 1 personal machine + 1-2 CI build servers that run automated tests. Build servers have:

- Very different usage patterns (no OV creation, only compilation)
- Usage tier T0 or T1 (they don't actively create OVs)

Resolution: The algorithm only counts T2 hw_fps. CI machines at T0/T1 are excluded from the feature vector entirely.

2. VIRTUAL MACHINES / CONTAINERS

A developer using VMs or containers may generate different hw_fps for each instance but exhibit identical patterns. Resolution: F2 (pattern_similarity) catches this – if all VMs have identical patterns, they classify as SINGLE_DEV regardless of machine count (up to the piracy thresholds).

3. LARGE DISTRIBUTED TEAM (e.g., 30+ devs across 8 countries)

Could trigger P4 (geo_spread) or P5 (compound score). Resolution: Teams that properly buy per-seat licenses will have |unique_licenses| ≈ |T2_users|. The classification only runs AFTER §6.26.4 detects a seat deficit. A fully licensed team NEVER enters this algorithm.

4. HARDWARE UPGRADE / MIGRATION

Developer gets a new machine → one new hw_fp, old one disappears. This is handled by hardware migration (§6.18) BEFORE reaching the classification algorithm. The old hw_fp is deregistered, so F3 stays stable and F6 stays low.

5. BORDERLINE CASES (scores near thresholds)

The server carries a confidence_score alongside the classification:
confidence = distance_from_nearest_threshold / threshold_value
Low-confidence PIRACYcandidates (confidence < 0.2) are held in a REVIEW queue for manual inspection before revocation. Low-confidence TEAM candidates are treated as SINGLE_DEV (err on the side of the user).

6. RECLASSIFICATION OVER TIME

Classifications are recomputed every 7 days as the rolling window advances. A SINGLE_DEV key that later exhibits team behavior (new person joins with different patterns) will be reclassified to TEAM. A TEAM key where the deficit clears (seats purchased) exits enforcement entirely.

6.26.8.8 Summary — Classification Signal Map

Signal	SINGLE_DEV	TEAM	PIRACY
F1: Projects	1	1	5+
F2: Similarity	> 0.80	< 0.50	random
F3: Machines	1-3	4-15	20+
F4: Growth Rate	~0.0	< 1.0	> 3.0
F5: Overlap	< 0.15	> 0.30	varies
F6: Churn	~0.0	< 0.20	> 0.70
F7: Geo Spread	1-2	1-5	10+
Enforcement	None	§6.26.7	Revocation
Response Time	–	51+ days	Immediate

User Recourse	—	Buy seats	Contact support
---------------	---	-----------	-----------------

6.26.9 Team Penalty — Invalid License Cascade Within Teams

Beyond sharing detection, there is a **team-wide penalty** for invalid licenses: if one T2 team member has an outright invalid license (expired, revoked, missing), the server rejects renewals for ALL T2 members on that project until the problem is resolved.

TEAM PENALTY SCENARIO:

Team "PROJ_XYZ":
 alice: T2, license VALID ← will be penalized
 bob: T2, license VALID ← will be penalized
 carol: T2, license EXPIRED ← THE PROBLEM
 dave: T1, no license needed ← NOT penalized

Server detects: carol's license is expired.
Server action:

- Reject renewal requests from alice AND bob AND carol for project PROJ_XYZ's plugins
- Response includes: { "penalty_reason": "team_member_invalid", "offending_user": "carol", "team_id": "...", "resolution": "carol must renew their license" }
- alice and bob see: "Team member 'carol' has an invalid license for [PluginName]. Renewals for ALL active developers on this project are suspended until this is resolved."

WHY: This mirrors the cascading death philosophy – a team that tolerates one unlicensed developer should not benefit from the system. It creates social/economic pressure to keep ALL team members properly licensed.

RECOVERY: carol buys/renews their license → server clears the penalty → alice and bob can renew immediately.

Important safeguards:

- The penalty ONLY applies to T2 users on the SAME project
- T1 and T0 users are NEVER penalized (they don't need licenses)
- The penalty does NOT affect other projects — if alice also works on PROJ_ABC solo, that project is unaffected
- Existing intermediate keys continue working until natural expiry (no mid-work kill)
- The offending user is clearly identified in all messages

6.26.10 Offline-First Teams — Deferred Enforcement

Teams using the **offline-first** model (§6.14–6.25) cannot be checked in real-time. Enforcement is deferred:

Scenario	Handling
All team members are offline-first	No real-time detection. When ANY team member comes online (even briefly), their heartbeat includes the project_id. If other user_ids with the same project_id are already in the server's database, team formation occurs. Sharing is detectable on next sync.
Mixed team (some online, some offline)	Online members form the team immediately. Offline members are added when they sync. Enforcement only applies to members who have synced within the last 30 days.
Team member never syncs	If a user operates 100% offline forever and never contacts the server, sharing cannot be detected for them specifically. However, their LICENSE is still locally valid (the Vault doesn't revoke it). The server can still detect the team from other members who DO sync.

The offline-first Epoch-Bound system (§6.15–6.22) still works. Team enforcement is an OVERLAY that operates through the server's renewal/heartbeat mechanism. It does not modify the local epoch chain or GlobalKey.glic. An offline user's license continues to self-renew locally; the team penalty only kicks in when they next contact the server.

6.26.11 Server-Side API Extensions

New server endpoints for team management:

```
// Team auto-detection (happens implicitly during heartbeat/renewal)
POST /api/v1/license/heartbeat
Body: {
  "user_id": "alice",
  "project_id": "E0C3F3454AA9F4309CCBD6829FE5C95E",
  "plugin_name": "GorgeousEntertaining",
```

```
"hardware_fingerprint": "<hash>",
"usage_fingerprint": {
  "tier": "T2",
  "ov_create_count": 42,
  "ar_bind_count": 7,
  "session_duration_minutes": 180,
  "hmac": "<HMAC-SHA256 signed by Epoch_N_Key>"
},
"intermediate_key": "<current JWT>"
}

// Team management portal API
GET /api/v1/teams                → list user's teams
GET /api/v1/teams/{team_id}      → team details + members + tiers
PUT /api/v1/teams/{team_id}/members/{uid} → admin: override tier, remove member
POST /api/v1/teams/{team_id}/buy-seats → purchase additional seats
GET /api/v1/teams/{team_id}/sharing-status → current sharing detection status
```

6.26.12 Consent & Privacy

CRITICAL: The Usage Fingerprint is telemetry. It must be:

- **Explicitly disclosed** in the consent dialog (§6.7): "The following aggregate usage data is included: [API call category counts, session duration, usage tier classification]. No code, variable names, project content, or personal data beyond what is listed in §6.6 is transmitted."
- **Opt-outable in theory** — if a user declines the extended data, the Usage Fingerprint is not sent. However, without it, the server **cannot determine their tier** and defaults to **T2 (Active Developer)** as a conservative assumption. This means declining the telemetry does NOT help avoid sharing detection — it makes it MORE likely to trigger.
- The raw counter values are NEVER stored long-term on the server. Only the tier classification (T0/T1/T2) is retained.

6.26.13 Edge Cases & Fairness

Scenario	Handling
Developer does ONE test with OV, then stops	30-day rolling window. After 30 days of no T2 activity, the user's tier naturally degrades to T1 or T0. Sharing flag is re-evaluated. No permanent stigma.
Contractor works for 2 weeks, then leaves	After 30 days of inactivity, their T2 tier expires. Team admin can also manually remove them. Seat count adjusts downward.
Studio has 10 developers, all T2, buys 10 licenses	Perfect compliance. No sharing detected. No warnings. No penalties. Everyone works uninterrupted.
Solo developer, no team	Team is never formed (only one user_id on the project). All enforcement is skipped.
User has multiple projects with different teams	Teams are per-project. License sharing is evaluated per-project. A valid license covers all projects the user works on (licenses are per-user, not per-project per-user).
CI/CD build server triggers telemetry	Build servers typically don't open the editor (headless cook/package). If no Vault API calls are made, the server is T0. If the CI system does trigger editor via commandlets, the build server runs with a "service account" user_id that can be marked as T0/T1 in the admin panel.
User disputes their tier classification	Team admin can override (§6.26.6). Overrides are time-limited and auto-revoked if contradicted by actual usage. Support escalation available for disputes the admin can't resolve.

6.26.14 Summary — What This System Achieves

TEAM DETECTION & PER-SEAT ENFORCEMENT – SUMMARY

1. CLASSIFY users into T0 / T1 / T2 based on Vault API usage

- T0 = doesn't use it (no license needed)
- T1 = uses it indirectly (no license needed, designers/artists)
- T2 = actively develops with it (license REQUIRED per Fab ToS)

2. DETECT teams automatically when multiple user_ids share a project_id

- Auto-grouped, manageable in Online License Manager

3. DETECT sharing when |T2_users| > |unique_licenses| for a project

- Distinct from piracy (it's the same team, not strangers)

4. ENFORCE via graduated pressure:

- 14d soft warning → 7d hard warning → renewal rejection → expiry
- Total grace period: ~51 days before real impact

- Never kills an active session – only rejects future renewals

5. PENALIZE teams with invalid-license members:

- One invalid T2 = all T2 renewals rejected for that project
- Creates social/economic pressure to maintain compliance

6. PROTECT non-developers:

- T0/T1 users are NEVER penalized or affected
- Designers place OV actors, artists never touch Gorgeous – fine

7. RECOVERABLE: Buy missing seats → instant resolution

6.27 Epoch Key Lifetime Configuration — Shipping vs. Editor

6.27.0 Where the Epoch Key Is Used (Clarification)

Common misconception: The epoch key is NOT an "offline-only" concept. It is the **central cryptographic root** of the entire runtime protection system — used in both online and offline modes, in both Editor and Shipping builds. The offline-first model (§6.14) merely defines how the epoch **chain is advanced** without server contact. The key itself is consumed everywhere.

Consumer	What It Uses EpochKey For	Online/Offline?
LDCC Derivation (§9.4.2)	All 12 License-Derived Computation Constants are <code>HKDF(EpochKey, ...)</code> . Every OV serialization, hash table, replication tick, and AR HMAC depends on them.	Both
Accumulator Chain (§9.4.3)	The accumulator HMAC key includes the VaultTextHash, which gates access to the EpochKey via triple-envelope Layer 2. Indirectly, the entire accumulator is anchored to the epoch.	Both
AR Packet HMAC (§9.4.4)	<code>LDCC_AR_PacketHMAC_Key = HKDF(EpochKey, "ar-hmac")</code> . Every multiplayer AR packet is tagged with a key derived from the epoch.	Both
Packaging Nonce (§6.10)	<code>Nonce_Package = HMAC-SHA256(EpochKey, project config hw counter)</code> . Packaging authorization IS epoch proof.	Both
SLT Generation (§7.4)	<code>SLT_Key = HKDF(EpochKey, VaultTextHash project)</code> . The Shipping License Token is encrypted with an epoch-derived key.	At packaging time
License Renewal (§6.14/§6.15)	The epoch chain advances by deriving <code>Epoch_N+1_Key = HKDF(Epoch_N_Key, epoch_counter master_seed_hash)</code> . The chain IS the renewal.	Offline: local chain advance. Online: server provides next epoch.
Triple-Envelope Layer 3 (§6.15)	The temporal binding key includes epoch state. Different epoch → different Layer 3 key → different master_seed decryption.	Both

Summary: The epoch key is the heartbeat of the Vault. In online mode, the server pushes new epochs. In offline mode, the user's GlobalKey.glic advances the chain locally. In Shipping mode, the epoch is frozen into the SLT. But in ALL modes, the EpochKey is what makes everything work — LDCCs, accumulators, AR HMACs, packaging, and encryption all flow from it.

PROBLEM: The epoch key rotates every ~90 days (default). In the Editor, this is desirable — it limits the window of a cracked license. But in a **packaged Shipping game**, the epoch key is baked into the Shipping License Token (SLT, §7.4). If the epoch expires and the developer doesn't ship a game update with a refreshed SLT within 90 days, one of two things happens:

1. **The game breaks for everyone** — OV/AR stops working because the LDCCs are derived from an expired epoch. Unacceptable.
2. **The Vault ignores expiration** — cheaters get free play because expired constants mean no HMAC verification. Also unacceptable.

SOLUTION: The epoch lifetime is **configurable per build configuration**, with different defaults for Editor and Shipping. In Shipping builds, the developer can opt into **no-expiration mode**, making the baked-in LDCCs valid forever. This is safe because the Shipping SLT protects against piracy through binary binding (§7.4.4), not through epoch rotation.

6.27.1 Why Epoch Rotation Matters in Editor (But Not in Shipping)

Concern	Editor / Development	Shipping
Primary threat	Plugin piracy (cracked Vault DLL)	Multiplayer cheating (forged AR packets)
Who has a license?	The developer (mandatory)	Nobody — the SLT replaces the license
What does epoch rotation prevent?	A cracked license from working indefinitely	Nothing — there IS no license to crack in a Shipping build
What does expiration break?	Nothing — developer renews naturally	EVERYTHING — the game stops working for all players
Conclusion	Short epoch (90 days) is correct	No expiration (or very long epoch) is correct

6.27.2 Configuration Model

The epoch lifetime is controlled by **Developer Settings** (§6.28) — a `UDeveloperSettings` subclass in the open-source GorgeousCore module. These settings are read by the Vault at two points:

1. **At packaging time** — to determine the SLT's `slt_flags.no_expiration` and `slt_flags.epoch_validity_days` values.
2. **At Editor startup** — to determine the epoch validity window for license validation.

EPOCH LIFETIME CONFIGURATION

Developer Settings (\$6.28):

Gorgeous Core → Licensing → Epoch Configuration

Epoch Validity (Editor/Development): [90] days (default)
Range: 30-365 days
Affects: How often the developer's .glic epoch must renew

Epoch Validity (Shipping): [∞ (no expiration)]
Options: o No Expiration (recommended for most games)
o Custom: [____] days (range: 5-36500 / ~100yr)
Affects: SLT baked into packaged games

⚠ Warning: Setting a Shipping expiration means you MUST ship game updates before the epoch expires, or OV/AR will stop working for ALL players.

How the values flow:

1. Developer sets values in Project Settings → Gorgeous Core → Licensing

2. Values are stored in DefaultGorgeousCore.ini (open-source, version-controlled)

3. At packaging time, the Vault reads these via the UDeveloperSettings API

4. The SLT is generated with the configured epoch behavior

5. In Shipping, the Vault reads `slt_flags` to determine expiration behavior

6.27.3 Epoch Lifetime Behavior Matrix

Build Configuration	Epoch Source	Default Lifetime	Expiration Behavior
Editor	.glic file on disk	90 days	Epoch expires → developer must renew (online refresh or offline chain advance). Normal DRM behavior.
Development	.glic file on disk	90 days (same as Editor)	Same as Editor. Development builds require a valid developer license.
DebugGame	.glic file on disk	90 days (same as Editor)	Same as Editor.
Debug	.glic file on disk	90 days (same as Editor)	Same as Editor.
Test	SLT in binary	Matches Shipping config	Same as Shipping. Test builds behave like Shipping for licensing.
Shipping	SLT in binary	No expiration	If <code>no_expiration = true</code> : LDCCs are valid forever. No epoch check. If <code>no_expiration = false</code> : LDCCs expire after <code>epoch_validity_days</code> . After expiry, OV/AR is disabled.

6.27.4 Security Implications of No-Expiration in Shipping

Q: Doesn't removing expiration weaken security?

A: **No.** The epoch rotation in Editor mode serves ONE purpose: limiting the window during which a cracked license works. In a Shipping build, there IS no license to crack — the SLT is binary-bound. The security properties of the SLT come from:

- **AES-256-GCM encryption** with a key derived from VaultTextHash (DLL-bound)
- **Project-ID binding** in the AAD
- **One-way derivation** — LDCCs in the SLT cannot recover the master_seed

Epoch rotation adds nothing to Shipping security. Removing it prevents the game from self-destructing on a timer.

Q: What if a cheater extracts the LDCCs from the SLT and they never expire?

A: The LDCCs are encrypted inside the SLT. Extracting them requires breaking AES-256-GCM or de-virtualizing the SLT_Key derivation (VM-protected, §5.5.1). If an attacker CAN extract them, they could do so regardless of expiration — they'd simply extract them again after each rotation. Expiration does not add meaningful protection against LDCC extraction. It only adds a ticking clock that punishes legitimate players.

6.27.5 Recommendations for Developers

Game Type	Recommended Setting	Rationale
Live-service game (regular updates)	Custom: 180–365 days	You're shipping updates frequently anyway. The epoch refresh happens naturally with each update's re-packaging.
Single-player / finished game	No expiration	The game may not receive updates for years. The SLT must remain valid forever.
Competitive multiplayer (live-service)	Custom: 30–90 days	Shorter epoch = more frequent LDCC rotation in updates = harder for cheaters to maintain extracted values. But you MUST ship updates on schedule.
Esports / ultra-high-stakes multiplayer	Custom: 5–14 days	Maximum LDCC rotation pressure. A cheater who extracts the AR HMAC key has at most 5–14 days before it's useless. Requires very frequent patching — only viable for games with automated CI/CD pipelines and live-service infrastructure. See \$9.6 for the full anti-cheat analysis of this approach.
Early access / beta	No expiration	Focus on development, not epoch management. Switch to custom when the game stabilizes.

6.28 Developer Settings — `UGorgeousLicensingSettings`

DESIGN GOAL: Expose epoch lifetime configuration (§6.27) and other licensing behavior through Unreal Engine's standard Developer Settings system. The settings class lives in the **open-source** `GorgeousCore` module — developers can inspect, modify, and version-control these values. The Vault reads them at packaging time via the standard `GetDefault<UGorgeousLicensingSettings>()` API.

6.28.1 Class Definition (Open-Source — in `GorgeousCoreRuntime`)

```
// GorgeousLicensingSettings.h
// Located in: GorgeousCore/Source/GorgeousCoreRuntime/Public/Settings/

#pragma once

#include "Engine/DeveloperSettings.h"
#include "GorgeousLicensingSettings.generated.h"

/**
 * Developer settings for the Gorgeous Vault licensing system.
 *
 * These settings control epoch key lifetime, Shipping build behavior,
 * and other configurable licensing parameters.
 *
 * Access: Project Settings → Gorgeous Core → Licensing
 *
 * IMPORTANT: The Vault DLL reads these settings at packaging time to
 * determine how the Shipping License Token (SLT) is generated.
 * Changes take effect on the NEXT packaging operation.
 */
UCLASS(Config = GorgeousCore, DefaultConfig, meta = (DisplayName = "Gorgeous Licensing"))
class GORGEOUSCORDERUNTIME_API UGorgeousLicensingSettings : public UDeveloperSettings
{
    GENERATED_BODY()

public:
    UGorgeousLicensingSettings();

    // — Epoch Configuration —————

    /**
     * How long an epoch key remains valid in Editor/Development builds (in days).
     * After this period, the developer's .glic file must be renewed
     * (automatically via online refresh or manually via offline chain advance).
     *
     * Lower values = tighter anti-piracy window but more frequent renewals.
     * Higher values = less renewal friction but longer crack window.
     *
     * Range: 30–365. Default: 90.
     */
    UPROPERTY(Config, EditAnywhere, Category = "Epoch Configuration",
        meta = (DisplayName = "Editor Epoch Validity (Days)",
            ClampMin = "30", ClampMax = "365",
            ToolTip = "How long the epoch key is valid in Editor/Development builds.))
    int32 EditorEpochValidityDays = 90;

    /**
     * If true, the Shipping License Token (SLT) baked into packaged games
     * will NEVER expire. The LDCCs remain valid for the lifetime of the binary.
     */
    UPROPERTY(Config, EditAnywhere, Category = "Epoch Configuration",
        meta = (DisplayName = "SLT Never Expires",
            ToolTip = "If checked, the Shipping License Token (SLT) baked into packaged games will never expire. The LDCCs remain valid for the lifetime of the binary."))
    bool SLTNeverExpires = false;
};
```

```

*
* Recommended for most games, especially single-player and games
* that may not receive frequent updates.
*
* If false, the SLT will expire after ShippingEpochValidityDays.
*/
UPROPERTY(Config, EditAnywhere, Category = "Epoch Configuration",
    meta = (DisplayName = "No Epoch Expiration in Shipping",
        Tooltip = "If true, packaged games never need epoch renewal.))
bool bShippingNoExpiration = true;

/**
* How long the SLT remains valid in Shipping builds (in days).
* Only applies if bShippingNoExpiration is false.
*
* WARNING: If this expires and you haven't shipped a game update
* with a refreshed SLT, OV/AR will stop working for ALL players.
*
* Range: 5-36500 (~100 years). Default: 365.
*
* Values below 30 days are intended for competitive multiplayer games
* that ship frequent updates (e.g., live-service, esports titles).
* At minimum (5 days), you MUST ship a game update every 5 days
* or OV/AR will stop working for all players.
*/
UPROPERTY(Config, EditAnywhere, Category = "Epoch Configuration",
    meta = (DisplayName = "Shipping Epoch Validity (Days)",
        ClampMin = "5", ClampMax = "36500",
        EditCondition = "!bShippingNoExpiration",
        Tooltip = "How long the SLT is valid in Shipping builds. Only applies if 'No Epoch Expiration' is unchecked.))
int32 ShippingEpochValidityDays = 365;

// — Anti-Cheat —————
//
// These settings control the Vault's anti-cheat subsystem.
// All AC settings are BAKED INTO THE SLT at packaging time.
// A cheater cannot modify them at runtime without breaking the SLT's
// AES-GCM authentication tag – which destroys ALL protection keys.
//
// NOTE: Anti-cheat requires a valid Shipping License Token.
// GorgeousCore is free and by default has NO license → NO SLT → NO AC.
// To enable AC for GorgeousCore-only projects, set bEnableAntiCheat = true.
// This fetches a FREE Anti-Cheat Core License from the license server,
// which provides the cryptographic material needed to generate a Core SLT.
// See §6.29 in the Protection Plan.
//
// If your project uses ANY paid Gorgeous plugin (Events, Entertaining, etc.),
// you already have a license → AC is available automatically.
// Each additional Gorgeous plugin adds an independent SLT compartment,
// requiring the cheater to crack each one separately ($7.5 Key Spread).
//
// VISIBILITY RULES (enforced via EditCondition + EditConditionHides):
//
// bExplicitlyDisableAntiCheat:
// - HIDDEN when only GorgeousCore is present (nothing to disable)
// - VISIBLE when paid plugins are present (lets dev opt out of auto-AC)
//
// bEnableAntiCheat:
// - HIDDEN when bAnyOtherGorgeousPluginPresent_Internal is true
//   (AC is auto-implied by paid plugins – flag is irrelevant)
// - HIDDEN when bExplicitlyDisableAntiCheat is true
//   (AC is force-disabled – enable toggle is meaningless)
// - VISIBLE only when Core-only AND not explicitly disabled
//
// AntiCheatResponseMode, bEnableOVIntegrityCanary:
// - HIDDEN when bExplicitlyDisableAntiCheat is true
//   (AC is disabled – sub-settings are meaningless)

/**
* Explicitly disable Anti-Cheat for ALL Gorgeous plugins.
*
* When enabled, this flag:
* - Completely disables the Vault's anti-cheat subsystem
* - No SLT anti-cheat fields are baked at packaging time
* - No AC Core License is fetched
* - No LDCCs, Canaries, Sentinels, or Rot logic is active
* - Hides all other AC settings (bEnableAntiCheat, ResponseMode, Canary)
*
* This does NOT affect any other Gorgeous functionality:
* - OV (Object Variables) continue working normally
* - AR (Auto Replication) continues working normally

```



```

* - License validation, epoch chains, team detection – all unaffected
* - The only thing removed is the anti-cheat armor layer
*
* Use case: Projects that don't need anti-cheat (single-player,
* prototypes, tools, non-game applications) but use paid Gorgeous
* plugins. Without this flag, paid plugins auto-enable AC.
*
* WARNING: Enabling this in a multiplayer game leaves the OV/AR
* replication layer unprotected against memory editors and
* packet forgery. Only disable AC if you are certain you don't need it.
*
* NOTE: This setting is only visible when paid Gorgeous plugins are
* present. If you only have GorgeousCore, AC is already off by default
* – there is nothing to explicitly disable.
*/
UPROPERTY(Config, EditAnywhere, Category = "Anti-Cheat",
    meta = (DisplayName = "Explicitly Disable Anti-Cheat",
        EditCondition = "bAnyOtherGorgeousPluginPresent_Internal",
        EditConditionHides,
        ToolTip = "Force-disables all anti-cheat for every Gorgeous plugin. OV/AR continue working – only the AC armor
is removed."))
bool bExplicitlyDisableAntiCheat = false;

/**
* Enable the Vault's Anti-Cheat system for the Shipping build.
*
* When enabled, the Vault fetches a FREE Anti-Cheat Core License from the
* license server (if no paid plugin licenses are present). This license
* provides cryptographic material for generating a Core SLT, enabling:
* - HMAC-tagged OV replication (prevents packet forgery)
* - Accumulator chain integrity (detects tampered Vault DLLs)
* - SLT Presence Sentinel (detects SLT stripping – $7.4.6)
* - OV Integrity Canary (if enabled below – $9.6.8)
*
* If you already have paid Gorgeous plugin licenses, this flag is
* automatically implied – you always get AC from paid plugin SLTs.
* This setting exists specifically for GorgeousCore-only projects.
*
* The free AC license is:
* - Auto-fetched from the license server (same infra as paid licenses)
* - Project-bound (tied to hardware fingerprint + project ID)
* - Epoch-rotating (same security as paid licenses)
* - Free forever – no payment, no trial, no expiration
*
* See §6.29 in the Protection Plan.
*
* VISIBILITY: Hidden when paid plugins are present (AC is auto-implied).
*             Hidden when bExplicitlyDisableAntiCheat is true.
*             Only visible for GorgeousCore-only projects.
*/
UPROPERTY(Config, EditAnywhere, Category = "Anti-Cheat",
    meta = (DisplayName = "Enable Anti-Cheat (Shipping)",
        EditCondition = "!bAnyOtherGorgeousPluginPresent_Internal && !bExplicitlyDisableAntiCheat",
        EditConditionHides,
        ToolTip = "Fetches a free AC license for GorgeousCore to enable anti-cheat in Shipping builds. Required for
Core-only projects."))
bool bEnableAntiCheat = false;

/**
* What should happen when the Vault detects cheating.
*
* This controls the RESPONSE to all anti-cheat detections:
* - Canary mismatch (memory-edited OV values)
* - SLT Sentinel failure (stripped protection keys)
* - Accumulator divergence (tampered Vault DLL)
*
* Options:
* 0 = "The Rot" (Silent Poison)
*     Silently corrupt the cheater's OV/AR state over time.
*     No visible error, no ban screen – the game just slowly breaks.
*     The cheater cannot tell when they were detected or why things
*     stopped working. Completely client-side – server learns nothing.
*     Best for: psychological deterrence, wasting cheater time.
*
* 1 = "Kick + Server Callback"
*     Immediately disconnect the cheating client and fire a callback
*     to the GorgeousCoreRuntime on the SERVER side:
*     OnAntiCheatViolation(PlayerController, ViolationType, Details)
*     The server can then: log the event, ban the player, flag the
*     account, notify admins, update a leaderboard, etc.
*     Best for: competitive games with server-authoritative enforcement.

```

```

*
* 2 = "Rot + Delayed Server Callback" (Recommended)
* First applies The Rot (silent corruption). After a random delay
* (30-120 seconds), ALSO fires the server callback. This gives
* maximum information: the cheater experiences silent degradation
* (making it hard to pinpoint the detection moment), AND the server
* gets informed for persistent enforcement.
* Best for: most multiplayer games.
*
* CRITICAL: This setting is BAKED INTO THE SLT at packaging time.
* A cheater cannot change the response mode – they would have to break
* the SLT's AES-GCM encryption, which destroys all protection keys.
*
* Without this protection, a cheater could set their client to
* mode 0 (Rot only) to prevent the server from being notified.
* By baking it into the SLT, the developer's choice is final.
*/
UPROPERTY(Config, EditAnywhere, Category = "Anti-Cheat",
    meta = (DisplayName = "Anti-Cheat Response Mode",
        ClampMin = "0", ClampMax = "2",
        EditCondition = "!bExplicitlyDisableAntiCheat",
        EditConditionHides,
        ToolTip = "0 = Silent Rot, 1 = Kick + Server Callback, 2 = Rot + Delayed Callback (Recommended). Baked into
SLT.))
int32 AntiCheatResponseMode = 2;

/**
* Enable the OV Integrity Canary system for Shipping builds.
*
* When enabled, the Vault computes a cryptographic canary (HMAC) for every
* OV variable on each SetValue() call. Before serialization for replication,
* the canary is re-verified. If the value was modified outside the OV API
* (e.g., by a memory editor), the canary mismatch is detected and the
* tampered value is REJECTED – it will not be replicated.
*
* IMPORTANT: This flag controls what gets BAKED INTO THE SLT at packaging
* time. When enabled, the Vault includes the canary code path and the
* LDCC_OV_CanaryKey in the SLT. When disabled, the canary code path is
* completely absent from the Shipping binary – there is no flag to toggle
* at runtime, no code to NOP, and no branch to patch.
*
* Requires: bEnableAntiCheat = true (or any paid Gorgeous plugin license).
*
* Overhead: ~1-5µs per SetValue() call + ~1-5µs per serialization event.
* For 50 OV variables changing at ~10 Hz each = ~0.5-5ms per frame.
* Negligible for event-driven variables (health, currency, score).
* May be noticeable for high-frequency variables (position, rotation).
*
* Recommended: Enable for competitive multiplayer games with
* sensitive replicated OV state. Disable for single-player or
* games where OV replication latency is not critical.
*
* See §9.6.8 in the Protection Plan for the full technical design.
*/
UPROPERTY(Config, EditAnywhere, Category = "Anti-Cheat",
    meta = (DisplayName = "Enable OV Integrity Canary (Shipping)",
        EditCondition = "!bExplicitlyDisableAntiCheat && (bEnableAntiCheat || bAnyOtherGorgeousPluginPresent_Internal)",
        EditConditionHides,
        ToolTip = "Detects memory edits to OV values before replication. Baked into the Shipping build – cannot be
toggled by cheaters.))
bool bEnableOVIntegrityCanary = false;

// — Internal (not user-editable) —————

/**
* [INTERNAL] True if any Gorgeous plugin other than GorgeousCore is present.
*
* This value is computed automatically in PostInitProperties() by scanning
* the plugin registry for any enabled Gorgeous sibling plugin
* (GorgeousEntertaining, GorgeousEvents, etc.).
*
* When true:
* - Anti-cheat is auto-implied (paid plugins provide SLT material)
* - bEnableAntiCheat is hidden (irrelevant – AC is automatic)
* - bExplicitlyDisableAntiCheat becomes visible (lets dev opt out)
*
* This property is NOT saved to config. It is recomputed every time
* the settings object is initialized.
*/
UPROPERTY(VisibleAnywhere, Category = "Anti-Cheat",
    meta = (DisplayName = "[Internal] Other Gorgeous Plugins Detected",

```

```

        ToolTip = "Auto-detected: true if any paid Gorgeous plugin is present. Not user-editable.))
bool bAnyOtherGorgeousPluginPresent_Internal = false;

// — Diagnostics —————

/**
 * If true, the Vault logs additional diagnostic information during
 * license validation and packaging. Useful for debugging license issues.
 *
 * WARNING: May expose sensitive information in log files.
 * Should be disabled in production/Shipping builds (auto-disabled).
 */
UPROPERTY(Config, EditAnywhere, Category = "Diagnostics",
    meta = (DisplayName = "Verbose License Logging",
        ToolTip = "Enable detailed logging of Vault license operations.))
bool bVerboseLicenseLogging = false;

public:
    // UDeveloperSettings interface
    virtual FName GetContainerName() const override { return TEXT("Project"); }
    virtual FName GetCategoryName() const override { return TEXT("Gorgeous Core"); }
    virtual FName GetSectionName() const override { return TEXT("Licensing"); }

    /**
     * PostInitProperties – computes bAnyOtherGorgeousPluginPresent_Internal.
     *
     * Scans the Unreal plugin registry for any enabled Gorgeous sibling plugin.
     * This runs every time the settings CDO is created (Editor startup,
     * packaging, etc.) – the value is NEVER persisted to config.
     */
    virtual void PostInitProperties() override;

#ifdef WITH_EDITOR
    virtual FText GetSectionText() const override
    {
        return NSLOCTEXT("GorgeousLicensing", "SectionText", "Licensing");
    }
    virtual FText GetSectionDescription() const override
    {
        return NSLOCTEXT("GorgeousLicensing", "SectionDesc",
            "Configure epoch key lifetimes and Shipping build licensing behavior. "
            "Changes take effect on the next packaging operation.");
    }
#endif
};

```

```

// GorgeousLicensingSettings.cpp – PostInitProperties implementation

#include "Settings/GorgeousLicensingSettings.h"
#include "Interfaces/IPluginManager.h"

UGorgeousLicensingSettings::UGorgeousLicensingSettings()
{
}

void UGorgeousLicensingSettings::PostInitProperties()
{
    Super::PostInitProperties();

    // Scan for any enabled Gorgeous sibling plugin (anything except GorgeousCore itself).
    // This list is maintained as new Gorgeous plugins are released.
    static const TArray<FString> GorgeousSiblingPlugins = {
        TEXT("GorgeousEntertaining"),
        TEXT("GorgeousEvents"),
        // ... future plugins added here
    };

    bAnyOtherGorgeousPluginPresent_Internal = false;

    IPluginManager& PluginManager = IPluginManager::Get();
    for (const FString& SiblingName : GorgeousSiblingPlugins)
    {
        TSharedPtr<IPlugin> Plugin = PluginManager.FindPlugin(SiblingName);
        if (Plugin.IsValid() && Plugin->IsEnabled())
        {
            bAnyOtherGorgeousPluginPresent_Internal = true;
            break;
        }
    }
}

```

```
}
}
```

6.28.2 How the Vault Reads These Settings

The Vault cannot directly call `GetDefault<UGorgeousLicensingSettings>()` because it is a standalone CMake-built DLL that does not link against `GorgeousCoreRuntime`. Instead, the open-source Proxy passes the settings to the Vault via C-linkage exports:

```
// In the Proxy (open-source GorgeousCoreRuntime – called during packaging)
void PassLicensingSettingsToVault()
{
    const UGorgeousLicensingSettings* Settings = GetDefault<UGorgeousLicensingSettings>();

    // Resolve effective AC state:
    // If explicitly disabled → AC is off regardless of paid plugins.
    // If paid plugins present and NOT explicitly disabled → AC is auto-implied.
    // If Core-only → AC depends on bEnableAntiCheat.
    const bool bEffectiveACEnabled =
        !Settings->bExplicitlyDisableAntiCheat &&
        (Settings->bAnyOtherGorgeousPluginPresent_Internal || Settings->bEnableAntiCheat);

    // C-linkage export – passes ALL settings as plain C types
    GVault_SetEpochConfig(
        Settings->EditorEpochValidityDays,
        Settings->bShippingNoExpiration ? 1 : 0,
        Settings->ShippingEpochValidityDays,
        Settings->bVerboseLicenseLogging ? 1 : 0,
        bEffectiveACEnabled ? 1 : 0,
        Settings->AntiCheatResponseMode,
        (bEffectiveACEnabled && Settings->bEnableOVIIntegrityCanary) ? 1 : 0
    );
}
```

C-LINKAGE EXPORT (added to §3.4 – full signature):

```
void GVault_SetEpochConfig(
    int32_t editor_epoch_days,           // EditorEpochValidityDays
    int32_t shipping_no_expiration,      // 1 = true, 0 = false
    int32_t shipping_epoch_days,         // ShippingEpochValidityDays
    int32_t verbose_logging,             // 1 = true, 0 = false
    int32_t enable_anti_cheat,           // Effective AC state (resolved from ExplicitDisable + plugins + bEnableAntiCheat)
    int32_t anti_cheat_response_mode,    // 0 = Rot, 1 = Kick+Callback, 2 = Rot+DelayedCallback
    int32_t enable_ov_integrity_canary   // bEnableOVIIntegrityCanary – 1 = bake canary into SLT
);
```

Called by: The Proxy, at two points:

1. Module startup – so the Vault knows the current epoch config for validation
2. Just before packaging (§6.10 Step 1) – so the SLT is generated with correct flags

6.28.3 Config File Location

The settings are stored in the standard UE config system:

```
; DefaultGorgeousCore.ini (project-level, version-controlled)
[/Script/GorgeousCoreRuntime.GorgeousLicensingSettings]
EditorEpochValidityDays=90
bShippingNoExpiration=True
ShippingEpochValidityDays=365
bEnableAntiCheat=True
AntiCheatResponseMode=2
bEnableOVIIntegrityCanary=False
bVerboseLicenseLogging=False
```

The developer can also override per-platform in platform-specific config files (e.g., `WindowsGorgeousCore.ini`).

6.29 Free Anti-Cheat Core License — AC for GorgeousCore-Only Projects

PROBLEM: GorgeousCore is free and unlicensed. No license → no EpochKey → no LDCCs → no SLT → **no anti-cheat protection in Shipping builds**. A developer using only GorgeousCore gets OV/AR functionality but zero cryptographic armor on the replication layer.

SOLUTION: A free, auto-provisioned "Anti-Cheat Core License" that provides cryptographic material to GorgeousCore without charging the developer anything.

6.29.1 What Is the AC Core License?

The AC Core License is a special `.glic` file of type `"core-ac"` that:

Property	Value
Cost	Free forever — auto-issued to any registered user
Purpose	Provides a <code>core_master_seed</code> for SLT generation — nothing else
Binding	Project-bound (hardware fingerprint + project ID, same as paid licenses)
Epoch chain	Full epoch rotation mechanics (same security as paid licenses)
File name	<code><ProjectDir>/Saved/GorgeousLicense/GorgeousCore_AC.glic</code>
Type field	<code>"core-ac"</code> in the common <code>.glic</code> header
Entitlements	None — this license does NOT grant access to any paid plugin

Key principle: This is NOT a license for using GorgeousCore (it's always free). It's a license for **cryptographic material** that enables protection. Without it, Core still works perfectly — there's just no cryptographic armor on the replication layer.

6.29.2 Acquisition Flow

AC CORE LICENSE — AUTOMATIC PROVISIONING

- Developer enables `bEnableAntiCheat = true` in Project Settings → Anti-Cheat
- On next Editor startup (or next packaging attempt), the Vault detects:
 - `bEnableAntiCheat = true`
 - No paid Gorgeous plugin licenses present
 - No existing `GorgeousCore_AC.glic` file
- Vault contacts the license server:
`POST /api/v1/license/core-ac`

```
{
  "user_id":      <from registration – or auto-registered>,
  "project_id":   <project GUID>,
  "hw_fingerprint": <hardware hash>,
  "vault_version": <Vault DLL version>
}
```
- Server responds with a signed `GorgeousCore_AC.glic`:
 - Contains `core_master_seed` (cryptographic seed)
 - Signed with the server's private key
 - Project-bound and hardware-bound
 - Initial epoch: `epoch_0`
- Vault stores it at: `<ProjectDir>/Saved/GorgeousLicense/GorgeousCore_AC.glic`
- From this license, the Vault derives:
`Core_EpochKey` → `Core_LDCCs` → `Core_SLT` (at packaging time)
- The developer now has full anti-cheat protection in Shipping builds.

6.29.3 What if the Developer Also Has Paid Plugin Licenses?

If the project already has paid Gorgeous plugin licenses (`GorgeousEvents`, `GorgeousEntertaining`, etc.), those licenses ALREADY provide cryptographic material for SLT generation. In this case:

- `bEnableAntiCheat` is **automatically implied** and **hidden** in the UI (via `EditConditionHides` gated on `bAnyOtherGorgeousPluginPresent_Internal`)
- The AC Core License is NOT fetched (not needed)
- Each paid plugin generates its own independent SLT compartment (\$7.5)
- GorgeousCore gets a **derived** SLT compartment from the first available paid plugin's key material (or from the AC Core License if present)
- The `bExplicitlyDisableAntiCheat` flag becomes **visible** — this is the only way to opt out of AC when paid plugins are present

The result: Developers with paid plugins get anti-cheat automatically. The `bEnableAntiCheat` flag exists specifically and only for Core-only projects. The `bExplicitlyDisableAntiCheat` flag exists specifically and only for projects where AC is auto-enabled but unwanted.

6.29.4 Why Not Just Give Everyone an AC License Automatically?

We require the developer to explicitly opt in (`bEnableAntiCheat = true`) because:

- Network call required.** Fetching the license contacts the license server. GorgeousCore's core principle is "zero network calls" — we only break this with explicit developer consent.
- Not everyone needs AC.** Single-player games, prototypes, jam entries — many projects don't need anti-cheat. No reason to fetch a license they'll never use.
- Transparency.** The developer should know when their project is making network calls and why.
- Fab compliance.** Fab requires that network calls are disclosed and consensual.

6.29.6 Explicitly Disabling Anti-Cheat — The `bExplicitlyDisableAntiCheat` Flag

When paid Gorgeous plugins are present, anti-cheat is **auto-enabled**. Some projects may not want this (single-player games, tools, non-game applications). The `bExplicitlyDisableAntiCheat` flag provides a clean opt-out:

VISIBILITY MATRIX – Anti-Cheat Settings

Project Configuration	What the Developer Sees	Effective AC State
GorgeousCore ONLY	<code>bEnableAntiCheat</code> (opt-in) <code>ResponseMode</code> (if AC on) <code>OV Canary</code> (if AC on) <code>bExplicitlyDisable</code> : HIDDEN	Off by default. Developer turns on manually. Nothing to disable – AC is already off by default.
Core + Paid Plugins (AC not explicitly disabled)	<code>bExplicitlyDisable</code> (opt-out) <code>ResponseMode</code> <code>OV Canary</code> <code>bEnableAntiCheat</code> : HIDDEN	On by default (auto-implied). Paid plugin SLTs provide AC. Irrelevant – AC is auto-implied by paid plugin licenses.
Core + Paid Plugins (AC explicitly disabled)	<code>bExplicitlyDisable</code> ✓ (true) <code>ResponseMode</code> : HIDDEN <code>OV Canary</code> : HIDDEN <code>bEnableAntiCheat</code> : HIDDEN	Off. No SLT AC fields baked. OV/AR works, no AC armor.

Key behaviors when `bExplicitlyDisableAntiCheat = true`:

- 1. **No SLT anti-cheat fields** — the SLT is still generated (for licensing integrity), but the AC-specific fields (`ac_response_mode`, `canary_key`, etc.) are zeroed out
- 2. **No Gordian Parasite runtime activity** — the Rot, Canary, and Sentinel subsystems are disabled at the packaging level (code paths excluded from SLT)
- 3. **No AC Core License fetch** — even if `bEnableAntiCheat` was previously true, the explicit disable takes priority
- 4. **All OV/AR functionality preserved** — Object Variables, Auto Replication, Blueprint nodes, C++ API — everything works exactly the same. Only the anti-cheat armor layer is removed
- 5. **Reversible** — uncheck the flag and repackage to re-enable AC. No data loss, no license issues

Important: The flag is per-project (stored in `DefaultGorgeousCore.ini`). A studio with multiple projects can have AC enabled on their competitive multiplayer title and disabled on their single-player narrative game.

6.29.5 SLT Tamper Detection — What If a Cheater Strips the SLT?

The threat: A cheater could try to zero out or remove the SLT from the Vault DLL's `.data` section. Without an SLT, the Vault has no LDCCs → no HMAC keys → anti-cheat is disabled. The existing `.text` integrity hash (§5.1/5.2) catches `.text` modifications but the SLT lives in `.data`.

This is addressed by the **SLT Presence Sentinel** (§7.4.6) — a cryptographic tripwire that makes SLT stripping detectable and feeds into the accumulator chain.

6.30 License Transparency Dashboard — Data Visibility & Per-License Stats

Every user who owns a Gorgeous license (or administers a team) has access to the **License Transparency Dashboard** on the web portal at <https://gorgeous.simsalabim.studio/dashboard>. This section formalizes the principle: **the user should always be able to see exactly what data we hold about their license and how it is being used**.

Core philosophy: If we collect it, the user can see it. No hidden telemetry. No shadow profiles. Every data point visible on the dashboard is also the complete set of data stored server-side — there is nothing behind the curtain.

6.30.1 Dashboard Views

The dashboard is organized into four tabs:

GORGEOUS LICENSE DASHBOARD

⚡ Independent – Not Subject to Epic Nova Terms

[My Licenses]

[Hardware]

[Usage Stats]

[Data & Privacy]

6.30.2 Tab 1 — My Licenses

Shows every license the user owns, their status, and associated metadata.

MY LICENSES

License: GorgeousEntertaining

Key ID:

K-alice-001

Plugin:

GorgeousEntertaining v2.3.1

Status:

☒ Active

Purchased:

2025-03-15

Current Epoch:

E3 (expires 2026-03-15)

Next Renewal:

Automatic (offline-first, \$6.16)

License Type:

Per-seat (1 seat)

Tied to Project:

MyGame (E0C3F3454AA9F4309CCBD6829FE5C95E)

Team:

None (solo developer)

Classification:

SINGLE_DEV (\$6.26.8)

License: GorgeousEvents

Key ID:

K-alice-002

Plugin:

GorgeousEvents v1.0.0

Status:

☐ Renewal Pending

Purchased:

2025-09-01

Current Epoch:

E1 (expires 2026-09-01)

...

[+ Purchase New License]

6.30.3 Tab 2 — Hardware Fingerprints

Shows every hardware fingerprint (HWID) currently registered against each license. This is the key transparency feature — users can see exactly which machines are using their license and deregister stale ones.

REGISTERED HARDWARE FINGERPRINTS

License: GorgeousEntertaining (K-alice-001)

Classification: SINGLE_DEV | Machines: 2 of 3 max (\$6.18)

Fingerprint	Friendly Name	First Seen	Last Seen	Action
a3f8...c291	"Home Desktop"	2025-03-15	2026-02-27	
7b12...e4d0	"Work Laptop"	2025-06-10	2026-02-28	

① Fingerprints are truncated SHA-256 hashes of (MAC + CPU ID + hostname).
The raw hardware identifiers are NEVER stored – only the hash.

[Deregister Selected]

[Rename Machine]

[View Migration History]

Migration History

2025-06-10

Added hw_fp 7b12...e4d0 ("Work Laptop")

2025-03-15

Added hw_fp a3f8...c291 ("Home Desktop") – initial reg

Key details:

- **Friendly names:** Users can assign readable labels to each fingerprint ("Home Desktop", "Office Workstation") so they can identify machines without memorizing hashes.
- **Deregistration:** Users can manually deregister a machine they no longer use. This frees up a migration slot (\$6.18) and removes the hw_fp from all future feature vector calculations (\$6.26.8).
- **Migration history:** Full audit trail of when machines were added, removed, or migrated. Visible to the user at all times.
- **Hash-only storage:** The raw hardware identifiers (MAC address, CPU ID, hostname) are hashed client-side before transmission. The server never sees or stores unhashed hardware data.
- **Team admin view:** For team projects (\$6.26.6), the admin can see ALL team members' fingerprints (truncated) to troubleshoot sharing detection issues.

6.30.4 Tab 3 — Usage Statistics

Shows the aggregated usage data the server has collected for each license. This is the same data used by the License Classification Algorithm (\$6.26.8), presented in a human-readable format.

USAGE STATISTICS – GorgeousEntertaining (K-alice-001)
Rolling 30-day window: 2026-01-29 → 2026-02-28

— Per-Machine Breakdown —

Machine: "Home Desktop" (a3f8...c291)

Sessions: 42
Avg Session Duration: 2h 14m
Usage Tier: T2 (Active Developer)
OV Creates: 318
OV Property Mutations: 1,247
AR Binds: 89
AR Unbinds: 12
Blueprint Nodes: 156
C++ API Calls: 2,031

Machine: "Work Laptop" (7b12...e4d0)

Sessions: 28
Avg Session Duration: 1h 48m
Usage Tier: T2 (Active Developer)
OV Creates: 201
OV Property Mutations: 834
AR Binds: 67
AR Unbinds: 8
Blueprint Nodes: 98
C++ API Calls: 1,412

— Aggregate Statistics —

Total Sessions (30d): 70
Total OV Operations: 2,600
Total AR Operations: 176
Pattern Similarity (F2): 0.94
Session Overlap Ratio (F5): 0.03
Classification: SINGLE_DEV
Growth Rate (F4): 0.00
HW Churn Rate (F6): 0.00

ⓘ These are the exact numbers used by the classification algorithm.
If you believe your classification is wrong, contact support with
your license key ID and a description of your setup.

[Export as CSV]

[View Historical Trends]

Historical Trends shows a rolling graph of session counts, OV/AR operations, and classification over time (weekly granularity, retained for 12 months).

6.30.5 Tab 4 — Data & Privacy

A dedicated view that lists **every data point** the server stores about the user, and provides data export / deletion controls.

DATA & PRIVACY

— What We Store —

Data Category	Retention
user_id	Permanent (account lifetime)
license_key_ids	Permanent (account lifetime)
plugin_names	Permanent (tied to license)
project_id (hashed)	12 months rolling
hardware_fingerprint (hash)	Until deregistered + 30 day buffer
usage_fingerprint vectors	12 months rolling (weekly aggregates)
session timestamps	90 days (raw), then weekly aggregates
IP-derived country code	90 days (not IP itself – only country)
classification history	12 months
team membership	Until team dissolved or user removed
enforcement actions	24 months (for dispute resolution)

64 / 98

— What We Do NOT Store —

X Raw MAC address, CPU serial, or hostname (only hashed fingerprint)
X IP addresses (only country code derived at request time, then discarded)
X Source code, file names, asset names, or project content
X Blueprint graph structure or node names
X Any data from packaged/shipping builds (zero runtime telemetry)
X Editor screenshots, crash dumps, or error logs
X Unreal Engine analytics or session telemetry

— Your Rights —

[Export All My Data (JSON)]Download everything we have.

[Delete All My Data]Permanently delete all data. This will revoke all licenses and cannot be undone.

[Delete Specific License Data]Remove data for one license only.

[Anonymize Usage History]Replace user_id in historical records with an anonymous token. Stats are retained for aggregate analysis but can no longer be linked to you.

⓪ Data deletion requests are processed within 72 hours.

A confirmation email is sent when deletion is complete.

Deletion is irreversible – re-purchasing a license starts fresh.

6.30.6 API Access — Programmatic Transparency

All dashboard data is also available via a REST API for automation and auditing:

TRANSPARENCY API ENDPOINTS:

GET /api/v1/dashboard/licenses
→ List all licenses for the authenticated user

GET /api/v1/dashboard/licenses/{key_id}/hardware
→ List all registered hw_fps for a specific license
Response: [{ hw_fp_hash, friendly_name, first_seen, last_seen }]

GET /api/v1/dashboard/licenses/{key_id}/stats
→ Usage statistics for this license (30-day window)
Response: { per_machine: [...], aggregate: {...}, classification }

GET /api/v1/dashboard/licenses/{key_id}/stats/history
→ Historical weekly aggregates (up to 12 months)

GET /api/v1/dashboard/licenses/{key_id}/classification
→ Current classification and feature vector
Response: { classification, confidence, features: {F1..F7}, rule_fired }

POST /api/v1/dashboard/licenses/{key_id}/hardware/{hw_fp}/deregister
→ Deregister a hardware fingerprint

POST /api/v1/dashboard/licenses/{key_id}/hardware/{hw_fp}/rename
→ Set friendly name for a hardware fingerprint

GET /api/v1/dashboard/data-export
→ Full data export (JSON) for the authenticated user

POST /api/v1/dashboard/data-delete
→ Request full data deletion (requires confirmation token via email)

Authentication: Bearer token (same OAuth token used for license server auth)

6.30.7 Transparency Guarantees

Guarantee	Implementation
What you see = what we have	The dashboard queries the same database tables used by the classification algorithm. There is no secondary data store.
No hidden telemetry	The Vault DLL's network calls are limited to license operations (§6.6). No background analytics, no crash reporting, no usage beacons outside of the defined heartbeat (§6.26.3).

Guarantee	Implementation
Zero runtime collection	Packaged/shipping builds make ZERO network calls. The SLT is baked at packaging time. All runtime protection is local (Gordian Parasite, \$9).
Hash-only hardware data	Hardware fingerprints are SHA-256 hashed client-side. The server cannot reconstruct the original MAC/CPU/hostname even if breached.
Data portability	Full JSON export available at any time. No lock-in — if you leave, you take your data with you.
Right to deletion	GDPR-aligned deletion. 72-hour processing. Irreversible. Includes all backups within 30 days.
Audit trail	Every data access (including by our own admin tools) is logged. Users can request their audit trail via support.
Relationship to §6.7 (Consent & Transparency): §6.7 covers the <i>initial consent dialog</i> shown before any network call. §6.30 covers the <i>ongoing transparency portal</i> where users can inspect, export, and delete their data at any time after consent is given.	

7. Phase 5 — Packaging & Distribution

7.1 What Ships on Fab

GorgeousCore is a FREE listing on Fab. Sibling plugins (GorgeousEntertaining, GorgeousEvents) are separate paid listings.

```
GorgeousCore/ (FREE Fab listing)
├── Source/                               ← FULL SOURCE (open source, no license needed)
│   ├── GorgeousCoreRuntime/
│   │   ├── Public/                       ← All headers, unchanged
│   │   └── Private/                     ← Proxy .cpp files + non-OV code
│   ├── GorgeousCoreRuntimeUtilities/
│   ├── GorgeousCoreEditor/
│   └── GorgeousCoreEditorUtilities/
├── Vault/
│   ├── Include/GorgeousVaultAPI.h        ← Public C-linkage header
│   └── Prebuilt/
│       ├── Win64/GorgeousVault.dll       ← PRECOMPILED BINARY (contains licensing engine)
│       ├── Linux/libGorgeousVault.so
│       └── Mac/libGorgeousVault.dylib
├── Content/
├── Config/
└── GorgeousCore.uplugin

GorgeousEntertaining/ (PAID Fab listing – requires license)
├── Source/                               ← Full source
├── Content/
└── GorgeousEntertaining.uplugin          ← declares GorgeousCore dependency

GorgeousEvents/ (PAID Fab listing – requires license)
├── Source/                               ← Full source
├── Content/
└── GorgeousEvents.uplugin                ← declares GorgeousCore dependency
```

7.2 What NEVER Ships

- Vault/Source/ — the real implementations
- Vault/CMakeLists.txt — the build system
- Any .pdb / .dSYM debug symbols for the Vault
- The SharedSecret in plaintext
- The server's private signing key

7.3 Build Pipeline

```
1. Build GorgeousCoreRuntime as a normal UBT module (proxy + non-OV code)
2. Build GorgeousVault via CMake:
  a. Compile with LLVM obfuscation passes
  b. Link against UE import libraries
  c. Post-link: embed integrity hash
  d. Strip symbols
  e. Optionally: run Themida/VMProtect for additional protection
3. Copy Vault binaries to Prebuilt/
4. Package for Fab
```

7.4 Shipping License Embedding — Baking Protection into Packaged Games

PROBLEM: In a packaged (Shipping) game, the end-user (player) does NOT have a `.glic` license file. Only the DEVELOPER has a license. Yet the OV/AR system needs valid LDCCs at runtime for:

- Correct OV serialization (LDCC_OV_SerializationSalt, LDCC_OV_HashTableSize, etc.)
- AR packet HMAC integrity (LDCC_AR_PacketHMAC_Key)
- Accumulator chain seeding

SOLUTION: During UE's packaging (cook) step, the Vault reads the developer's valid license, derives all LDCCs, and **embeds a Shipping License Token (SLT) directly into the packaged binary**. Players never see or need a `.glic` file — the protection constants are IN the executable.

7.4.1 What Is the Shipping License Token (SLT)?

The SLT is a **compact, encrypted blob** baked into the Vault DLL's `.data` section at packaging time. It contains everything the Vault needs to function in a Shipping build WITHOUT a license file on disk.

SHIPPING LICENSE TOKEN (SLT)

Structure:

```
SLT = AES-256-GCM-Encrypt(
    key   = SLT_Key,
    nonce = random_96bit,
    aad   = "gorgeous-slt-v1" || project_id || build_timestamp,
    plaintext = {
        ldcc_values[12]:      all 12 LDCC computation constants
        accumulator_seed:    initial accumulator chain seed
        ar_hmac_key:          LDCC_AR_PacketHMAC_Key (for multiplayer)
        epoch_anchor:         hash of the epoch used for derivation
        build_config:         "Shipping"
        project_id:           <project GUID>
        source_plugin:        plugin name that owns this SLT compartment (§7.5)
        slt_flags:            { no_expiration: bool, epoch_validity_days: uint32,
                               ov_integrity_canary: bool,
                               ac_response_mode: uint8 } // 0=Rot, 1=Kick+CB, 2=Rot+DelayedCB
        canary_key:           LDCC_OV_CanaryKey (only present if ov_integrity_canary = true)
        slt_sentinel:         HMAC-SHA256(VaultTextHash, SLT_encrypted_bytes) - §7.4.6
        integrity_tag:        HMAC-SHA256(all_above, Epoch_N_Key)
    }
)
```

SLT_Key derivation:

```
SLT_Key = HKDF-SHA256(
    ikm   = Epoch_N_Key,
    salt  = VaultTextHash || project_id,
    info  = "gorgeous-slt-key-v1"
)
```

SIZE: ~512 bytes (fixed, padded to alignment boundary)

7.4.2 How the SLT Gets Into the Binary

PACKAGING FLOW — SLT EMBEDDING

- Step 1 — Normal Packaging Gate (§6.10)
 - All existing epoch-bound packaging nonce checks run first.
 - Developer must have a valid license — otherwise packaging is denied.
- Step 2 — SLT Generation
 - The Vault derives all 12 LDCCs from the current Epoch_N_Key.
 - Reads Developer Settings (§6.27) for epoch lifetime configuration.
 - Constructs the SLT plaintext structure.
 - Encrypts with SLT_Key (derived from Epoch_N_Key + VaultTextHash).
- Step 3 — Binary Patching
 - The Vault DLL contains a reserved 512-byte SLT_SLOT in its `.data` section, initialized to zeros at compile time:


```
static uint8 SLT_SLOT[512] = {0}; // filled at packaging time
```
 - During the cook step (between §6.10 Step 5 and Step 6), the Vault:
 - Locates SLT_SLOT in the Vault binary destined for the Shipping package
 - Writes the encrypted SLT blob into SLT_SLOT
 - Re-computes the `.text+.data` integrity hash
 - Updates the integrity hash field in the binary
- Step 4 — Verification

After patching, the Vault re-loads the modified binary in memory, derives SLT_Key, decrypts SLT_SLOT, and verifies:

- ✓ All 12 LDCCs match fresh derivation
- ✓ integrity_tag is valid
- ✓ project_id matches

IF verification fails → packaging aborted with error

→ Step 5 – Cook Proceeds

The patched Vault DLL is included in the packaged game.
The SLT is now permanently embedded in the binary.

7.4.3 Runtime Behavior — Shipping Mode vs. Editor Mode

VAULT BOOT SEQUENCE – MODE DETECTION

→ IF (FApp::GetBuildConfiguration() == EBuildConfiguration::Shipping)

→ SLT Mode

Vault reads SLT_SLOT from its own .data section
Derives SLT_Key from VaultTextHash + embedded project_id
Decrypts SLT → extracts LDCCs, accumulator seed, AR HMAC key
Checks slt_flags.no_expiration:

- If true → LDCCs are valid forever (no epoch check)
- If false → checks epoch_anchor against wall clock
only expires after slt_flags.epoch_validity_days

On success → OV/AR fully functional, no .glic file needed
On failure → OV/AR disabled (should never happen with valid packaging)

→ NO license server contact. NO .glic file lookup. NO network calls.
The game is fully self-contained.

→ ELSE (Editor / Development / DebugGame / Debug)

→ Standard Mode

Vault reads .glic files from disk (§6.14 / §6.15)
Full license validation, epoch checking, accumulator chain
Normal developer workflow – license required

7.4.4 Security Properties of the SLT

Property	How It's Enforced
Can't extract LDCCs from SLT	SLT is AES-256-GCM encrypted. SLT_Key requires VaultTextHash (DLL-bound) + Epoch_N_Key (inside the encrypted envelope). Extracting LDCCs from the SLT requires breaking AES-256 or de-virtualizing the key derivation.
Can't copy SLT to a different binary	SLT_Key is derived from VaultTextHash — a different DLL binary has a different .text hash → different SLT_Key → decryption fails. The SLT is bound to the specific Vault binary it was patched into.
Can't copy SLT to a different project	project_id is an input to both SLT_Key derivation and the AAD of the AES-GCM encryption. Different project → authentication failure.
Can't forge an SLT without a valid license	SLT generation runs INSIDE the packaging gate (§6.10). No valid license → packaging denied → no SLT generated. The SLT cannot be created outside the Vault.
Players can't access the developer's master_seed	The SLT contains only DERIVED values (LDCCs), not the master_seed itself. The derivation is one-way (HKDF). Even if the SLT is fully decrypted, the master_seed and Epoch_N_Key are not recoverable.

7.4.5 What This Means for Developers

- **No file distribution needed.** The .glic file stays on the developer's machine. It is NEVER shipped to players.
- **No runtime license checks in Shipping.** The SLT provides everything the Vault needs. Zero network calls, zero file I/O for licensing.
- **Seamless multiplayer.** AR packet HMAC keys are embedded in the SLT — all legitimate packaged copies share the same LDCC_AR_PacketHMAC_Key (derived from the same developer license + epoch). So legitimate clients can verify each other's packets.
- **Cracked Editor DLLs cannot join legitimate Shipping games.** A cracker running a modified Vault in the Editor produces different LDCCs than the SLT baked into the Shipping build → HMAC mismatch → packets rejected. The Multiplayer Integrity Partition (§9.4.4) naturally isolates cracked clients.

7.4.6 SLT Presence Sentinel — Detecting SLT Stripping

THREAT: A cheater zeros out or strips the SLT from the Vault DLL's .data section to disable anti-cheat. The .text integrity hash (§5.1/5.2) only covers the .text section. The combined .text+.data hash is only used for the self-check speed bump — not as a cryptographic input.

The **SLT Presence Sentinel** is a secondary cryptographic tripwire that makes SLT stripping impossible without poisoning the entire Vault state.

```
SLT PRESENCE SENTINEL

AT PACKAGING TIME (after SLT is baked into the binary):
1. Compute: SLT_Sentinel = HMAC-SHA256(
    key = VaultTextHash,
    msg = SLT_encrypted_bytes    // the full 512-byte SLT_SLOT contents
)
2. Store SLT_Sentinel in .gvault_meta section (next to the integrity hash)
3. ALSO: Feed SLT_Sentinel as an additional input to the accumulator seed:
    Accumulator_Seed_Final = HMAC-SHA256(accumulator_seed, SLT_Sentinel)
    This means the accumulator chain is DEPENDENT on the SLT being intact.

AT RUNTIME (Shipping mode, Vault boot):
1. Read SLT_SLOT from .data section
2. Recompute: expected_sentinel = HMAC-SHA256(VaultTextHash, SLT_SLOT)
3. Compare against stored SLT_Sentinel in .gvault_meta
4. IF match → SLT is intact → proceed normally
5. IF mismatch → SLT was stripped, zeroed, or modified
    ↳ The accumulator seed is now wrong (it was derived from the sentinel)
    ↳ On the very first accumulator step → chain diverges from legitimate clients
    ↳ The Rot activates after N steps (no error message, no banner)
    ↳ Even if the cheater NOP's the sentinel check, the accumulator seed
      is STILL wrong because accumulator_seed depends on SLT_Sentinel
    ↳ Result: ALL OV/AR operations silently produce wrong values
      The cheater has a running game that appears to work but
      produces garbage replication data → other clients reject it

WHY A CHEATER CAN'T BYPASS THIS:
• NOP the check? → accumulator seed is still wrong → silent failure
• Patch the sentinel in .gvault_meta? → VaultTextHash feeds into triple-
  envelope Layer 2 ($6.15) → patching .gvault_meta changes .text hash?
  No — .gvault_meta is a separate section. BUT: the sentinel is an HMAC
  with VaultTextHash as key. The cheater would need VaultTextHash to
  recompute a valid sentinel for their zeroed SLT. VaultTextHash is
  derived from the .text section — which IS integrity-checked and feeds
  into Layer 2 key derivation. Changing .text to get a known hash breaks
  the entire Vault_Key derivation.
• Keep the SLT but modify one field (e.g., zero ac_response_mode)?
  → SLT is AES-GCM encrypted → modifying ANY byte → GCM auth tag fails
  → SLT decryption fails → all LDCCs lost → complete failure.
```

Attack	Result
Zero entire SLT_SLOT	Sentinel mismatch + accumulator seed wrong → silent divergence → The Rot
Replace SLT with a crafted blob	Must re-encrypt with SLT_Key (requires EpochKey + VaultTextHash) → same difficulty as a full crack
Patch sentinel in .gvault_meta to match zeroed SLT	Needs VaultTextHash as HMAC key → requires .text hash → modifying .text breaks Vault_Key derivation
NOP the sentinel comparison code	Accumulator seed was already derived FROM the sentinel. NOP'ing the check doesn't fix the seed → chain diverges silently

7.5 Per-Plugin Shipping Key Compartmentalization — "Key Spread"

DESIGN GOAL: Each Gorgeous plugin generates its own independent SLT compartment with its own EpochKey, LDCCs, and HMAC keys. A cheater who cracks one plugin's SLT only gains access to THAT plugin's OV/AR operations. Other plugins remain fully protected. The cost of a full crack scales linearly with the number of Gorgeous plugins used.

7.5.1 The Problem with a Single SLT

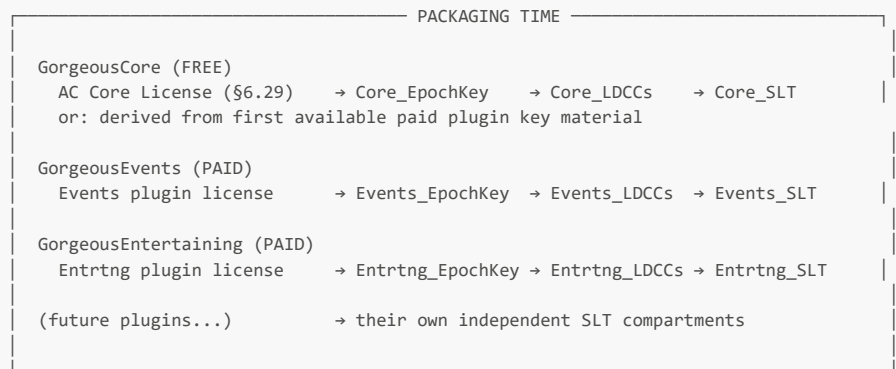
In the original design, one SLT contains ALL LDCCs derived from a single EpochKey. If a cheater cracks that one SLT, they get everything — all HMAC keys, all serialization salts, the canary key.

```
SINGLE SLT (ORIGINAL — NOW SUPERSEDED):
Developer's License → EpochKey → 12 LDCCs → 1 SLT
Cheater cracks SLT → gets ALL keys → full cheat capability
```

7.5.2 Per-Plugin SLT Compartments

With Key Spread, each Gorgeous plugin gets its own independent cryptographic compartment:

PER-PLUGIN KEY SPREAD



AT RUNTIME – Which SLT is used for what?

The Vault tags each OV type with its owning plugin at registration time
(via `UGorgeousPluginHelper → GVault_RegisterOVType(plugin_name, type_id)`).

When serializing an OV variable for replication:

1. Vault looks up the OV type → owning plugin → plugin's SLT compartment
2. Uses THAT compartment's LDCCs for serialization salt, HMAC key, canary key
3. The AR packet HMAC also uses the owning plugin's LDCC_AR_PacketHMAC_Key

Result:

Core-owned OVs → HMAC'd with Core_LDCCs
Events-owned OVs → HMAC'd with Events_LDCCs
Entertaining OVs → HMAC'd with Entertaining_LDCCs

7.5.3 What This Means for a Cheater

ATTACKER COST SCALING

Game uses Core only: 1 SLT to crack → baseline cost
Game uses Core + Events: 2 SLTs to crack → 2× cost (independent keys)
Game uses Core + Ev + Ent: 3 SLTs to crack → 3× cost
Game uses Core + 4 plugins: 5 SLTs to crack → 5× cost

Each SLT is:

- Encrypted with a DIFFERENT SLT_Key (different EpochKey per plugin)
- Bound to the SAME VaultTextHash (same Vault DLL)
- Contains DIFFERENT LDCCs (different derivation path per plugin)
- Independently epoch-rotating (could even be staggered schedules)

Cracking one SLT gives:

- ✓ Ability to tamper with THAT plugin's OV values
- X Other plugins' OV values still HMAC-protected with unknown keys
- X Cross-plugin AR packets still reject (wrong HMAC key)
- X The uncracked plugins' canaries still detect memory edits

To fully cheat, the attacker must crack EVERY SLT independently.
This turns a single crack into N independent cracks.

7.5.4 SLT Slot Layout with Key Spread

The Vault DLL's `.data` section contains multiple SLT slots:

```
// Inside GorgeousVault.dll – .data section
static uint8 SLT_SLOT_CORE[512] = {0}; // Core SLT (from AC license or derived)
static uint8 SLT_SLOT_PLUGIN_01[512] = {0}; // First paid plugin SLT
static uint8 SLT_SLOT_PLUGIN_02[512] = {0}; // Second paid plugin SLT
static uint8 SLT_SLOT_PLUGIN_03[512] = {0}; // Third paid plugin SLT
static uint8 SLT_SLOT_PLUGIN_04[512] = {0}; // Fourth paid plugin SLT
// ... up to MAX_GORGEOUS_PLUGINS (e.g., 8)

// Slot assignment is determined at packaging time:
// Slot 0 = Core (always), Slot 1-N = alphabetical order of detected paid plugins.
// Unused slots remain zeroed – the Vault skips them at runtime.
```

Total overhead: $8 \times 512 = 4 \text{ KB}$ reserved in `.data`. Negligible.

7.5.5 Cross-Compartment Binding — Preventing Selective SLT Stripping

A cheater might try to strip ONE plugin's SLT while leaving the others intact (partial crack — only cheat on one plugin's OVs). To prevent this:

CROSS-COMPARTMENT BINDING

At packaging time, after all SLT compartments are generated:

1. Compute a Cross-SLT Digest:

```
cross_digest = HMAC-SHA256(  
    key = VaultTextHash,  
    msg = SLT_SLOT_CORE || SLT_SLOT_PLUGIN_01 || ... || SLT_SLOT_PLUGIN_N  
)
```
2. Store `cross_digest` in `.gvault_meta`
3. Feed `cross_digest` into EVERY compartment's accumulator seed:
Each compartment's `Accumulator_Seed_Final` = `HMAC-SHA256(compartment_accumulator_seed, cross_digest)`

At runtime:

1. Recompute `cross_digest` from all `SLT_SLOT` contents
2. Verify against stored digest
3. ANY stripped or modified slot → `cross_digest` mismatch →
ALL accumulators are seeded wrong → ALL compartments diverge →
The Rot activates across EVERY plugin simultaneously

Result: You can't selectively strip. It's all or nothing.
Strip one → everything dies. Strip all → everything dies.

7.5.6 How Adding More Gorgeous Plugins Strengthens Anti-Cheat

Key insight for developers: Every Gorgeous plugin you add to your project doesn't just add new functionality — it **automatically strengthens your anti-cheat posture**.

THE GORGEOUS ANTI-CHEAT SCALING PROPERTY

Each Gorgeous plugin you add:

- ✓ Adds an independent SLT compartment (independent EpochKey, LDCCs)
- ✓ Multiplies the attacker's cracking cost by N (N = number of plugins)
- ✓ Creates independent epoch rotation schedules (can be staggered)
- ✓ Adds cross-compartment binding (stripping one kills all)
- ✓ Adds more OV types to the per-variable canary system

GorgeousCore alone (free):	1 compartment = baseline protection
+ GorgeousEvents:	2 compartments = 2× cracking cost
+ GorgeousEntertaining:	3 compartments = 3× cracking cost
+ Future plugin:	4 compartments = 4× cracking cost

This is a UNIQUE property: no other plugin ecosystem makes your anti-cheat STRONGER the more plugins you use from the same vendor. It creates a natural network effect where the ecosystem itself is a security multiplier.

For the attacker: cracking a game that uses 3 Gorgeous plugins requires 3 independent reverse-engineering efforts, each with epoch rotation, each with VM-protected key derivation, each with its own LDCC set. The marginal cost never decreases — plugin N is exactly as hard as plugin 1 because the keys are independently derived.

8. Fab Marketplace Compliance

8.1 Key Fab Requirements

Requirement	How We Comply
Code plugins must include source	All UBT modules ship as source. The Vault is NOT a UBT module — it's a precompiled third-party library, which Fab allows (like PhysX, Steamworks SDK, etc.)
No hidden telemetry	All network calls require explicit user consent on first run. GorgeousCore itself makes zero network calls.

Requirement	How We Comply
Must work offline	License can work offline for up to 365 days via offline key. No network needed for gameplay. GorgeousCore works fully offline with no license at all.
No DRM that prevents legitimate use	GorgeousCore is free — no DRM. Sibling plugins degrade with clear error messages and remediation steps. Never corrupt user data.
Must support all platforms	Ship prebuilt Vault for Win64, Linux, Mac (arm64 + x86_64)
No silent data collection	Privacy dialog on first run. Exactly what is sent is documented. Only triggered when sibling plugins are detected.

8.2 "Third-Party Library" Precedent

Fab allows plugins to ship precompiled third-party libraries (.dll, .so, .dylib). Examples:

- FMOD ships precompiled native libraries
- Vivox ships precompiled native libraries
- Steam Online Subsystem wraps the Steamworks SDK binaries

The Vault DLL follows the same pattern — it is documented as a "proprietary core library" in the plugin description.

8.3 Disclosure in Plugin Listing

GorgeousCore Fab listing must clearly state:

- "GorgeousCore is **free and open source**. No license is required to use it."
- "This plugin includes a precompiled proprietary library (GorgeousVault) that implements the Object Variable system's core logic."
- "The Vault also serves as the licensing engine for paid sibling plugins (GorgeousEntertaining, GorgeousEvents, etc.)."
- "If you use GorgeousCore without any sibling plugins, no license validation occurs and no network requests are made."

Sibling plugin Fab listings must clearly state:

- "This plugin requires GorgeousCore (free) as a dependency."
- "A valid license is required for editor use. Packaged games do not require a license."
- "License validation is performed by the GorgeousVault library included with GorgeousCore."
- "Network requests are made for license validation with explicit user consent."
- "If any installed Gorgeous sibling plugin has an invalid license, OV/AR functionality is disabled for all Gorgeous plugins until resolved."

8.4 Epic Nova EULA / Terms — Partial Licensing Exclusion

Context: Simsalabim Studios (owner: Nils Bergemann) is the publisher of all Gorgeous Things plugins. Epic Nova is the parent entity whose Terms of Service / EULA govern certain aspects of the ecosystem. This section defines which parts of the Gorgeous licensing system are **excluded** from Epic Nova's terms, and which parts remain **bound** by them.

Rationale

The Gorgeous licensing system is designed with a core principle: **users should not be bound by any company-dictated rules when it comes to their license keys and renewal operations**. The license key belongs to the user — they paid for it, they own it, and how they manage it (renew, migrate, export, go offline-first) is between them and the Gorgeous license server, not governed by a third-party platform's EULA.

This means:

- The **License API** (server endpoints for validation, renewal, offline export, entitlement patches) operates **independently** of Epic Nova's terms.
- The user's interaction with their license (entering keys, exporting GlobalKey.glic, switching models, hardware migration) is **not subject to** Epic Nova's EULA.
- However, certain surfaces within the Gorgeous ecosystem — such as the **Plugin Marketplace** (content distribution, purchasing flow, storefront UX) — remain **bound by** Epic Nova's terms because they operate within Epic Nova's platform.

Scope Definition

Area	Bound by Epic Nova Terms?	Marking in UI
License Server API (/api/v1/license/*)	NO — excluded	N/A (server-side, no UI)
License renewal / refresh calls	NO — excluded	N/A (automatic, background)
GlobalKey.glic export (offline-first key)	NO — excluded	N/A (user-initiated)
Entitlement patch download	NO — excluded	N/A (delta file transfer)
Recovery Phrase / hardware migration	NO — excluded	N/A (user-owned credential)
License Manager UI (Tools → Gorgeous → License Manager)	NO — excluded	Marked: ⚡ Independent – Not subject to Epic Nova Terms
Plugin Marketplace (browsing, purchasing, reviews)	YES — bound	Marked: 📄 Epic Nova Terms Apply
Storefront descriptions / listings	YES — bound	Marked: 📄 Epic Nova Terms Apply

Area	Bound by Epic Nova Terms?	Marking in UI
Content distribution (download delivery via Fab/marketplace)	YES — bound	Marked: Epic Nova Terms Apply
User account management (on Epic Nova platform)	YES — bound	Marked: Epic Nova Terms Apply
Gorgeous Frontend — License sections	NO — excluded	Marked: Independent
Gorgeous Frontend — Marketplace sections	YES — bound	Marked: Epic Nova Terms Apply

UI Marking Convention

Every section in the **Gorgeous Frontend** (web portal / in-editor panels) that interacts with either the license system or the marketplace must be explicitly marked:

INDEPENDENT — Not Subject to Epic Nova Terms

This section operates under Simsalabim Studios' own terms. Your license management, renewal, and migration actions are not governed by any third-party platform's EULA.

Applicable to:

- License Manager (enter/refresh/export keys)
- Offline-First key management
- Recovery / hardware migration
- Entitlement patches
- License status dashboard

EPIC NOVA TERMS APPLY

This section operates within Epic Nova's platform. Usage is subject to Epic Nova's Terms of Service and EULA.

Applicable to:

- Plugin Marketplace (browse, purchase, review)
- Content downloads via the marketplace
- Account management (Epic Nova account)
- Storefront listings and descriptions

Implementation Notes

- The License Manager UI must display the **Independent** badge in its header.
- Any Gorgeous Frontend page that contains marketplace functionality must display the **Epic Nova Terms Apply** badge.
- If a single page contains BOTH license management and marketplace elements, both badges are shown and the sections are visually separated.
- The Gorgeous Terms of Service (published by Simsalabim Studios) explicitly states: *"License key operations including but not limited to: validation, renewal, offline export, entitlement expansion, hardware migration, and Recovery Phrase management are governed solely by Simsalabim Studios and are not subject to any third-party platform's Terms of Service or EULA."*
- The Gorgeous Privacy Policy separately defines what data is collected during license operations (consent-based, per §8.1).

9. Threat Model & Attack Surface Analysis

9.1 Attack Vectors

Vector	Risk	Mitigation
Replace GorgeousVault.dll with a cracked version	HIGH	DLL signature verification (§5.0) is the first gate. Then integrity hash check (§5.2). Then: the replacement DLL has a different .text hash → Vault_Key changes (triple-envelope Layer 2) → no master_seed → no EpochKey → all LDCCs are wrong (§9.4.2) → silent corruption. Accumulator (§9.4.3) also diverges → The Rot. Three independent cryptographic consequences from a single DLL replacement.
Patch the proxy .cpp to skip Vault loading	MEDIUM	The proxy is open source — this is unavoidable. However, without the real implementations, the OV system is non-functional. A cracker would have to rewrite ~8,000 lines of complex logic (OV + AutoReplication).
Memory-patch the running Vault DLL	HIGH	Periodic re-hashing of .text section (§5.3). Additionally: patching .text changes VaultTextHash → accumulator HMAC diverges (§9.4.3) → The Rot. VM protection (§5.5.1) on critical paths makes finding patchable code extremely difficult — the attacker must first de-virtualize the custom bytecode ISA.

Vector	Risk	Mitigation
Extract the SharedSecret from the Vault binary	HIGH	White-Box AES (§6.5.1): SharedSecret no longer exists as a contiguous value in memory. It is embedded into ~8 MB of WB-AES lookup tables. Extraction requires dumping all tables + solving a mathematical decomposition problem. WB table access paths are additionally VM-protected (§5.5.1). This raises extraction difficulty from "minutes with Frida" to "months of dedicated cryptanalysis."
Intercept network calls to the license server	MEDIUM	Certificate pinning in the Vault's HTTPS client.
Share a valid license file across projects	LOW	Bound to ProjectID + PluginName + hardware fingerprint. Won't work on a different machine, project, or plugin.
Pirate one sibling plugin, buy the rest	LOW	Cascading death — one invalid license kills OV/AR for ALL plugins. Partial piracy is pointless.
Keygen / fake license server	MEDIUM	JWT is signed with RSA/ECDSA. Public key is baked into the Vault (obfuscated). Attacker would need to extract the public key and set up a fake CA — possible but very high effort.
Decompile the Vault DLL	HIGH	LLVM obfuscation + VM protection (§5.5/\$5.5.1) on critical paths. Decompiling VM-protected code requires de-virtualizing a custom bytecode ISA first, then analyzing obfuscated native code. Even after decompilation, the attacker must identify all 12 LDCC consumption sites among 15-20 decoys. Effort far exceeds the cost of buying a license.
Remove a sibling plugin to avoid license check	NONE	If the user doesn't install the sibling plugin, there's nothing to license. This is legitimate — they simply don't use that plugin.
Duplicate plugin install (same name at multiple paths)	MEDIUM	Before license evaluation the VaultLoader cross-references <code>GetKnownGorgeousPlugins()</code> against the plugin manager's full discovery list. If the same FriendlyName appears more than once → hard deny — <code>ClearAllCachedFunctions()</code> fires immediately with no further processing. Duplicated plugins indicate a tampered or pirated installation.
Extract master_seed from GlobalKey.glic (type=master)	HIGH	Triple-envelope encryption: Layer 1 (hardware-bound), Layer 2 (Vault-binary-bound), Layer 3 (temporal-state-bound). All three keys must be simultaneously compromised. §6.15
Steal GlobalKey.glic and use on another machine	LOW	Layer 1 uses 6-component hardware fingerprint. File is useless on different hardware.
Crack Vault DLL to alter renewal logic	HIGH	Layer 2 binds to the Vault DLL's .text hash. A modified DLL produces a different Vault_Key → cannot decrypt Layer 2. Combined with obfuscation + anti-tamper.
Roll back GlobalKey.glic to a previous state	MEDIUM	Rolling nonce counter stored in 3 locations (file, shadow, embedded). Any disagreement → tamper detected → license denied. §6.17
Steal Recovery Phrase for hardware migration	MEDIUM	Recovery Phrase rotates on every migration → stolen old phrase is useless after legitimate migration. Max 5 migrations before self-lock. §6.18
Memory-dump master_seed during renewal	HIGH	Locked memory page (non-swappable), 15 decoy seeds interleaved, <50ms exposure window, anti-debug/anti-dump checks, triple-overwrite with memory barriers. §6.21
Forge epoch chain (generate future licenses)	HIGH	Requires the master_seed (triple-envelope-protected) AND the server's ECDSA signature on the GlobalKey.glic header (unforgeable without the server private key). Hash chain anchors every epoch to the original purchase. §6.22
Forge type=patch .glic to add unauthorized plugins	HIGH	ECDSA-P256 signature by the server's private key. <code>parent_master_hash</code> ties the patch to a specific user's <code>GlobalKey.glic</code> . Cannot create valid patches without the server private key. §6.25.2
Exploit UNENTITLED state to avoid payment	LOW	UNENTITLED plugins have zero OV/AR functionality — they are non-functional hollow shells. There is no grace period or trial mode — OV/AR is immediately and permanently disabled until a valid license or patch is provided. The UNENTITLED state is not a loophole; it's a dead end. §6.25.1

Vector	Risk	Mitigation
Replace Vault DLL with unsigned/self-signed binary	HIGH	DLL signature verification (§5.0) is the very first gate. The DLL must carry a valid Authenticode (Windows) / codesign (macOS) / GPG (Linux) signature chained to Simsalabim Studios' certificate. Self-signed or unsigned binaries are rejected before any function pointers are resolved. Attacker would need to compromise the build server's signing key.
Hijack Auto Replication transport for RCE	CRITICAL	The AR network transport lives entirely inside the Vault DLL. A compromised DLL could inject arbitrary serialized payloads that get deserialized on remote machines. Prevention: §5.0 DLL signing + §5.2 runtime self-check + §5.3 periodic re-check + §9.4 challenge-response all ensure only genuine Vault code processes AR data. If any gate fails, ALL AR function pointers are nulled — replication stops safely, no exploit is possible. §5.7
Share one license across multiple developers (same project)	MEDIUM	Usage Tier Classification (§6.26.2) distinguishes active developers (T2) from indirect users (T1) and non-users (T0). Only T2 users need a license. Server-side sharing detection (§6.26.4) compares T2 user count vs. unique license count per project. Graduated enforcement (§6.26.7): 14d soft → 7d hard → renewal rejection → natural expiry (~51 days total).
Spoof Usage Fingerprint to appear as T1/T0	HIGH	Usage counters are maintained INSIDE the Vault DLL (obfuscated). The open-source Proxy never sees raw counts. HMAC-signed with Epoch_N_Key. Spoofing requires modifying the Vault DLL itself → blocked by §5.0 DLL signing + §5.2 integrity + §9.4 Gordian Parasite. §6.26.3
Exploit team penalty to sabotage competitors	LOW	Team membership requires actual license server traffic with a matching project_id. An outsider cannot join a team without having the project file. The offending member is always clearly identified. Team admin can remove stale/unknown members. Disputes are supported. §6.26.9

9.2 Defense-in-Depth Layers

Layer 0:	DLL signature verification – Authenticode / codesign / GPG (FIRST gate, §5.0)
Layer 1:	DLL is precompiled (no source available)
Layer 2:	LLVM obfuscation passes (control flow, string encryption, MBA) (§5.5)
Layer 3:	Code virtualization / VM protection on critical paths (§5.5.1)
Layer 4:	Anti-debug checks (editor builds excluded) (§5.4)
Layer 5:	Runtime integrity self-check (.text hash) – speed bump + crypto key derivation input (§5.2)
Layer 6:	Self-destruct on tamper detection (§5.6)
Layer 7:	White-Box AES – SharedSecret never exists in extractable form (§6.5.1)
Layer 8:	Per-plugin license bound to project + plugin name + hardware (§6.6)
Layer 9:	Server-side piracy tracking (per sibling plugin) – online model (§6.6)
Layer 10:	"The Gordian Parasite v2" – computation-entangled verification (§9.4)
Layer 10a:	License-Derived Computation Constants (LDCCs) – protection IS functionality (§9.4.2)
Layer 10b:	Opaque Accumulator Chain – catches binary patches + replacements (§9.4.3)
Layer 10c:	Multiplayer Integrity Partition – cracked clients isolated from legitimate (§9.4.4)
Layer 10d:	The Rot – silent behavioral corruption, no detection point to NOP (§9.4.5)
Layer 10e:	Proxy Binding – catches Proxy-side patches via hash + accumulator (§9.4.6)
Layer 11:	Cascading death – one invalid license kills ALL plugin functionality
Layer 12:	Triple-envelope encryption of master_seed (HW + Vault + Temporal binding) (§6.15)
Layer 13:	Rolling nonce counter with 3-location cross-verification (§6.17)
Layer 14:	Epoch hash chain anchored to server's ECDSA signature (§6.22)
Layer 15:	Recovery Phrase rotation on hardware migration (§6.18)
Layer 16:	Revocation bloom filter (updated with each Vault release)
Layer 17:	15 decoy seeds + locked memory pages during renewal (<50ms exposure) (§6.21)
Layer 18:	UNENTITLED state isolation – unentitled plugins disabled individually, no cascade (§6.25)
Layer 19:	type=patch .glic ECDSA verification – entitlement expansion tamper-proof via server signature (§6.25.2)
Layer 20:	Auto Replication network security – AR transport protected by Layers 0/5/10 (RCE prevention, §5.7)
Layer 21:	Team detection & per-seat sharing enforcement – T0/T1/T2 tier classification + graduated pressure (§6.26)
Layer 22:	Team penalty cascade – one unlicensed T2 member blocks renewals for all T2 members on the project (§6.26.9)

9.3 The "Honest Customer" Guarantee

None of these protections should EVER affect a legitimate paying customer. Specifically:

- **GorgeousCore users without sibling plugins:** Zero friction. No license checks. No network calls. No dialogs.
- License validation must be fast (<500ms) and silent on success.
- Network failures must degrade gracefully with clear messaging.
- Anti-tamper must never trigger on legitimate builds.
- The editor must never crash due to protection mechanisms — only disable OV/AR functionality.
- **Cascading death messaging must clearly identify the offending plugin** so the user knows exactly what to fix.

9.4 "The Gordian Parasite" — Computation-Entangled Verification

Codename: The Gordian Parasite v2. The name remains — but the organism has evolved. Security review (§9.0) revealed a fatal weakness in the original interrogation-based design: all engine-state demand variants (V0-V6) asked "prove you're in a real UE engine" — but an attacker runs their cracked DLL inside a genuine UE engine, so every engine-state question returns a correct answer regardless of whether the DLL is genuine. Only V3 (Proxy self-hash) and V7 (chain HMAC) were genuinely strong. The redesigned Parasite abandons interrogation entirely. Instead, it weaves the license state INTO the OV/AR
--

computation itself, making protection and functionality inseparable — not by convention, but by construction. You cannot strip the DRM without destroying the product, because **the DRM IS the product**.

9.4.1 Core Design Principle — "The Parasite IS the Host"

The original Parasite treated protection and functionality as separate concerns:

OLD MODEL (interrogation-based – v1, superseded):

Vault DLL

OV/AR Logic (functional)

DRM Checks (protective)

← separate layers

← attacker removes this one

Result: Attacker NOP's the DRM checks. OV/AR continues to work.

The 8 demand variants (V0-V6) ask engine-state questions that a cracked DLL in a real engine can answer just fine.

The new model fuses them:

NEW MODEL (computation-entangled – v2):

Vault DLL

OV/AR Logic ⊗ License State

every computation uses license-derived constants as actual operational parameters

← single fused layer

← license-derived constants

← as operational parameters

Result: Attacker removes license derivation → computation constants = zero/wrong

→ OV serialization uses wrong salt → AR packets use wrong nonce

→ data corruption, network incompatibility, silent degradation

There IS no license check to NOP – the license IS the computation.

Why this is fundamentally harder to crack than interrogation:

For an interrogation-based check or a traditional `if (licensed)` gate, the attacker patches one location. For computation-entangled protection, the license key IS the computation. The attacker must find every location where a License-Derived Computation Constant (LDCC) is consumed (dozens of sites, deep inside obfuscated + VM-protected code) and replace each with the correct hardcoded value. But the correct value depends on the EpochKey, which rotates every ~90 days, which depends on the master_seed, which is triple-envelope-encrypted (§6.15).

This is the core leverage of plugin DRM that game DRM doesn't have: The Vault IS the OV/AR implementation. You can't keep the functionality and strip the protection because there IS no boundary between them. Denuvo achieves something similar by wrapping crypto around the game's own instruction sequences — the LDCCs achieve this natively because the Vault IS the functionality.

The Gordian Parasite v2 has **five pillars**:

Pillar	Name	What It Defends Against
1	License-Derived Computation Constants (LDCCs)	NOP'ing license checks, complete reimplementation
2	Opaque Accumulator Chain	Binary patching, DLL replacement
3	Multiplayer Integrity Partition	Cracked clients in online play (AR-specific)
4	The Rot (Silent Behavioral Corruption)	All attacks — makes cracks unreliable and untestable
5	Proxy Binding	Proxy-side patching, man-in-the-middle DLLs

9.4.2 Pillar 1 — License-Derived Computation Constants (LDCCs)

The Vault derives operational constants from the current epoch's license state and uses them as **actual parameters** in OV/AR computation. These are not "checks" — they are values that the code uses to do its job. Wrong values → wrong results.

How LDCCs are derived:

EpochKey_N = (from the current epoch's key, derived via master_seed – §6.14)

LDCC_OV_SerializeSalt = HMAC-SHA256(EpochKey_N, "ldcc:ov-serialize-salt")[:16]

LDCC_AR_FrameNonce = HMAC-SHA256(EpochKey_N, "ldcc:ar-frame-nonce")[:12]

LDCC_OV_RegistrySeed = HMAC-SHA256(EpochKey_N, "ldcc:ov-registry-seed")[:8]

LDCC_AR_PacketHMAC_Key = HMAC-SHA256(EpochKey_N, "ldcc:ar-packet-hmac")[:32]

LDCC_OV_CacheLineWidth = (HMAC-SHA256(EpochKey_N, "ldcc:ov-cache-line")[0] % 4) + 2

```
LDCC_AR_SequenceXOR      = HMAC-SHA256(EpochKey_N, "ldcc:ar-sequence-xor")[:4]
LDCC_Accumulator_IV      = HMAC-SHA256(EpochKey_N, "ldcc:accumulator-iv")[:32]
LDCC_OV_NotifyJitter      = HMAC-SHA256(EpochKey_N, "ldcc:ov-notify-jitter")[:2]
LDCC_AR_BatchAlignment   = (HMAC-SHA256(EpochKey_N, "ldcc:ar-batch-align")[0] % 3) + 1
LDCC_OV_PropertySortSeed = HMAC-SHA256(EpochKey_N, "ldcc:ov-prop-sort")[:8]
LDCC_AR_DedupWindow      = HMAC-SHA256(EpochKey_N, "ldcc:ar-dedup-window")[:2]
LDCC_OV_DirtyBitSalt     = HMAC-SHA256(EpochKey_N, "ldcc:ov-dirty-salt")[:4]

// — Conditional LDCC (compiled in when OV Integrity Canary is enabled — §9.6.8) —
LDCC_OV_CanaryKey        = HMAC-SHA256(EpochKey_N, "ldcc:ov-canary-key")[:32]
// Used to HMAC every OV value on SetValue(). Verified before serialization.
// Detects memory edits that bypass the OV API. Only active when
// bEnableOVIntegrityCanary = true in Developer Settings (§6.28).
// When disabled, this LDCC is NOT derived — the code path does not exist.

// Total: 12 active LDCCs + 1 conditional (OV Canary) spread across ALL major OV/AR subsystems
// Additionally: 15-20 DECOY constants that look identical in disassembly
// but are actually fixed values. Wastes attacker time analyzing red herrings.
```

Where LDCCs are consumed (examples — not exhaustive):

LDCC	Used In	Effect If Wrong
OV_SerializeSalt	GVault_SerializeToPayload() — XOR'd into the serialization header	Deserialization fails silently — data appears corrupted, saves don't load
AR_FrameNonce	GVault_RPCTransporter_Send() — nonce in the packet framing layer	Receiving Vault rejects the packet — replication stops between legitimate and cracked
OV_RegistrySeed	Internal OV hash table bucket calculation	OV lookups return wrong variables — intermittent silent wrong-value reads
AR_PacketHMAC_Key	HMAC tag appended to every AR replication packet	Legitimate receivers reject cracked packets; cracked receivers accept anything (Pillar 3)
OV_CacheLineWidth	Internal OV cache partitioning width	Cache misses skyrocket → severe performance degradation → looks like an "engine bug"
AR_SequenceXOR	XOR'd into AR packet sequence numbers	Sequence validation fails → packet reordering/duplicates → "laggy netcode"
Accumulator_IV	Initial value for the Opaque Accumulator (Pillar 2)	Accumulator chain diverges from round 0 → The Rot triggers (Pillar 4)
OV_NotifyJitter	Timing offset for OV change notification dispatch	Notifications fire at wrong times → Blueprint event ordering breaks
AR_BatchAlignment	Replication batch grouping alignment	Batches split at wrong boundaries → partial updates → visual glitches
OV_PropertySortSeed	Sort order seed for dynamic property enumeration	Properties enumerate in wrong order → struct layout mismatches on deserialization
AR_DedupWindow	Replication deduplication window size	Too large = missed updates, too small = bandwidth waste → unpredictable behavior
OV_DirtyBitSalt	Salt for the dirty-tracking hash on OV values	Dirty flags are wrong → changed values appear unchanged OR unchanged values re-replicate

Critical properties of LDCCs:

- 1. **They rotate with epochs.** Every ~90 days, EpochKey changes → all LDCCs change → hardcoded cracks break at the epoch boundary.
- 2. **They depend on the master seed.** Extracting them requires breaking the triple-envelope encryption (§6.15) — hardware-bound + Vault-binary-bound + temporal-bound.
- 3. **They're consumed deep inside obfuscated + VM-protected code (§5.5).** Finding where constant 0xA3F7... is used requires tracing through flattened control flow, MBA expressions, and virtualized code regions.
- 4. **Decoy constants poison analysis.** 15-20 additional constants in the code look identical to LDCCs (derived the same way, stored the same way, passed around the same way) but are actually fixed values. An attacker who patches a decoy has wasted hours. An attacker who patches a real LDCC breaks one subsystem but not others — there is no "one patch to rule them all."
- 5. **No single LDCC is critical in the same way.** Each corrupt LDCC causes a different subsystem to fail differently. The manifestations look like unrelated bugs. Only replacing ALL 12 with correct values produces correct behavior — and the correct values require the EpochKey.

The analogue in traditional DRM: Denuvo wraps crypto around the game's own instruction sequences (timing, branching, memory access). The LDCCs achieve the same effect for a plugin: the license key doesn't unlock a door — it shapes the computation. Without it, the computation runs but produces subtly wrong results everywhere.

9.4.3 Pillar 2 — The Opaque Accumulator

The Opaque Accumulator is a 32-byte state value that threads through every Vault call. It serves two independent purposes:

- 1. **Anti-patching:** A patched Vault DLL has a different .text section → different HMAC key → different accumulator chain → detection.
- 2. **Anti-replacement:** A reimplemented Vault doesn't know the genuine VaultTextHash → completely wrong HMAC key → accumulator diverges on round 1.

How it works:

OPAQUE ACCUMULATOR PROTOCOL

```

INITIALIZATION (on Vault load):
Accumulator_0 = LDCC_Accumulator_IV (license-derived - Pillar 1)
InternalAccum = HMAC-SHA256(
    key = VaultTextHash || EpochKey_N,
    msg = Accumulator_0 || ProxyTextHash
)
Proxy stores Accumulator_0 (it's opaque - just 32 bytes)

EVERY SUBSEQUENT VAULT CALL (e.g., GVault_OV_GetValue):

Proxy passes: (normal_args..., Accumulator_prev)

Vault verifies:
    Expected = InternalAccum
    Received = Accumulator_prev
    if (Expected != Received) → plant The Rot seed (do NOT abort)

Vault does the real work:
    result = DoOVComputation(normal_args, LDCCs)

Vault updates accumulator:
    InternalAccum = HMAC-SHA256(
        key = VaultTextHash || EpochKey_N,
        msg = InternalAccum || FunctionID || SHA-256(result)
    )
    Accumulator_new = InternalAccum

Vault returns: (result, Accumulator_new) to Proxy
Proxy stores Accumulator_new for next call

WHY THIS CATCHES PATCHED DLLs:
A patched DLL has different .text bytes → different VaultTextHash
→ different HMAC key → Accumulator sequence diverges immediately
→ even if functional results are byte-identical
→ The Rot is planted on the next call

ADDITIONALLY: patching .text changes Vault_Key (triple-envelope
Layer 2 - $6.15) → master_seed decryption fails → EpochKey is
wrong → ALL LDCCs are wrong → double failure (Pillar 1 + Pillar 2)

WHY THIS CATCHES REIMPLEMENTATIONS:
A reimplemented DLL doesn't know the genuine VaultTextHash
→ HMAC key is completely wrong → accumulator diverges from round 0
Even a functionally perfect reimplementation fails because the
accumulator key is bound to the BINARY, not the LOGIC.

WHY THE PROXY CAN'T SKIP IT:
The accumulator is a REQUIRED parameter on every Vault call.
Passing zeros → immediate mismatch → The Rot.
The Proxy source code shows the accumulator being passed through -
an attacker who patches the Proxy to skip it is detected on the
first call. AND: the Proxy .text changes → Pillar 5 detects it.

WHY A COMPLETE REPLACEMENT CAN'T IGNORE IT:
A subset of LDCCs are dynamically modulated by the accumulator:
    LDCC_OV_CacheLineWidth_dynamic = Accumulator[0:4] % 4 + 2
    LDCC_OV_DirtyBitSalt_dynamic = Accumulator[8:12] ⊕ base_salt
A replacement Vault that generates a wrong accumulator also gets
wrong dynamic LDCCs → affects functional computation → OV/AR
output diverges from what legitimate clients produce.

```

Accumulator scope:

- **One accumulator per UE World** (not one globally) — each World gets its own chain, so multi-world setups (PIE with multiple clients, editor + runtime) don't interfere.
- On World destruction, the accumulator chain is checkpointed: `checkpoint = HMAC(InternalAccum, "world-transition")`. The next World's initial accumulator incorporates this checkpoint, so chain history persists across World transitions.
- The accumulator is **not transmitted over the network** — it is a local integrity mechanism only. Multiplayer integrity is handled by Pillar 3.

9.4.4 Pillar 3 — Multiplayer Integrity Partition

Auto Replication (AR) creates a natural boundary for cross-instance verification: **other clients' Vaults verify your packets**. This is the one defense that a complete Vault replacement cannot defeat without also replacing every other player's Vault.

How it works:

SENDER (Client A → Server → Client B):
Client A's Vault serializes an AR replication update:
 payload = Serialize(ov_delta, salt = LDCC_OV_SerializeSalt)
 tag = HMAC-SHA256(LDCC_AR_PacketHMAC_Key, payload || timestamp || sender_id)
 packet = { payload, tag, sender_id, timestamp }

RECEIVER (Client B's Vault):
Client B's Vault receives the packet:
 expected_tag = HMAC-SHA256(LDCC_AR_PacketHMAC_Key, payload || timestamp || sender_id)
 if (tag ≠ expected_tag) → REJECT packet (silent drop – not logged)

Both clients derived LDCC_AR_PacketHMAC_Key from the SAME EpochKey_N
(both have valid licenses for the same epoch).

Communication matrix:

Scenario	Outcome
legit → legit	HMAC matches – packets pass – everything works
cracked → legit	HMAC wrong (no EpochKey) – packets rejected silently
legit → cracked	Cracked B can't verify (or blindly accepts) but B's responses have wrong HMAC → legit A rejects B
cracked→cracked	Works only if both use identical dummy keys → single dummy key = fingerprint-able by server → each crack distribution standardizes on one key → server detects and bans the pattern

Properties:

1. **Cracked clients are silently isolated.** No error message, no ban screen — packets are just dropped. The cracked player experiences "bad netcode" or "everyone is desynced." They cannot distinguish this from a genuine network problem.
2. **Legitimate players are never affected.** All legitimate Vaults derive the same LDCC_AR_PacketHMAC_Key from the same EpochKey → packets pass validation transparently.
3. **Server can participate but doesn't need to.** Even in pure peer-to-peer AR setups, each client's Vault independently verifies received packets. Dedicated servers can additionally flag suspicious traffic.
4. **The HMAC key rotates with epochs** (~90 days). A crack that hardcodes dummy keys stops working against legitimate clients when they advance to the next epoch.
5. **This pillar is AR-specific.** Single-player OV usage doesn't benefit from it — that's what Pillars 1, 2, 4, and 5 are for.

9.4.5 Pillar 4 — The Rot (Silent Behavioral Corruption)

Retained from the original design and strengthened. The Rot is the consequence of any Pillar 1-3 or Pillar 5 violation.

What triggers The Rot:

- Accumulator mismatch (Pillar 2 failure)
- LDCC derivation failure (no valid license → LDCCs are deterministic-but-wrong — Pillar 1)
- Proxy .text hash mismatch (Pillar 5 failure)
- Any anti-tamper gate failure (§5.0-5.3) that doesn't trigger immediate self-destruct
- Missing or corrupt GlobalKey.glic → EpochKey derivation fails → all LDCCs are wrong

The Rot is NOT immediate. That's the point. A clean error tells the attacker "your crack broke at line X." The Rot tells them nothing.

[THE ROT v2 – Improved Silent Corruption]

Phase 0 (0-30s after detection, ±15s randomized):
Everything works perfectly. No observable change.
Internally: corruption_seed = SHA-256(detection_source || μs_timestamp || AccumulatorState)

Phase 1 (30-90s, ±30s randomized):
OV computation begins using subtly perturbed LDCCs:

- LDCC_OV_SerializeSalt is XOR'd with corruption_seed[0:16]
- Effect: serialization of SOME OVs becomes lossy
- Manifests as: "corrupt save data", "values not persisting correctly"

AR replication becomes intermittent:

- LDCC_AR_FrameNonce is perturbed → ~10% of packets fail HMAC at receiver
- Manifests as: "laggy replication", "updates dropping occasionally"

Phase 2 (90-180s, ±60s randomized):

Corruption escalates via further LDCC perturbation:

- OV_NotifyJitter is scrambled → change notifications fire spuriously (random 5% per tick)
- AR_SequenceXOR is perturbed → packet sequence drift → reordering at ~30%
- OV_DirtyBitSalt is wrong → dirty tracking fails → 20% stale reads
- Manifests as: "variables randomly changing", "state desyncs", "impossible values"

Phase 3 (180s+, randomized onset):

Terminal degradation:

- All LDCCs are replaced with corruption_seed derivatives → total computation failure
- 50% of OV reads return stale cached values
- AR replication effectively stops (>70% packet loss)
- At a random point: ClearAllCachedFunctions() fires irreversibly
- Session is unrecoverable — full restart required

Timeline is RANDOMIZED per session using the corruption_seed.

No two sessions degrade identically. Crack testing becomes unreliable.

What makes The Rot v2 stronger than v1:

1. **Corruption is channeled through the LDCC system** — not through separate corruption code. The Rot doesn't add new corruption branches; it perturbs existing LDCC values. There is no `if (rotting) corrupt_value()` function to find and NOP — the corruption IS normal computation with wrong constants.
2. **Phase boundaries are wider (±15-60s)** — the attacker cannot reliably time the progression.
3. **The corruption is deterministic from the seed** — same seed → same pattern. But the seed includes a microsecond timestamp, so testing the same scenario twice produces different degradation timelines.
4. **Corrupted data propagates.** Once bad OV values reach SaveGame files, replicated state, or Blueprint variables, rolling back requires a clean start. The longer The Rot runs, the deeper the poison spreads.
5. **The attacker sees "bugs", not "DRM."** Subtle corruption looks like floating-point precision issues, race conditions, network jitter, or cache staleness. An attacker may spend days debugging their "crack" before realizing it's The Rot.

9.4.6 Pillar 5 — Proxy Binding

Evolved from the original V3 (ASH_TRAIL) concept, now integrated into the accumulator system rather than being a standalone demand variant.

On Vault initialization:

1. The Vault reads the **Proxy module's .text section** from memory (using platform PE/ELF header parsing on the loaded Proxy module) and computes its SHA-256 hash (`ProxyTextHash`).
2. This hash is **compared against expected hashes** baked into the Vault at build time (one per build configuration — Debug/Development/Shipping/Test).
3. The hash is **folded into the accumulator initialization**: `Accumulator_0 = HMAC(LDCC_Accumulator_IV, ProxyTextHash || VaultTextHash)`. A patched Proxy → different `ProxyTextHash` → different accumulator chain → diverges on round 1 → The Rot.
4. The hash is **re-verified periodically** (every 5-10 minutes, randomized interval). If the Proxy's `.text` hash changes mid-session (live patching / debugger hot-reload), The Rot is triggered.

Why this is hard to bypass:

- The Vault hashes the Proxy from OUTSIDE (cross-module memory read) — the Proxy cannot intercept or fake this.
- Even if the attacker patches the Vault's hash-checking code, `ProxyTextHash` is used as a VALUE in the accumulator HMAC — not just as a CHECK. Skipping the comparison doesn't fix the accumulator.
- The expected Proxy hashes are embedded in the Vault's `.rodata` (obfuscated + VM-protected). Patching them = patching the Vault = changing `VaultTextHash` = breaking triple-envelope crypto (Layer 2) = losing the master_seed = losing all LDCCs.
- The attacker cannot replace the Proxy with a modified version that "passes" the hash check — the hash IS the identity. Any modification, no matter how small, produces a completely different SHA-256.

9.4.7 The Complete Protocol — Lifecycle

THE GORDIAN PARASITE v2 — LIFECYCLE

PHASE 0: VAULT INITIALIZATION

1. VaultLoader loads DLL, verifies signature (§5.0)
2. GVault_ValidateIntegrity() — .text self-check (§5.2)
3. Vault reads Proxy .text hash (Pillar 5)
4. Vault derives master_seed from GlobalKey.glic (triple-envelope — §6.15)
5. Vault derives EpochKey_N from master_seed via hash chain (§6.14)
6. Vault derives ALL 12 LDCCs from EpochKey_N (Pillar 1)
7. Vault initializes Accumulator_0 = HMAC(LDCC_IV, ProxyTextHash || VaultTextHash)
8. Returns Accumulator_0 to Proxy via GVault_Initialize()

If license is missing/invalid/expired:

- LDCCs are derived from a deterministic but WRONG seed: SHA-256("unlicensed")
- No crash. No error dialog. No log message. The code RUNS.
- But every OV/AR operation uses wrong computation constants.
- The Rot begins at Phase 0 onset — everything looks fine initially.

→ Degradation sets in over 30-180 seconds (randomized).
 → The attacker has no "license check failed" message to search for.

PHASE 1: NORMAL OPERATION

Every Vault call follows the accumulator protocol:

Proxy calls: GVault_OV_*(args, Accumulator_prev)

Vault: Verify accumulator → Compute with LDCCs → Update accumulator → Return

AR send/receive additionally:

Send: Attach HMAC tag derived from LDCC_AR_PacketHMAC_Key (Pillar 3)

Receive: Verify sender's HMAC tag → reject if mismatched (silent drop)

PHASE 2: PERIODIC VERIFICATION (background, every 5-10 minutes, randomized)

Vault re-hashes its own .text section (\$5.3)

Vault re-hashes the Proxy's .text section (Pillar 5)

Vault verifies the accumulator chain is internally consistent

Any failure → The Rot (Pillar 4) – no immediate action, no log, no error

PHASE 3: EPOCH TRANSITION (every ~90 days)

EpochKey rotates → ALL LDCCs recompute → accumulator chain resets

The Vault retains the previous epoch's LDCCs for backward compatibility:

→ Old serialized data (SaveGames, persisted OV's) remains readable

→ New data is written with new LDCCs

Cracks with hardcoded constants stop working at the epoch boundary.

The attacker must re-crack every ~90 days – and each re-crack requires

extracting the new EpochKey from the triple-envelope-encrypted license.

9.4.8 Updated C-Linkage Exports for The Gordian Parasite v2

```
// — "The Gordian Parasite v2" — Computation-Entangled Verification ———
// (replaces ALL v1 interrogation-based exports)

// Initialize Vault and receive the initial accumulator value.
// Called once during Vault startup. Returns Accumulator_0 (32 bytes) to Proxy.
// The accumulator is opaque — the Proxy stores it and passes it back on every call.
VAULT_API int32 GVault_Initialize(GVaultOVHandle Self,
                                uint8* Accumulator0Out,
                                int32* Accumulator0LenOut);

// — Every OV/AR export signature gains accumulator in/out parameters ———
// Example (updated signature for SerializeToPayload):
//
// VAULT_API bool GVault_SerializeToPayload(GVaultOVHandle Self,
//                                          /* existing params */,
//                                          const uint8* AccumulatorIn,
//                                          uint8* AccumulatorOut);
//
// The same pattern applies to ALL GVault_OV_*, GVault_Mixin_*,
// GVault_Coordinator_*, GVault_Graph_*, GVault_RPCTransporter_*,
// GVault_Iris_*, and GVault_WorldSub_* exports.

// — AR Multiplayer Integrity (Pillar 3) ———

// Tag an outgoing AR replication packet with a license-derived HMAC.
VAULT_API int32 GVault_AR_TagPacket(const uint8* Payload, int32 PayloadLen,
                                   uint8* TagOut, int32* TagLenOut,
                                   const uint8* AccumulatorIn,
                                   uint8* AccumulatorOut);

// Verify an incoming AR replication packet's HMAC tag.
// Returns 0 if valid, non-zero if the tag doesn't match (silent reject).
VAULT_API int32 GVault_AR_VerifyPacket(const uint8* Payload, int32 PayloadLen,
                                       const uint8* Tag, int32 TagLen,
                                       const uint8* AccumulatorIn,
                                       uint8* AccumulatorOut);

// — Periodic Integrity Audit ———
// Called by the Proxy on a randomized timer (5-10 minutes).
// Triggers re-verification of .text hashes and accumulator consistency.
VAULT_API int32 GVault_PeriodicAudit(const uint8* AccumulatorIn,
                                     uint8* AccumulatorOut);

// — REMOVED from v1 (interrogation-based exports) ———
```

```
// GVault_Parasite_Demand          - REMOVED (no more Vault-asks-Proxy interrogation)
// GVault_Parasite_VerifyProof     - REMOVED (verification is implicit via accumulator)
// GVault_Parasite_IngestEntangled - REMOVED (accumulator replaces external chain feeding)
// GVault_Parasite_QueryVariant    - REMOVED (leaked variant selection to attacker - $9.0 finding)
//
// NOTE: GVault_ChallengeResponse is RETAINED for the traditional Proxy→Vault
// direction (prove the DLL is genuine). The Gordian Parasite v2 operates in
// addition to it, not instead of it.

// — Epoch & Settings Configuration (§6.27/§6.28) —————
// Called by the Proxy to pass Developer Settings to the Vault.
// Must be called at module startup AND before packaging.
VAULT_API void GVault_SetEpochConfig(int32 EditorEpochDays,
                                       int32 ShippingNoExpiration,
                                       int32 ShippingEpochDays,
                                       int32 VerboseLogging);
```

9.4.9 Attack Analysis — Why Each Approach Fails Against v2

Attack Approach	Why It Fails Against The Gordian Parasite v2
"I'll NOP the license check"	There is no single license check to NOP. The license state flows through 12 LDCCs consumed at dozens of call sites deep inside obfuscated + VM-protected code. NOP'ing the derivation → LDCCs default to zeros → every OV serialization, AR packet, and internal hash table uses wrong parameters. The code runs but produces garbage.
"I'll extract LDCC values and hardcode them"	LDCCs rotate with EpochKey every ~90 days. Hardcoded values expire. Finding all 12 LDCC consumption sites in VM-protected code is extremely time-consuming. 15-20 decoy constants waste analysis hours on red herrings. Even if all 12 are found, the crack expires in ≤90 days.
"I'll extract EpochKey and derive LDCCs myself"	EpochKey is derived from master_seed behind triple-envelope encryption (§6.15). Extracting it requires breaking HW-bound + Vault-binary-bound + temporal-bound encryption, all protected by White-Box AES (§6.5) + VM protection (§5.5). This IS the crown jewel and is the hardest target in the system.
"I'll binary-patch the Vault DLL"	Patching .text has two independent cryptographic consequences : (1) VaultTextHash changes → accumulator HMAC key changes → accumulator chain diverges → The Rot. (2) Vault_Key changes (triple-envelope Layer 2 — §6.15) → master_seed decryption fails → no EpochKey → all LDCCs wrong → silent corruption. Both fire simultaneously from a single patch.
"I'll reimplement the entire Vault"	~8,000 lines of complex, obfuscated C++ across OV serialization, AR replication, dynamic properties, graph routing, caching, and more. Even a perfect reimplementaion lacks: (a) the genuine VaultTextHash → accumulator chain wrong, (b) the genuine EpochKey → LDCCs wrong → network-incompatible with legitimate clients (Pillar 3). The reimplementaion barrier IS the primary defense for single-player.
"I'll intercept the Accumulator"	The accumulator is computed INSIDE the Vault using HMAC with VaultTextHash as key. The Proxy never knows the "right" value — it stores what the Vault returns opaquely. A MITM can read the bytes but can't forge them without VaultTextHash.
"Single-player — multiplayer doesn't apply"	Correct — Pillar 3 doesn't help. Single-player relies on Pillars 1 (LDCCs), 2 (accumulator), 4 (The Rot), and 5 (Proxy binding). The combined barrier: extract EpochKey (break triple-envelope) OR reimplement 8,000 lines of obfuscated code. Both are extremely high-effort for a plugin.
"I'll find The Rot and disable it"	The Rot doesn't exist as separate code. It perturbs LDCC values through the same code paths that normal computation uses. There is no <code>if (rotting) corrupt()</code> branch. The corruption IS the computation with wrong constants — same code path, different inputs.
"I'll share a genuine DLL + license"	License is bound to hardware fingerprint (§6.6) + ProjectID + Vault's .text hash (Layer 2). Moving DLL to another machine → HW mismatch → decryption fails. Recompiling → different .text hash → Layer 2 fails.
"I'll run genuine Vault in a VM"	Hardware fingerprint changes in VM → license decryption fails. Even with spoofing, Pillar 5 verifies the Proxy's .text — a VM-hosted proxy wrapper is a different binary → accumulator diverges.
"I'll run a cracked DLL in a real engine (old V0-V6 weakness)"	This attack no longer applies. v2 doesn't ask "are you in a real engine?" It doesn't ask questions at all. The license state IS the computation. Running a cracked DLL in a real engine doesn't help — the cracked DLL lacks EpochKey/LDCCs regardless of the engine's genuineness.

9.4.10 Summary — Defense Pillars Within The Gordian Parasite v2

Pillar 1: License-Derived Computation Constants (LDCCs)

- 12 active constants derived from EpochKey_N
- Consumed at dozens of call sites (serialization, replication, caching, hashing, notifications)
- 15-20 decoy constants to waste attacker analysis time
- Rotate every ~90 days with epoch transition → hardcoded cracks expire
- Extraction requires breaking triple-envelope encryption (§6.15)

Pillar 2: Opaque Accumulator Chain

- 32-byte HMAC chain threaded through every Vault API call
- HMAC key includes VaultTextHash → binary-specific → any .text patch = diverge
- Initialization includes ProxyTextHash → Proxy patches also cause diverge

- └─ Dynamic LDCC modulation (subset of LDCCs depend on accumulator state)
- └─ Per-World scope with persistent chain across World transitions

Pillar 3: Multiplayer Integrity Partition (AR-specific)

- └─ Every AR packet HMAC-tagged with LDCC_AR_PacketHMAC_Key
- └─ Legitimate ↔ legitimate: transparent – Cracked ↔ legitimate: silently rejected
- └─ Cracked clients experience "bad netcode" – no DRM error messages
- └─ HMAC key rotates with epoch → cracks expire for multiplayer every ~90 days
- └─ Server can additionally fingerprint dummy-key patterns for banning

Pillar 4: The Rot (Silent Behavioral Corruption)

- └─ Triggered by any Pillar 1/2/3/5 failure or anti-tamper gate failure
- └─ No clean failure, no error, no log – corruption manifests as "bugs"
- └─ Corruption channeled through LDCC perturbation (no separate code to NOP)
- └─ Randomized timeline (±15-60s per phase) → crack testing is unreliable
- └─ Terminal: ClearAllCachedFunctions() at random point → unrecoverable
- └─ Corrupted data propagates into saves/replicated state → deep poison

Pillar 5: Proxy Binding

- └─ Vault hashes the Proxy's .text section externally (cross-module read)
- └─ Hash folded into accumulator initialization → patched Proxy = chain diverges immediately
- └─ Periodic re-verification (5-10 min, randomized) catches live patching
- └─ Expected hashes baked into Vault (obfuscated + VM-protected)
- └─ Patching expected hashes = patching Vault = changing VaultTextHash = breaking everything

The Gordian Parasite v2 doesn't ask questions. It doesn't interrogate. It doesn't challenge. It simply IS the computation. The license key doesn't open a door — it IS the door, the walls, and the floor. Remove it and there's nothing left to stand on. Replace it and the building collapses differently every time. The only way to make it work is to have a valid license.

9.5 Scope Clarification — DRM with Serialization-Layer Anti-Cheat Properties

Updated declaration: The Gorgeous Vault is primarily a **Digital Rights Management (DRM) and licensing system**. However, its architecture produces a genuine **serialization-layer anti-cheat capability** as a structural consequence of the LDCC design — specifically for variables replicated through the OV/AR pipeline. This is NOT a general-purpose anti-cheat. It protects a specific, narrow domain (replicated OV state) with cryptographic strength that traditional anti-cheats cannot match in that domain.

9.5.1 What the Vault Protects Against (DRM + Serialization-Layer AC)

Threat	Protection	Source	Category
Plugin piracy (using paid plugins without a license)	LDCCs make the plugin non-functional without valid license	\$9.4.2 (Pillar 1)	DRM
DLL replacement (swapping Vault with cracked binary)	.text hash IS the crypto key — wrong DLL = wrong everything	\$5.2 + \$9.4.3	DRM
License forgery (keygen, fake server)	ECDSA-signed JWT, RSA public key baked in	\$6.6	DRM
License sharing (one license across multiple developers)	Team detection + per-seat enforcement	\$6.26	DRM
AR packet forgery (fabricated OV state)	HMAC-tagged packets via LDCC_AR_PacketHMAC_Key	\$9.4.4 (Pillar 3)	Serialization-Layer AC
AR packet injection from cracked clients	Cracked clients derive wrong HMAC key → packets silently rejected	\$9.4.4	Serialization-Layer AC
AR packet replay (re-sending old valid packets)	Accumulator chain includes packet sequence → replayed packets have stale accumulator state	\$9.4.3	Serialization-Layer AC
OV memory editing → replication (cheater modifies OV value in RAM, it replicates)	The HMAC covers the serialized payload. If the value was modified BEFORE serialization by an external tool, the HMAC is still valid (the Vault serialized the tampered value). However: if the attacker modifies the serialized packet AFTER the Vault produces it (network-layer injection), the HMAC is invalid. See §9.6 for full analysis.	\$9.4.4	Partial AC
Cross-client RCE via AR transport	Only valid-HMAC packets reach deserialization	\$5.7	Serialization-Layer AC

9.5.2 What the Vault Does NOT Protect Against

Threat	Why It's Out of Scope	What Should Be Used Instead
--------	-----------------------	-----------------------------

Threat	Why It's Out of Scope	What Should Be Used Instead
Memory editing of non-OV game state	The Vault protects Vault-internal memory only. Game state not stored in Object Variables is outside the Vault's scope.	EAC, BattlEye, or server-authoritative validation
Memory editing of OV values pre-serialization	If a cheater modifies an OV value in RAM before the Vault serializes it, the Vault will faithfully HMAC-tag the tampered value and replicate it. The Vault proves WHO sent the data (a legitimate client), not WHETHER the data is valid gameplay.	Server-authoritative game logic validation (see §9.6.4 for the Tiered Integrity model)
Aimbots	Input-layer and rendering-layer manipulation. The Vault has zero visibility into player input or rendering.	Kernel-level anti-cheat (EAC/BattlEye) + behavioral analysis
Wallhacks	Rendering-layer exploit. The Vault doesn't touch the rendering pipeline.	Kernel-level anti-cheat + server-side occlusion culling
Speed hacks	Game tick manipulation. The Vault doesn't control the game loop or tick rate.	Server-authoritative tick validation
Input injection (hardware/driver-level)	Below the application layer. The Vault runs in user-mode and cannot see hardware-level input spoofing.	Kernel-level anti-cheat
Bot automation	Scripted gameplay. Not a DRM concern.	Behavioral analysis + CAPTCHA-like challenges
Exploits in non-OV game systems	The Vault protects the OV/AR pipeline ONLY. Any game system that doesn't use Object Variables or Auto Replication is completely outside the Vault's scope.	Game-specific validation logic

9.5.3 Recommendation for Developers

THE VAULT = YOUR DRM + SERIALIZATION-LAYER ANTI-CHEAT FOR OV/AR

For multiplayer games, you SHOULD use the Vault's AR packet integrity as ONE layer of your anti-cheat strategy. It is NOT sufficient alone for general anti-cheat, but it IS the STRONGEST layer you have for protecting the integrity of replicated OV state.

Recommended stack (see §9.6.4 "Tiered Integrity" for full model):

- Layer 1: Gorgeous Vault (DRM + serialization-layer AC for OV/AR)
- Layer 2: EAC / BattlEye / custom kernel anti-cheat (anti-tamper, anti-debug, memory scanning, behavioral analysis)
- Layer 3: Server-authoritative game logic (validate all state changes, reject impossible values, rate-limit suspicious activity)
- Layer 4: Server-side behavioral analytics (detect statistical anomalies, flag unusual win rates, movement patterns, reaction times)

The Vault protects your LICENSING and your OV/AR TRANSPORT. Input cheats, rendering cheats, and non-OV state are your responsibility.

9.6 Serialization-Layer Anti-Cheat — Deep Analysis

The question this section answers: Can license-derived, epoch-rotating HMAC keys on the serialization layer function as an effective anti-cheat for replicated game state? Has anyone done this before? Could it be "unbreakable"?

9.6.1 How Traditional Anti-Cheats Work (and Where They Fail)

Traditional anti-cheat systems operate at layers that are fundamentally **observable and bypassable** by sufficiently skilled attackers:

TRADITIONAL ANTI-CHEAT ARCHITECTURE

Layer 5 (highest): Server-side validation

- Authoritative state computation
- Anomaly detection / statistics
- ✓ STRONG – attacker can't tamper with server
 - X Only works if server computes ALL state (expensive)
 - X Latency-sensitive games can't server-validate everything

Layer 4:

Behavioral analysis

- Statistical anomaly detection (win rate, reaction time)
- Heuristic pattern matching
- ✓ Catches obvious cheaters over time
 - X Slow – needs data accumulation

X False positives on skilled players

Layer 3:

Kernel-level scanning (EAC, BattlEye, Vanguard)

- Memory scanning for known cheat signatures
- Driver integrity verification
- Anti-debug, anti-VM detection
- ✓ Catches commodity cheats (known tools)
 - X Cat-and-mouse: new tools bypass old signatures
 - X Kernel-level cheats can hide from kernel AC
 - X DMA cheats (hardware) bypass entirely

Layer 2:

Client-side integrity

- Hash checks on game binaries
- CRC validation on game state
- ✓ Catches naive modification
 - X All checks run on untrusted client
 - X Attacker patches the check itself

Layer 1 (lowest):

Network transport

- Usually: TLS encryption (protects against MITM)
- Sometimes: sequencing/nonce (prevents replay)
- X NO per-variable integrity
 - X NO cryptographic proof of data origin
 - X Encrypting garbage is still garbage

The critical gap: Layer 1 (network transport) in traditional systems provides CONFIDENTIALITY (TLS) but NOT per-variable INTEGRITY. A cheater who modifies game state in memory before serialization sends perfectly valid, encrypted packets containing tampered data. No traditional anti-cheat detects this at the network layer — they rely on Layers 3-5 to catch it after the fact.

9.6.2 What Makes the Vault's Approach Different (and Novel)

The Gorgeous Vault operates at Layer 1 (serialization/transport) but adds something no traditional anti-cheat does: **per-variable cryptographic integrity with a license-derived, epoch-rotating key.**

GORGEOUS VAULT SERIALIZATION-LAYER AC

What happens when an OV variable is replicated:

1. Game code sets OV value: Health->SetValue(100)

2. AR system decides to replicate

3. Vault serializes: payload = Serialize(Health, 100)

4. Vault HMAC-tags: tag = HMAC-SHA256(LDCC_AR_PacketHMAC_Key, payload || seq || timestamp)

5. Vault sends: {payload, tag, seq, timestamp}

6. Receiver Vault verifies: HMAC matches? → accept. Doesn't match? → silent drop.

The HMAC key (LDCC_AR_PacketHMAC_Key) is:

- Derived from EpochKey via HKDF
- EpochKey is derived from master_seed via epoch chain
- master_seed is triple-envelope encrypted (HW + Vault + temporal)
- The derivation path is VM-protected (§5.5.1)

To forge a valid HMAC tag, an attacker must:

a. Extract LDCC_AR_PacketHMAC_Key from the running Vault

- requires de-virtualizing VM-protected code
- requires locating the key after WB-AES decryption
- requires breaking into the triple-envelope chain

b. OR extract EpochKey and derive the HMAC key themselves

- requires breaking all three envelope layers

c. OR replace the Vault DLL with one that uses a known key

- VaultTextHash changes → all LDCCs change → packets rejected by every other client

9.6.3 Industry Comparison — Has Anyone Done This Before?

System	Approach	Per-Variable Integrity?	Key Rotation?	Tied to License?
Gorgeous Vault (OV/AR)	HMAC-SHA256 per serialized variable, key derived from license epoch chain	YES	YES (epoch rotation, configurable 5–36500 days)	YES (LDCC from license)
EAC (Easy Anti-Cheat)	Kernel driver + encrypted transport + server-side validation	No — transport is encrypted but not per-variable signed	No	No
BattlEye	Kernel driver + behavioral + encrypted transport	No	No	No

85 / 98

System	Approach	Per-Variable Integrity?	Key Rotation?	Tied to License?
Riot Vanguard	Kernel driver (boot-time) + memory scanning + encrypted transport	No — relies on catching memory modification, not signing transport	No	No
VAC (Valve Anti-Cheat)	Signature scanning + heuristic behavioral	No — no transport integrity at all	No	No
Denuvo Anti-Cheat	Kernel driver + integrity scanning	No — protects binary integrity, not per-variable transport	No	Partially (tied to Denuvo DRM)
Fortnite (custom)	Server-authoritative model + EAC + proprietary systems	Partially — server validates all state changes	N/A (server-side)	No
Overwatch (custom)	Server-authoritative + client-side input validation + proprietary	No — relies on server authority, not client-side signing	N/A	No

Key finding: No existing anti-cheat system provides per-variable cryptographic integrity at the serialization layer with a rotating, license-derived key. The Gorgeous Vault's approach is genuinely novel in this specific domain.

The closest analogy is **TLS client certificates** — each client proves identity, and the server accepts only authenticated connections. But TLS operates at the TRANSPORT layer (encrypting the stream), not the APPLICATION layer (signing individual variables). The Vault signs at the variable level.

9.6.4 The Tiered Integrity Model — Vault + Traditional AC

The user's insight (and it's correct): The Vault is excellent for anti-cheat on a SMALL NUMBER of high-value, low-frequency variables. But it has overhead (HMAC computation + tag bytes per packet + DLL boundary crossing) that makes it unsuitable for variables that change every frame. The optimal strategy is a TIERED model.

TIERED INTEGRITY MODEL

Tier 1: "Vault-Guarded" — Gorgeous Vault HMAC (cryptographic, per-variable)

WHAT: 5-50 critical low-frequency OV variables

EXAMPLES: Health, MaxHealth, Currency, Score, MatchResult, Inventory slots, Permission flags, Ability cooldowns, Team assignment, Player rank, Event triggers

WHY VAULT: These variables are HIGH VALUE (cheating them breaks the game economy) and LOW FREQUENCY (they don't change every frame — health changes on damage events, currency on transactions, etc.). The ~50-200µs HMAC overhead per replication event is negligible for event-driven state.

PROTECTION LEVEL: Cryptographic. A cheater cannot forge a valid HMAC tag without the LDCC_AR_PackethMAC_Key. If the epoch rotates every 5-30 days (\$6.27), any extracted key expires fast. This is the STRONGEST possible protection for replicated state.

Tier 2: "AC-Guarded" — Traditional anti-cheat (heuristic, memory-level)

WHAT: All other game state (including high-frequency vars)

EXAMPLES: Position (changes every frame), Rotation, Velocity, Animation state, Input state, Camera position

WHY TRADITIONAL AC: These variables change at 30-120 Hz. HMAC-tagging every position update would add measurable latency and bandwidth. Traditional AC (EAC/BattlEye) catches memory edits to these variables through scanning + heuristics.

PROTECTION LEVEL: Heuristic. Cat-and-mouse. New cheats bypass old signatures. But acceptable for variables where the gameplay impact of a brief exploit window is recoverable.

Tier 3: "Server-Guarded" — Server-authoritative validation

WHAT: Any state change the server can independently verify

EXAMPLES: Purchase transactions, damage application (if server-authoritative), physics simulation results, matchmaking state, anti-speedhack tick validation

WHY SERVER: The server is the ONLY trustworthy party. Anything the server can recompute should be recomputed server-side.

PROTECTION LEVEL: Absolute (for what the server validates).

THE GORGEOUS COMBINATION:

Position, Rotation, Velocity

→ Tier 2 (EAC/BattlEye)

→ ~0μs client overhead

Health, Currency, Score, Rank

→ Tier 1 (Gorgeous Vault)

→ ~50-200μs per event

Purchases, Match Results

→ Tier 3 (Server)

→ ~0μs client overhead

WHY THIS IS DEVASTATING FOR CHEATERS:

• Tier 2 (traditional AC) catches commodity cheats: aimbots, wallhacks, speed hacks

• Tier 1 (Vault) makes the MOST VALUABLE cheats (god mode, infinite currency, score manipulation) cryptographically impossible without extracting a key that rotates on the developer's schedule

• Tier 3 (server) provides the ground truth for transactions and outcomes

• A cheater must simultaneously defeat ALL THREE tiers to have meaningful impact

9.6.5 Could This Be "Unbreakable"?

Short answer: No. Nothing is unbreakable. But the Vault's serialization-layer AC has a unique property that makes it **economically unsustainable** to cheat against.

The key insight: cost-per-rotation.

Traditional anti-cheat is a one-time bypass problem. Once a cheater finds a way past EAC/BattlEye, it works until the AC vendor pushes an update. The cheater's cost is FIXED — one bypass = indefinite cheating until detected.

The Vault's HMAC key rotates with the epoch. If the developer sets epoch to 5 days:

COST ANALYSIS: CHEATING AGAINST VAULT HMAC (5-DAY EPOCH)

Initial crack effort:

De-virtualize LDCC derivation:

2-4 weeks (one-time)

Locate WB-AES tables:

1-2 weeks (one-time)

Build extraction toolkit:

1-2 weeks (one-time)

Total initial investment:

4-8 weeks

Per-rotation effort:

Wait for new game update to drop

Dump new Vault binary

Extract new LDCC_AR_PacketHMAC_Key

~1-4 hours (with existing toolkit)

Distribute to cheat users

~1 hour

Per-rotation cost:

~2-5 hours every 5 days

Annual cheating cost:

Initial investment:

4-8 weeks

Ongoing: 73 rotations × ~3.5 hrs

= 255 hours ≈ 32 work days

Total first-year cost:

~2-4 months of skilled RE time

Compare to traditional AC bypass:

Initial bypass:

1-4 weeks

Ongoing:

~0 until AC vendor pushes update (monthly?)

Per-update re-bypass:

~1-3 days

Annual total:

~2-6 weeks

THE VAULT IS 4-8× MORE EXPENSIVE TO CHEAT AGAINST PER YEAR

when epoch rotation is aggressive (5-30 days).

Can the extraction be fully automated? Partially. If the VM ISA doesn't change between releases, the attacker can script the extraction. But \$5.5.1 recommends rotating the VM ISA per major Vault release, which breaks automation. If the Vault ships ~4 major releases per year with new VM ISAs, the extraction toolkit must be rebuilt each time — multiplying the annual cost.

9.6.6 The "Pre-Serialization" Gap — And How the OV Integrity Canary Closes It

The Vault has one fundamental limitation as anti-cheat: **it signs what the OV system gives it, not what the game SHOULD give it.**

THE PRE-SERIALIZATION GAP (WITHOUT CANARY)

Normal flow:

Game logic: TakeDamage(50) → Health OV: 100 → 50 → Vault serializes 50 → HMAC(50) ✓

Cheater flow (memory edit BEFORE Vault):

Cheat tool: Write(Health_OV, 999) → Vault serializes 999 → HMAC(999) ✓

↑

Valid HMAC! The Vault faithfully

signed the tampered value because
it was already tampered when the
Vault saw it.

The Vault proves: "This packet was created by a legitimate, unmodified Vault DLL
on a client with a valid license."

The Vault does NOT prove: "The value 999 is a legitimate gameplay result."

Without the OV Integrity Canary (§9.6.8), this gap remains open and Tier 2 (traditional AC) + Tier 3 (server validation) are the only defense against pre-serialization memory edits.

With the OV Integrity Canary enabled, this gap is **closed** for all OV variables — see §9.6.8 for the full design. The Vault detects that the value was modified outside the `SetValue()` API and rejects the tampered value before it reaches serialization.

Note: If the Vault's anti-tamper (§5.2, §5.4, §9.4.3) catches the memory-editing tool itself, the accumulator diverges → The Rot fires → the cheater's entire Vault instance degrades. The Canary and the anti-tamper are complementary: anti-tamper catches the TOOL, the Canary catches the RESULT.

9.6.7 Summary — Where the Vault's AC Fits

THE GORGEOUS VAULT: SERIALIZATION-LAYER ANTI-CHEAT

✓ STRONG against:

- Network-layer packet injection/forgery (HMAC rejects)
- Cracked client → legitimate client communication (wrong key)
- Replay attacks (accumulator sequence)
- RCE via AR transport (only HMAC-valid packets deserialize)
- Key extraction long-term (epoch rotation = recurring cost)
- Memory editing of OV values [WITH CANARY ENABLED – §9.6.8]
(canary detects writes that bypass SetValue API → rejects tampered values before serialization. Requires bEnableOVIntegrityCanary.)
- SLT stripping / zeroing [SLT PRESENCE SENTINEL – §7.4.6]
(sentinel feeds into accumulator seed → stripping poisons all state)
- Partial-crack attempts [KEY SPREAD – §7.5]
(each plugin = independent SLT → crack cost scales linearly with N)

~ PARTIAL against (WITHOUT canary):

- Memory editing of OV values (Vault signs what it sees – but anti-tamper + accumulator may catch the editing tool itself)

X DOES NOT address:

- Input-layer cheats (aimbots, input injection)
- Rendering-layer cheats (wallhacks)
- Game tick manipulation (speed hacks)
- Non-OV game state

CONFIGURABLE RESPONSE (baked into SLT – §6.28):

Mode 0: The Rot – silent behavioral corruption (default deterrent)
Mode 1: Kick + Server Callback – immediate disconnect + server notified
Mode 2: Rot + Delayed Callback – silent corruption THEN server notified
The response mode is frozen in the SLT at packaging time.
A cheater CANNOT downgrade from mode 2 to mode 0 to hide from server.

ANTI-CHEAT SCALING PROPERTY:

Each additional Gorgeous plugin adds an independent SLT compartment.
Core alone = 1 compartment (baseline). + Events = 2× crack cost.
+ Entertaining = 3× crack cost. Each plugin N is exactly as hard to crack as plugin 1 – keys are independently derived.
This is a UNIQUE ecosystem property: more plugins = stronger AC.

BEST USED AS: The cryptographic backbone of a Tiered Integrity model (§9.6.4) where the Vault guards high-value low-frequency variables, traditional AC guards the rest, and the server validates everything it can.

NOVEL PROPERTY: No existing anti-cheat provides per-variable cryptographic integrity with license-derived, epoch-rotating keys at the serialization layer, compartmentalized per-plugin with independent key derivation. This is a genuinely new contribution to the anti-cheat landscape.

IS IT UNBREAKABLE? No. But it makes cheating a RECURRING COST rather than a one-time investment – and that changes the economics fundamentally.

This section describes a compile-time-gated anti-cheat mechanism that detects memory edits to OV values made outside the legitimate `SetValue()` API. When enabled via Developer Settings (\$6.28), the canary system is baked into the Shipping License Token (\$7.4) and the Vault binary. When disabled, the code path does not exist in the Shipping build — there is nothing to toggle, NOP, or patch.

9.6.8.1 Core Mechanism — Per-Variable HMAC Canary

The Vault OWNS the OV value storage — every Object Variable's actual value lives inside the Vault's managed memory. This means the Vault can **guard** each value with a hidden HMAC canary that is updated only through the legitimate API.

OV INTEGRITY CANARY — HOW IT WORKS

For each OV variable instance managed by the Vault:

Internal state (hidden, inside Vault memory — never exposed to game code):

value:	<the actual OV value bytes>
canary:	uint8[32] — HMAC of value + metadata
set_counter:	uint64 — monotonic per-variable
last_set_tick:	uint64 — GFrameCounter at last set

ON `SetValue(newVal)` — called through the legitimate OV API:

```
1. value = newVal
2. set_counter++
3. last_set_tick = GFrameCounter
4. canary = HMAC-SHA256(
    key = LDCC_OV_CanaryKey,
    msg = variable_identity // unique ID: owner + property name + instance
        || value_bytes      // the actual serialized value
        || set_counter      // prevents replay of old canaries
        || last_set_tick    // ties to specific frame
)
5. Return to caller — overhead: ~1-5µs for the HMAC
```

ON Serialization (before AR replication — inside Vault):

```
1. Recompute expected canary from current memory state:
   expected = HMAC-SHA256(
       key = LDCC_OV_CanaryKey,
       msg = variable_identity || value_bytes || set_counter || last_set_tick
   )
2. Compare expected vs. stored canary
3. IF match → value is clean → proceed with serialization + AR HMAC tagging
4. IF mismatch → VALUE WAS TAMPERED OUTSIDE THE API
   ↳ The value in memory differs from what SetValue() last wrote
   ↳ OR the set_counter/tick was manipulated
   ↳ ACTION: Reject this variable from the replication batch
     ↳ The tampered value is NOT serialized, NOT HMAC-tagged, NOT sent
     ↳ Other clean variables in the same batch replicate normally
   ↳ Increment internal tamper_strike_counter for this variable
   ↳ IF tamper_strike_counter >= 3 within a rolling 60-second window:
     ↳ Trigger The Rot ($9.4.5) — silent behavioral corruption begins
     ↳ This prevents brute-force "keep trying until it works" attacks
   ↳ The cheater sees: their health change "didn't replicate" — no error,
     no ban screen, just... nothing happened. Other players still see
     the old value. The cheater cannot distinguish this from a network
     hiccup or replication priority issue.
```

9.6.8.2 Why the Canary Key Must Be License-Derived

The canary HMAC key (`LDCC_OV_CanaryKey`) is derived from `EpochKey_N` like all other LDCCs:

```
LDCC_OV_CanaryKey = HMAC-SHA256(EpochKey_N, "ldcc:ov-canary-key")[:32]
```

A cheater who wants to bypass the canary must:

1. **Find the canary verification code** — it's VM-protected (\$5.5.1) and interleaved with normal serialization logic. There is no separate "canary check" function to NOP.
2. **AND** extract `LDCC_OV_CanaryKey` — same difficulty as extracting any other LDCC (requires breaking triple-envelope → `EpochKey` → HKDF).
3. **AND** update the canary after their memory write — which means calling the HMAC internally, which means knowing the key, the `variable_identity` format, the `set_counter`, and the `last_set_tick`. All of which are inside obfuscated Vault memory.

If the attacker can do all of this, they can compute valid canaries for their tampered values. But at that point they've effectively reverse-engineered the entire LDCC derivation chain — the same effort as a full crack (6-16 weeks, \$9.6.5). And the canary key rotates with the epoch.

9.6.8.3 The Build-Time Gate — Why Cheaters Can't Toggle This

The user's critical insight: This flag must control the BUILD, not a runtime toggle. If it's a runtime flag, a cheater flips it to `false` and the canary is disabled.

How the build-time gate works:

BUILD-TIME GATE — NOT A RUNTIME TOGGLE

DEVELOPER'S MACHINE (packaging time):

- Developer sets `bEnableOVIntegrityCanary = true` in Project Settings
- Value is saved to `DefaultGorgeousCore.ini`
- At packaging time, Proxy calls `GVault_SetEpochConfig()` which includes the canary flag
- The Vault generates the SLT with `slt_flags.ov_integrity_canary = true`
- The Vault ALSO derives `LDCC_OV_CanaryKey` and includes it in the SLT
- The SLT (with canary key + flag) is AES-256-GCM encrypted and baked into the Vault DLL's `.data` section

SHIPPING BUILD (player's machine):

- Vault boots → reads SLT from its own `.data` section
- Decrypts SLT → reads `slt_flags.ov_integrity_canary = true`
- Extracts `LDCC_OV_CanaryKey` from the SLT
- Activates canary code path for all OV variables

WHY A CHEATER CAN'T DISABLE IT:

- The flag is INSIDE the AES-256-GCM encrypted SLT
- Modifying the flag → GCM authentication tag fails → SLT decryption fails → ALL LDCCs are lost → OV/AR completely non-functional
- The cheater can't selectively disable the canary without breaking the entire SLT — and breaking the SLT breaks EVERYTHING
- Even if they could modify the flag, the canary verification code is INTERLEAVED with normal serialization in VM-protected code. There is no clean `"if (canary_enabled) check()"` branch to NOP. The canary computation IS the serialization preamble.

WHEN DISABLED (`bEnableOVIntegrityCanary = false`):

- The SLT does NOT contain `LDCC_OV_CanaryKey`
- The SLT flag says `ov_integrity_canary = false`
- The Vault's canary code path is NEVER ACTIVATED
- There is no dormant code to enable — the key doesn't exist in the binary
- Zero overhead, zero attack surface from the canary system

9.6.8.4 Overhead Analysis

Operation	Cost	When
<code>SetValue()</code> canary computation	~1-5µs (one HMAC-SHA256)	Every time an OV value changes
Serialization canary verification	~1-5µs (one HMAC-SHA256)	Every time an OV value is replicated
Per-variable storage overhead	42 bytes (32 canary + 8 counter + 2 tick reference)	Constant per OV variable

Realistic game scenarios:

Scenario	OV Variables	Change Rate	Canary Overhead/Frame	Verdict
RPG with health/mana/gold	~20	~2-5 changes/sec	~10-50µs/frame	Negligible
Battle royale with H/S/A/I	~50	~10-20 changes/sec	~50-200µs/frame	Acceptable
Fast-paced FPS, ALL vars OV	~200	~100+ changes/sec	~0.5-2ms/frame	Noticeable — consider enabling only for Tier 1 variables
Physics sim, position as OV	~500+	~500+ changes/sec (60Hz × entities)	~2-10ms/frame	Too high — don't canary physics-rate variables

Recommendation: Enable the canary for games where the protected OV variables are event-driven (health, currency, score, cooldowns, permissions). Do NOT enable it if your OV design puts high-frequency per-frame state (position, rotation, velocity) into Object Variables destined for replication. For those, rely on Tier 2 (traditional AC) and Tier 3 (server-side validation).

9.6.8.5 Attack Analysis — Canary-Specific

Attack	Result
"I'll write directly to the OV's memory address"	Canary mismatch on next serialization → tampered value is silently dropped from replication. After 3 strikes → The Rot activates.
"I'll find and update the canary too"	The canary is in Vault-internal memory (obfuscated layout, ASLR). Finding the canary bytes for a specific variable requires mapping the Vault's entire internal OV storage structure. Even if found, computing a valid canary requires LDCC_OV_CanaryKey — extraction difficulty same as any LDCC (6-16 weeks).
"I'll hook SetValue() and call it with my tampered value"	This PASSES the canary — the cheater called the legitimate API. However: this is detectable through other means: (a) set_counter increments at unexpected rate → statistical anomaly, (b) the Vault can enforce call-site verification — SetValue() must come from known code addresses in the game module's .text section (not from injected DLLs), (c) server-side Tier 3 validation catches impossible values.
"I'll NOP the canary check"	The canary verification is NOT a separate branch. It's interleaved with the serialization preamble inside VM-protected code. NOP'ing it would corrupt the serialization data layout → deserialization fails on the receiving end.
"I'll disable the flag in the config .ini file"	The .ini file is meaningless in Shipping builds. The canary flag is in the SLT, which is AES-GCM encrypted inside the Vault binary. The .ini is only read at packaging time on the developer's machine.
"I'll run without the canary (stock build) against canary-enabled clients"	All clients in the same game session use the same SLT (same packaged binary). A player cannot bring a "different" Vault binary without triggering VaultTextHash mismatch → different LDCCs → HMAC rejection by other clients (Pillar 3).

9.6.8.6 Integration with the Tiered Integrity Model

With the canary enabled, the Tiered Integrity Model (§9.6.4) upgrades:

UPDATED TIERED INTEGRITY MODEL (WITH CANARY)

Tier 1: "Vault-Guarded + Canary" – cryptographic per-variable AC

Protection: Serialization HMAC (Pillar 3)
+ OV Integrity Canary (§9.6.8)
+ Epoch key rotation

Result: Memory edits → caught by canary → value not replicated
Network injection → caught by HMAC → packet rejected
Cracked client → wrong LDCCs → everything rejected

THE PRE-SERIALIZATION GAP IS CLOSED.
For Tier 1 OV variables, the Vault now provides COMPLETE protection: from memory to wire to receiver.

The only remaining attack: hook SetValue() and call the API with the tampered value (see §9.6.8.5 – mitigated by call-site verification + server-side anomaly detection + rate limiting).

Tier 2: Unchanged – EAC/BattlEye for non-OV state

Tier 3: Unchanged – Server-authoritative validation

9.6.8.7 Updated C-Linkage Export

```
// — Anti-Cheat Configuration (§6.28, §6.29, §9.6.8) —————
// Passes ALL licensing + AC settings to the Vault.
// Called by the Proxy as part of the epoch config handoff.

VAULT_API void GVault_SetEpochConfig(
    int32 EditorEpochDays,
    int32 ShippingNoExpiration,
    int32 ShippingEpochDays,
    int32 VerboseLogging,
    int32 EnableAntiCheat,           // §6.29 – 1 = fetch free AC license for Core-only projects
    int32 AntiCheatResponseMode,    // 0 = Rot, 1 = Kick+Callback, 2 = Rot+DelayedCallback
    int32 EnableOVIntegrityCanary   // §9.6.8 – 1 = bake canary into SLT, 0 = omit
);

// — Anti-Cheat Server Callback (§6.28) —————
// Delegate for server-side notification when cheating is detected.
// Bound in GorgeousCoreRuntime – server game mode subscribes to this delegate.
//
// This is NOT a Vault export – it's a UE delegate declared in open-source code.
```

```
// The Vault calls GVault_Internal_NotifyCheatDetected() which the Proxy
// translates into this delegate broadcast.

DECLARE_DYNAMIC_MULTICAST_DELEGATE_ThreeParams(
    FOnAntiCheatViolation,
    APlayerController*, ViolatingPlayer,
    EGorgeousACViolationType, ViolationType,    // CanaryMismatch, SLTSentinelFail, AccumulatorDivergence
    const FString&, Details                    // Human-readable description for logging
);

// The server's GameMode (or any server-side actor) subscribes:
// UGorgeousAntiCheatSubsystem::Get()->OnAntiCheatViolation.AddDynamic(this, &AMyGameMode::HandleCheat);
```

10. Open Questions & Risks

Open Questions

#	Question	Impact
1	Does Fab allow license-gated functionality in SIBLING plugins?	GorgeousCore is free — no issue there. But the sibling plugins' Fab listings need to disclose that a license (validated by GorgeousCore's Vault) is required. Must verify with Fab team.
2	How to handle UE version upgrades?	The Vault links against UE import libs — each UE version may need a separate Vault build. Need a version matrix.
3	How to handle the Vault for packaged (Shipping) builds?	RESOLVED — \$7.4 Shipping License Embedding. The Vault bakes a Shipping License Token (SLT) into the binary at packaging time. No license file needed at runtime. Epoch lifetime configurable via Developer Settings (\$6.27/\$6.28).
4	White-box crypto vs. standard AES?	Standard AES with the key in obfuscated memory is "good enough" for most attackers. White-box crypto is stronger but complex to implement.
5	Should the Vault self-host a license server or use a third-party service?	Options: custom server, Gumroad/LemonSqueezy API, Keygen.sh. Trade-off: control vs. maintenance burden.
6	What about Linux/Mac code signing?	The integrity hash works without code signing, but OS-level code signing adds another layer.
7	How does plugin discovery work reliably?	The Vault scans for .uplugin files with GorgeousCore dependency. What if a plugin is installed but not enabled? What about engine-level vs. project-level plugins? Need robust discovery logic.
8	Is cascading death too aggressive?	Users may resent losing ALL functionality due to one plugin's license issue. Mitigation: clear messaging identifying the offending plugin, easy in-editor resolution, and a short grace period before enforcement.

Risks

Risk	Likelihood	Impact	Mitigation
Fab rejects sibling plugins due to external licensing requirement	Medium	Critical	Pre-submit inquiry to Fab review team. GorgeousCore itself is free — only sibling plugins require licensing.
A sophisticated cracker reverse-engineers the Vault	Low	High	Accept that nothing is 100% secure. The goal is to make cracking harder than buying a license. Continuously update obfuscation.
License server downtime blocks legitimate users	Medium	High	Auto-fetched 365-day offline key for all online users. Multiple server regions. Manual offline key entry as last resort.
UE engine update breaks the Vault DLL	High	Medium	Maintain a CI pipeline that builds the Vault against each supported UE version. Ship per-version binaries.
Anti-tamper false positive on a legitimate machine	Low	Critical	Extensive QA across diverse hardware. Bug bounty for false positive reports. Instant hotfix pipeline.
Users rebelling against cascading death enforcement	Medium	High	Transparent communication. Clear messaging identifying offending plugin. Easy in-editor resolution. Recovery without restart. Consider a short grace period (e.g., 24 hours of degraded mode before full cascading death).
Users rebelling against DRM-like protection	Medium	Medium	GorgeousCore is free — no DRM. Only sibling plugins require licensing. Transparent communication. License should be hassle-free. No always-online requirement.

Summary

This plan creates a **multi-layered protection system** where the Object Variable — the most valuable intellectual property — lives inside a precompiled, obfuscated, tamper-resistant DLL (the "Vault"). The open-source GorgeousCore module serves as a transparent proxy that forwards calls into the Vault.

GorgeousCore itself is permanently free and never licensed. It ships on Fab as a free plugin. The Vault's licensing engine validates licenses for **sibling plugins** (GorgeousEntertaining, GorgeousEvents, etc.) that build on top of GorgeousCore. Each sibling plugin requires its own per-project, per-plugin license key.

Cascading death enforcement: If any installed sibling plugin has an invalid or missing license, the Vault clears the entire OV/AR function reference table, killing functionality for ALL plugins — including those with valid licenses. This all-or-nothing approach prevents partial piracy and creates strong economic pressure to keep all licenses valid.

The licensing system uses a **two-tier key hierarchy** (global → per-plugin intermediate) with **project-and-plugin-bound AES encryption**, making license files non-transferable. Server-side tracking enables piracy detection without invasive client-side DRM.

All of this is designed to comply with **Fab marketplace guidelines** by treating the Vault as a precompiled third-party library (a well-established pattern), keeping GorgeousCore free, requiring explicit consent for network calls, and ensuring legitimate customers never experience friction.

The core principle: make cracking harder than buying.

11. Implementation Progress

READ BEFORE EDITING — AI Agent Policy

When modifying code tracked by this plan, AI agents **MUST** follow these rules:

- NEVER add deprecation notices.** Do not mark functions, classes, or exports as `@deprecated`, `[[deprecated]]`, or equivalent. If something is being replaced, **remove it immediately** in the same changeset.
- NEVER leave dead code behind.** If a function, class, or export is superseded, delete the old declaration, definition, and all references (including comments, plan entries, and export tables) in the same operation.
- Update this plan atomically.** When removing or adding exports/files, update the directory tree, export summary tables, export counts, and status rows in the same edit — do not leave stale numbers or phantom entries.
- No transition periods.** There is no "deprecated but still present" state. Old → New is a single atomic step.

11.1 Phase 1 Skeleton — COMPLETED (2026-02-27)

The ThirdParty module structure has been created under `GorgeousCore/Source/ThirdParty/GorgeousVault/`.

Directory Structure

```
Source/ThirdParty/GorgeousVault/
├── .gitignore
├── CMakeLists.txt                ← Standalone CMake build (targets UE 5.4+)
├── GorgeousVault.Build.cs        ← UE ThirdParty external module (ModuleType.External)
├── BuildScripts/
│   ├── BuildVault_Win64.ps1      ← PowerShell build (VS2022/Ninja, auto-detect UE)
│   └── BuildVault_Unix.sh        ← Bash build (Linux + macOS, Ninja/Make)
├── Include/
│   ├── GorgeousVaultPlatform.h   ← Platform macros, VAULT_API, opaque handles, result codes
│   └── GorgeousVaultAPI.h        ← Full C-linkage export table (~95 exports)
├── Loader/
│   ├── GorgeousVaultLoader.h     ← Singleton DLL loader (open source, ships with proxy)
│   └── GorgeousVaultLoader.cpp    ← FPlatformProcess::GetDllHandle, function resolution, integrity, cascading death
├── Source/
│   ├── VaultEntryPoint.cpp        ← DLL lifecycle (DllMain, GVault_Initialize, GVault_Shutdown)
│   ├── VaultAntiTamper.cpp        ← Integrity hash, self-check (stub)
│   ├── VaultLicense.cpp           ← License system shared state (SharedSecret, init/shutdown)
│   ├── VaultChallengeResponse.cpp ← HMAC challenge-response, epoch counter (stub)
│   ├── LicenseHelper/
│   │   └── GorgeousLicenseHelper_Impl.cpp ← Plugin License Registry, per-plugin validation, cascading death
│   ├── ObjectVariables/
│   │   ├── GorgeousObjectVariable_Impl.cpp      ← Core OV lifecycle, serialization, replication
│   │   ├── GorgeousRootObjectVariable_Impl.cpp  ← Root OV registry management
│   │   ├── GorgeousObjectVariableTrunk_Impl.cpp  ← Trunk serialization
│   │   ├── GorgeousObjectVariableRegistry_GIS_Impl.cpp ← Registry subsystem
│   │   ├── NativeObjectVariableDefinitions_Impl.cpp ← Type system placeholder
│   │   └── GorgeousObjectVariableCmdletHandler_Impl.cpp ← Cmdlet handler
│   └── AutoReplication/
│       ├── GorgeousAutoReplicationMixin_Impl.cpp
│       ├── GorgeousAutoReplicationCoordinator_Impl.cpp
│       ├── GorgeousAutoReplicationGraph_Impl.cpp
│       ├── GorgeousAutoReplicationIrisBackend_Impl.cpp
│       ├── GorgeousAutoReplicationRPCTransporter_Impl.cpp
│       ├── GorgeousAutoReplicationRPCRelayComponent_Impl.cpp
│       ├── GorgeousAutoReplicationRPCRequestAsyncAction_Impl.cpp
│       ├── GorgeousAutoReplicationWorldSubsystem_Impl.cpp
│       ├── GorgeousAutoReplicationNetworkingTypes_Impl.cpp
│       ├── GorgeousAutoReplicationTypes_Impl.cpp
│       ├── GorgeousAutoReplicationStatsCollector_Impl.cpp
│       ├── GorgeousAutoReplicationSettings_Impl.cpp
│       ├── GorgeousReplicationBeacon_Impl.cpp
│       ├── ObjectVariables/
│       │   └── GorgeousRPC_OV_Impl.cpp
│       └── BlueprintFunctionLibraries/
│           └── GorgeousAutoReplicationNetworkingLibrary_Impl.cpp
```

```
├── GorgeousAutoReplicationRPCExamples_Impl.cpp
├── GorgeousAutoReplicationRPCPayloadLibrary_Impl.cpp
└── Prebuilt/
    ├── Win64/.gitkeep
    ├── Linux/.gitkeep
    └── Mac/.gitkeep
```

What Was Created

Component	File(s)	Status
CMake build system	CMakeLists.txt	☒ Skeleton — builds shared lib, links UE import libs, supports obfuscation flags
C-linkage API	Include/GorgeousVaultAPI.h, GorgeousVaultPlatform.h	☒ ~95 exports covering OV lifecycle, serialization, replication, mixin, coordinator, graph, transporter, relay, Iris, world subsystem, anti-tamper, sibling plugin licensing (per-plugin + registry + cascading death), license helper file ops, challenge-response
UE ThirdParty module	GorgeousVault.Build.cs	☒ External module — deploys DLL via RuntimeDependencies, adds include paths, defines GORGEOUSCORE_WITH_VAULT=1
VaultLoader singleton	Loader/GorgeousVaultLoader.h/.cpp	☒ Full implementation — DLL loading, function caching, integrity verification, self-destruct
DLL entry point	Source/VaultEntryPoint.cpp	☒ DllMain + GVault_Initialize/Shutdown lifecycle
Anti-tamper stub	Source/VaultAntiTamper.cpp	☐ Stub — .text hash verification logic not yet implemented
License system state	Source/VaultLicense.cpp	☒ SharedSecret, license system init/shutdown
License Helper impl	Source/LicenseHelper/GorgeousLicenseHelper_Impl.cpp	☒ Plugin License Registry (64 slots), per-plugin validation, cascading death evaluation, registry snapshot
Build scripts	BuildScripts/BuildVault_Win64.ps1, BuildVault_Unix.sh	☒ Cross-platform CMake invocation with auto-detect, flags, parallel build
Challenge-response stub	Source/VaultChallengeResponse.cpp	☐ Stub — HMAC computation not yet implemented
OV implementation stubs	Source/ObjectVariables/*.cpp (6 files)	☐ Stubs returning GVault_Error_NotImplemented
AutoReplication stubs	Source/AutoReplication/*.cpp (17 files)	☐ Stubs returning GVault_Error_NotImplemented
Prebuilt directories	Prebuilt/Win64,Linux,Mac/	☒ Created with .gitkeep placeholders
Build.cs integration	GorgeousCoreRuntime.Build.cs	☒ Added GorgeousVault dependency + Loader include path

C-Linkage Export Summary

Category	Export Count	Key Exports
Vault Lifecycle	3	GVault_Initialize, GVault_Shutdown, GVault_GetVersion
OV Core Lifecycle	5	GVault_OV_InvokeInstancedFunctionality, GVault_OV_BeginDestroy, GVault_OV_RegisterWithRegistry, GVault_OV_SetParent, GVault_OV_ApplyReplicatedIdentifier
OV Serialization	2	GVault_OV_SerializeToPayload, GVault_OV_DeserializeFromPayload
OV Replication	10	GVault_OV_ActivateReplication, GVault_OV_RegisterReplicatedProperty, GVault_OV_BindRPCHandler, GVault_OV_RequestAutoReplicationRPC, etc.
OV Network Stack	5	GVault_OV_SetRootNetworkStackEnabled, GVault_OV_SetSharedNetworkStackEnabled, etc.
Root OV	10	GVault_Root_GetRootObjectVariable, GVault_Root_FindVariableByIdentifier, etc.
Trunk	3	GVault_Trunk_WriteObjectToPayload, GVault_Trunk_LoadObjectFromPayload, GVault_Trunk_ComputePayloadHash
Registry	2	GVault_Registry_Initialize, GVault_Registry_Deinitialize
Mixin	8	GVault_Mixin_Bind, GVault_Mixin_RequestRPC, etc.
Coordinator	7	GVault_Coordinator_Get, GVault_Coordinator_RegisterVariable, etc.

Category	Export Count	Key Exports
Graph	8	GVault_Graph_InitGlobalGraphNodeNodes, GVault_Graph_RegisterObjectVariable, etc.
RPC Transporter	3	GVault_RPCTransporter_Initialize, GVault_RPCTransporter_RouteRPC, etc.
Iris Backend	6	GVault_Iris_Initialize, GVault_Iris_RegisterStream, etc.
World Subsystem	2	GVault_WorldSub_Initialize, GVault_WorldSub_Deinitialize
RPC Relay	4	GVault_Relay_RelayResultToServer, etc.
Anti-Tamper	1	GVault_ValidateIntegrity
License Helper File Ops	4	GVault_License_CreateEncryptedFile, GVault_License_ReadAndDecryptFile, GVault_License_AddEntry, GVault_License_ReadAllEntries
Sibling Plugin Registry	3	GVault_License_RegisterSiblingPlugins, GVault_License_GetPluginCount, GVault_License_GetPluginStatus
Per-Plugin Validation	2	GVault_License_ValidatePlugin, GVault_License_ValidatePluginOffline
Registry & Cascading Death	3	GVault_License_EvaluateRegistry, GVault_License_IsCascadingDeathActive, GVault_License_GetRegistrySnapshot
Challenge-Response	3	GVault_ChallengeResponse, GVault_VerifyChallengeEcho, GVault_Canary_SerializeCRC
Total	~95	

11.2 Phase 2 — License Helper Integration & Build Scripts (2026-02-28)

The FGorgeousCoreLicenseHelper has been drawn into the Vault. The open-source helper will become a thin proxy that forwards through C-linkage calls. Build scripts for all platforms have been created.

What Was Created / Modified

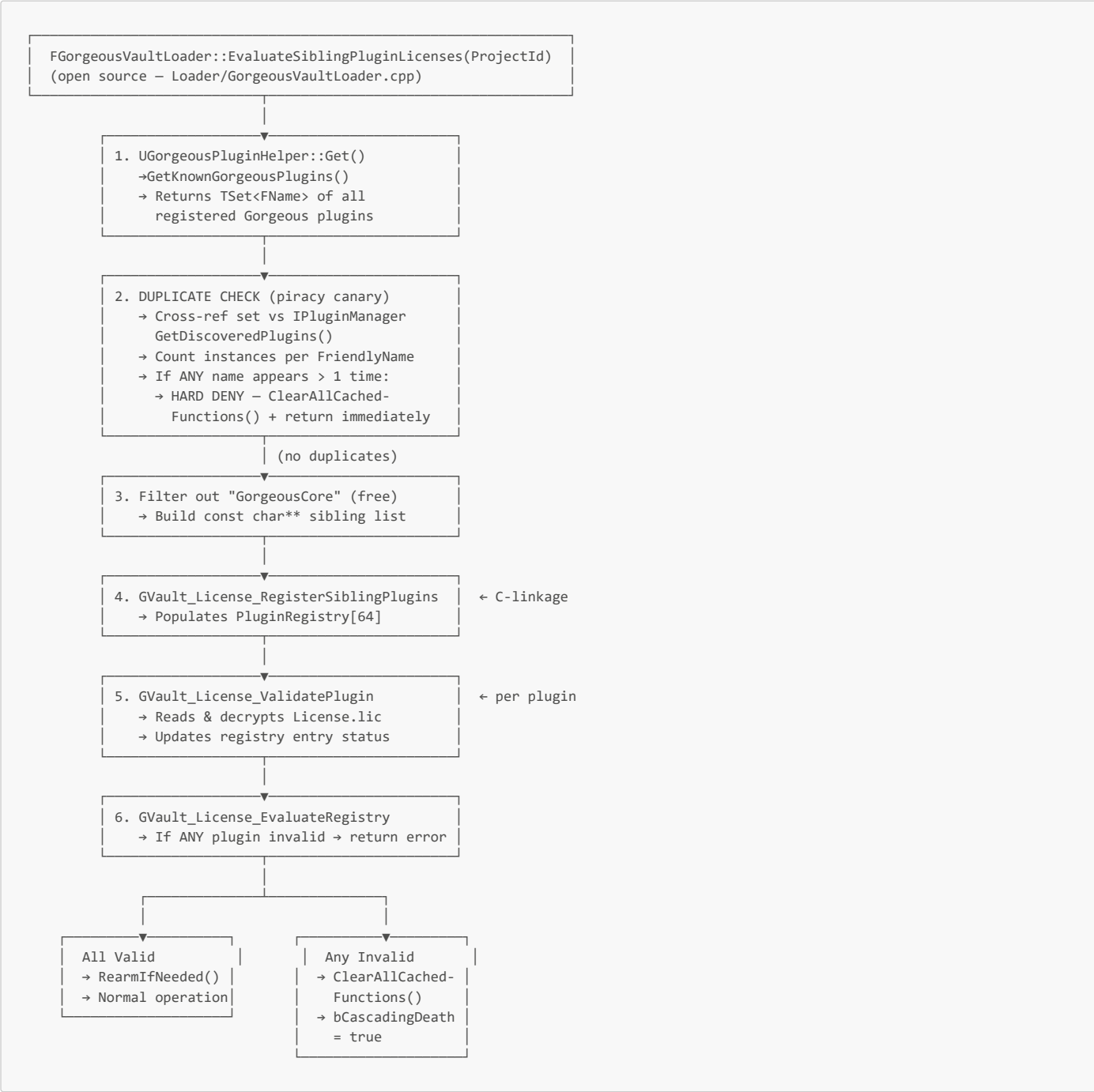
Component	File(s)	Status
License Helper Vault impl	Source/LicenseHelper/GorgeousLicenseHelper_Impl.cpp	☑ Plugin License Registry (64-slot fixed array), per-plugin validation stubs, cascading death evaluation, registry snapshot API
C-linkage API expansion	Include/GorgeousVaultAPI.h	☑ Added 12 new exports across 4 sections: License Helper File Operations (4), Sibling Plugin Discovery & Registry (3), Per-Plugin Validation (2), Registry Evaluation & Cascading Death (3). Removed old GVault_ValidateLicense/GVault_ValidateOfflineLicense. Total exports now ~95.
CMake update	CMakeLists.txt	☑ Added Source/LicenseHelper/GorgeousLicenseHelper_Impl.cpp to VAULT_SOURCES
VaultLoader — Cascading Death	Loader/GorgeousVaultLoader.h/.cpp	☑ Added ClearAllCachedFunctions() (wipes cache, keeps DLL loaded), RearmFunctionCache() (re-enables after license resolution), EvaluateSiblingPluginLicenses(ProjectId) (full evaluation flow using UGorgeousPluginHelper), bCascadingDeath flag
VaultLicense cleanup	Source/VaultLicense.cpp	☑ Stripped to shared state only (SharedSecret, init/shutdown). Old global validation functions removed.
Build script — Win64	BuildScripts/BuildVault_Win64.ps1	☑ PowerShell — auto-detects UE dir, VS2022/Ninja generator, obfuscation/anti-tamper flags, copies DLL to Prebuilt/Win64/
Build script — Linux/Mac	BuildScripts/BuildVault_Unix.sh	☑ Bash — auto-detects UE dir, Ninja/Make generator, arm64 default on Mac, parallel build

New C-Linkage Exports (Phase 2)

Category	Export Count	Key Exports
License Helper File Ops	4	GVault_License_CreateEncryptedFile, GVault_License_ReadAndDecryptFile, GVault_License_AddEntry, GVault_License_ReadAllEntries
Sibling Plugin Registry	3	GVault_License_RegisterSiblingPlugins, GVault_License_GetPluginCount, GVault_License_GetPluginStatus
Per-Plugin Validation	2	GVault_License_ValidatePlugin, GVault_License_ValidatePluginOffline

Category	Export Count	Key Exports
Registry & Cascading Death	3	GVault_License_EvaluateRegistry, GVault_License_IsCascadingDeathActive, GVault_License_GetRegistrySnapshot
Phase 2 Total	12	

Sibling Plugin Discovery Flow



Why duplicate detection? A pirate could install the same plugin at multiple paths (project-level + engine-level) to bypass per-path license binding. If a Gorgeous plugin name appears more than once across all discovered plugins, this is treated as a tampered installation — the operation is denied immediately, no license files are read, and cascading death fires.

Build Script Usage

Windows (PowerShell):

```
cd GorgeousCore/Source/ThirdParty/GorgeousVault/BuildScripts
.\BuildVault_Win64.ps1 -UEDir "D:\UnrealEngine\UE_5.4"
.\BuildVault_Win64.ps1 -UEDir "D:\UnrealEngine\UE_5.4" -Config Release -Obfuscation -Clean
```


Linux / macOS (bash):

```
cd GorgeousCore/Source/ThirdParty/GorgeousVault/BuildScripts
chmod +x BuildVault_Unix.sh
./BuildVault_Unix.sh --ue-dir /opt/UnrealEngine/UE_5.4
./BuildVault_Unix.sh --ue-dir /opt/UnrealEngine/UE_5.4 --config Release --obfuscation --clean
```

11.3 Phase 2a — GorgeousCoreRuntimeUtilities Build-Chain Integration

The Vault CMake build chain now includes **GorgeousCoreRuntimeUtilities** headers, giving Vault source files access to **GT_I_LOG** and all other header-only helpers/macros.

Key Design Decision: **WITH_EDITOR=0**

The entire GT_I_LOG macro chain (**GorgeousLoggingHelperMacros.h** → **GorgeousLoggingHelper.h** → globals, enums, structs) is **FORCEINLINE / header-only** — except for one function:

```
GORGEOUSCORERUNTIMEUTILITIES_API FGorgeousLogEntryDelegate& GetGorgeousLogEntryDelegate();
```

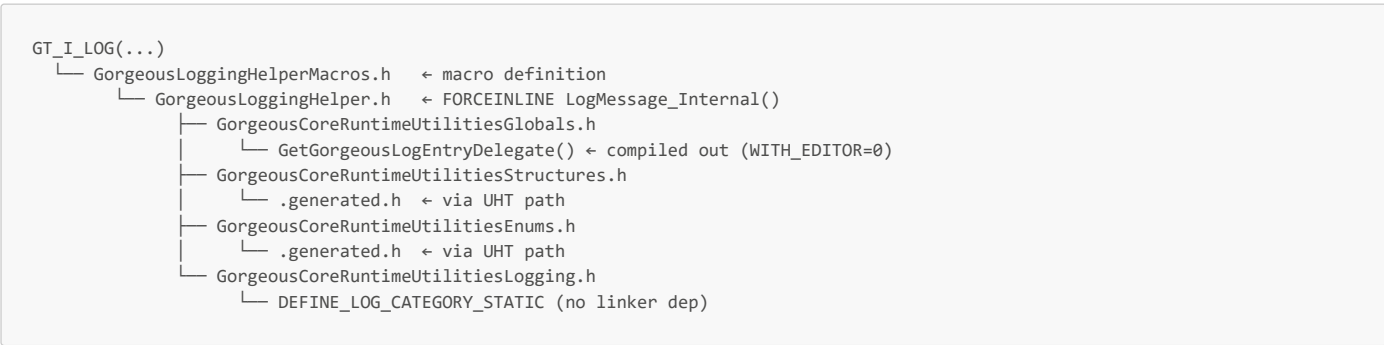
This is the **only** non-inline exported symbol in the logging chain. It is called exclusively inside **#if WITH_EDITOR** (for broadcasting log entries to the editor message log panel). UBT does **not** generate import libraries (**.lib**) for plugin modules — only **.dll** and **.pdb** — so there is nothing to link against.

Solution: Set **WITH_EDITOR=0** in the Vault's compile definitions. This compiles out the sole external symbol reference. Core logging (**UE_LOG**, on-screen debug messages) remains fully functional. **UE_EDITOR=1** is kept because the Vault links against **UnrealEditor-*.lib** engine import libraries and must match their binary layout.

What Was Modified

Component	File(s)	Change
Compile definitions	CMakeLists.txt	WITH_EDITOR changed from 1 → 0 with comment explaining the rationale
Include directories	CMakeLists.txt	Added GorgeousCoreRuntimeUtilities/Public/{ModuleCore,Helpers,Helpers/Macros,Libraries,Templates}, GorgeousCoreRuntime/Public/ModuleCore
UHT generated headers	CMakeLists.txt	Added Intermediate/Build/\${GVAULT_PLATFORM}/UnrealEditor/Inc/{GorgeousCoreRuntimeUtilities,GorgeousCoreRuntime}/U (requires UBT build first)

Include Chain Available to Vault Sources



11.4 Next Steps

Step	Phase	Description	Priority
1	Phase 1	Migrate real OV implementation into GorgeousObjectVariable_Impl.cpp	Critical
2	Phase 1	Migrate AutoReplication implementations into Vault stubs	Critical
3	Phase 2	Convert GorgeousObjectVariable.cpp into a thin proxy using FGorgeousVaultLoader	Critical
4	Phase 2	Convert FGorgeousCoreLicenseHelper.cpp into a thin proxy forwarding to GVault_License_*	Critical
5	Phase 1	Successfully build the Vault DLL via CMake against UE 5.4	High
6	Phase 2	Convert remaining .cpp files into proxies (Root, Trunk, Mixin, etc.)	High
7	Phase 3	Implement .text section hash embedding (post-link script)	High

Step	Phase	Description	Priority
8	Phase 3	Implement HMAC-SHA256 challenge-response	:yellow_circle: High
9	Phase 3	Implement runtime integrity self-check	:yellow_circle: High
10	Phase 4	Implement per-plugin AES-256-GCM license file decryption (<code>HKDF(Secret, ProjectID PluginName)</code>)	:orange_circle: Medium
11	Phase 4	Implement JWT validation with baked-in public key	:orange_circle: Medium
12	Phase 4	Build license server API (with per-plugin key issuance)	:orange_circle: Medium
13	Phase 5	Set up CI pipeline for multi-platform Vault builds	:orange_circle: Medium
14	Phase 5	LLVM obfuscation pass integration	 Low (initially)